Contents

Deep Learning Mein MNIST Ka Use Kab Karte Hain?

MNIST ko use karte hain:

1. **Deep Learning Seekhne ke Liye:**
   - Beginners ke liye perfect hai — simple hai, fast train hota hai
   - CNNs (Convolutional Neural Networks) practice karne ke liye use hota hai
2. **Model Testing ke Liye:**
   - Agar aapne koi naya neural network banaya hai, toh pehle usko MNIST pe test karte hain
3. **Hyperparameter Tuning:**
   - Learning rate, batch size, epochs, etc. test karne ke liye MNIST best hai
4. **Research Papers/Demo Projects:**
   - Research mein naye methods ko MNIST pe benchmark kiya jata hai

2. Decoder kya hai?

- Decoder ka kaam hota hai **output generate karna**.
- Ye encoder ke diye huye representation ko use karta hai aur apna khud ka **self-attention** bhi apply karta hai.
- Step-by-step output sequence banata hai.

🔧 Example:

- Encoder se mila representation = "I love Pakistan"
- Decoder agar English → French translation kar raha hai, toh step by step output generate karega:
  - "J'"
  - "aime"
  - "le Pakistan"

Yani decoder encoder ke "notes" ko use karke naya "sentence" likhta hai.

Agar aapko **dusri languages** mein translation karni ho, toh aapko sirf **model_name change** karna hoga.
Examples:

- `"Helsinki-NLP/opus-mt-en-de"` → English → German
- `"Helsinki-NLP/opus-mt-en-fr"` → English → French
- `"Helsinki-NLP/opus-mt-en-hi"` → English → Hindi
- `"Helsinki-NLP/opus-mt-ur-en"` → Urdu → English

1. HuggingFace Basics

🔑 HuggingFace ek **open-source library** hai jo **pretrained NLP models** provide karti hai.
Sabse popular library: 🤗 **Transformers**

Important components:

1. **Model Hub** – jagah jaha 100k+ pretrained models available hote hain.
   👉 https://huggingface.co/models ↗
2. **Transformers library** – Python package jo models ko load aur use karne ke liye functions deta hai.
3. **AutoTokenizer** – text ko tokens (numbers) me convert karta hai.
4. **AutoModel / AutoModelFor...** – pretrained model load karne ke liye.
   - `AutoModelForSequenceClassification` → classification tasks
   - `AutoModelForCausalLM` → text generation
   - `TFAutoModel...` → TensorFlow version

2. BERT for Text Classification

BERT kya hai?

- **BERT (Bidirectional Encoder Representations from Transformers)**
- Google ka model jo text ke **context** ko samajhne ke liye train hai.
- Input sentence ko tokens me todta hai aur context-based embeddings banata hai.

Text Classification use-case:

Example:

- Sentiment Analysis (Positive / Negative)
- Spam Detection (Spam / Ham)
- Topic Classification

Chapter 1.

Intro to Deep Learning

Deep Learning (DL) asal mai Machine Learning ka aik hissa hai. Is mai hum **Artificial Neural Networks (ANNs)** use kartay hain jo insaan ke **dimaagh (brain)** ke neurons se inspired hotay hain.

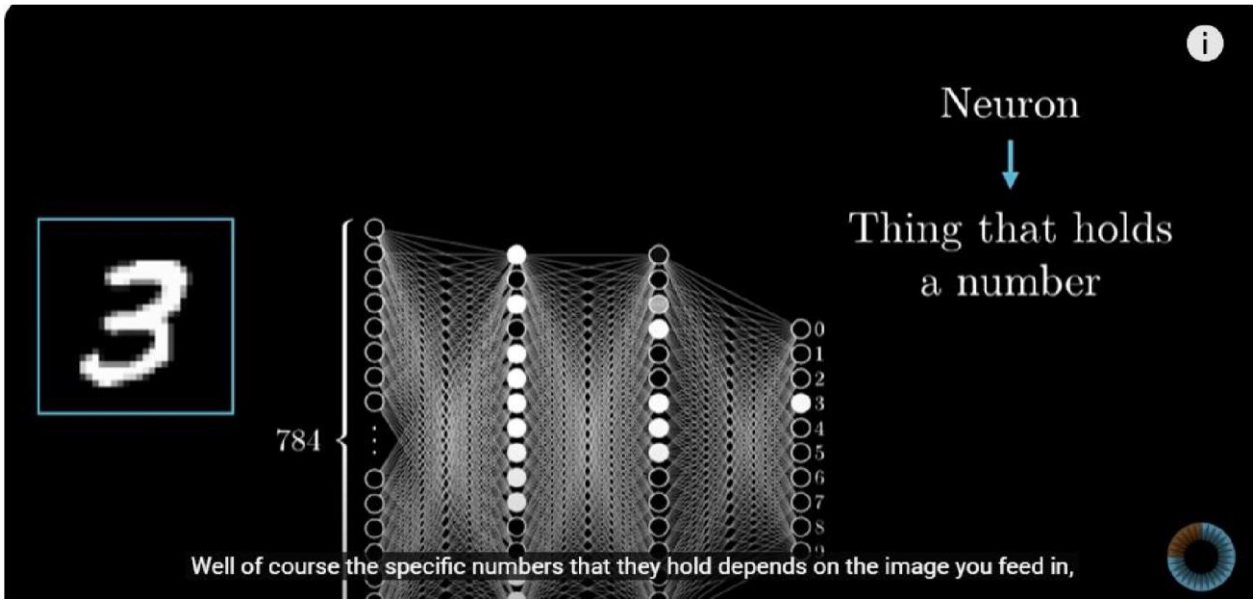
☞ Simple zuban mai:

- **ML** = Computer ko rules sikhaana bina explicitly rules likhnay ke.
- **DL** = Bohat bade aur complex data ko process karna using layered neural networks.

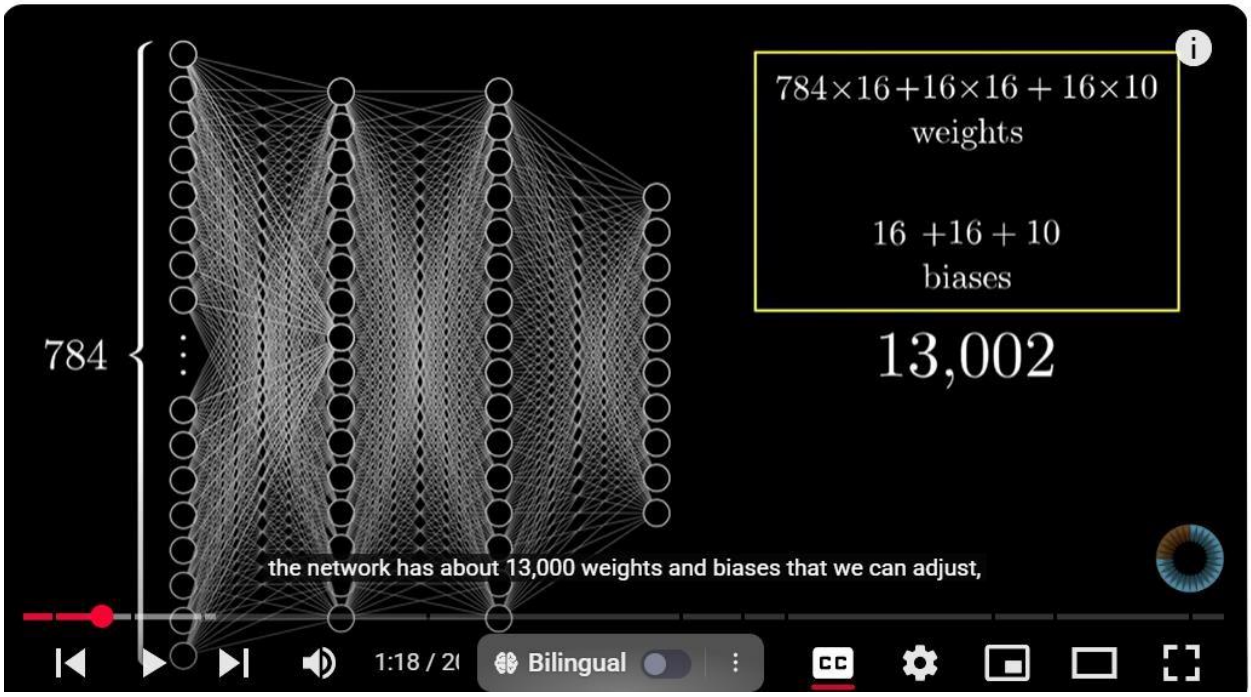
Real-life example:

- Tum Netflix pr movie dekhti ho, aur next tumhein recommendations milti hain → yeh Deep Learning model ki wajah se hota hai jo tumhari pasand seekh leta hai.

🔷 2. Neural Network Basics



Basically neuron aik network hai jo input mai 784 numbers laita hai or out put mai just 10 numbers daita hai.



Yaha hm nai input liya hai 784 neuron ka uskai bad hm nai 16,16 pixels ki 2 hidden layers bna di hai first layer edge fnd karai gi second pattern ko, hm nai bias yaha 10 rakha hai to yai kul 13002 wights or biases hai is oper k porai nwtwok mai jo just aik pattern ko identiy kr raha hai… hr neuron previus neuron sai connect hota hai

a) Perceptron

Perceptron aik **basic neuron** hai jo input leta hai, usay process karta hai aur aik output deta hai.

Formula:

Output = Activation(Σ(weights × inputs) + bias) ☞

Tum isay **decision making unit** samajh lo.

Real-life example:

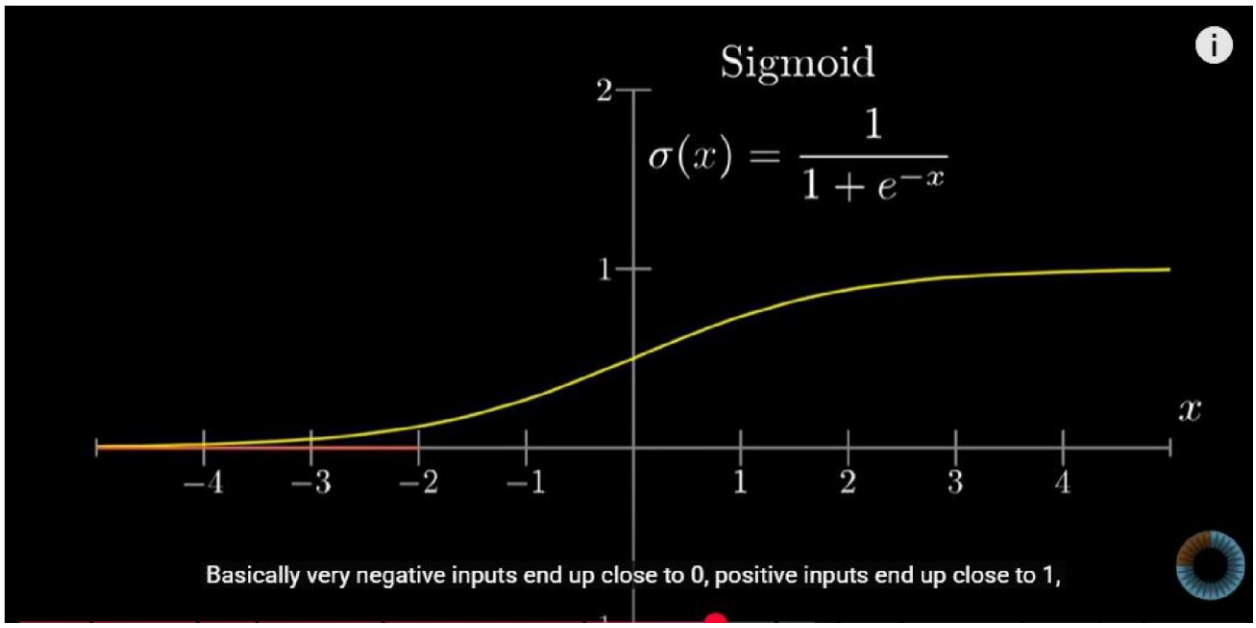
- Socho tum bazar ja rahi ho aur decide karna hai ke **barish mai chhatri le kar jao ya nahi**. ○ Input 1: Aasmaan mai badal hain (1 ya 0)
  - Input 2: Mausam app barish ka chance dikha rahi hai (percentage) ○ Perceptron in dono inputs ko mila kar decision deta hai → "Chhatri le lo" ya "Nahi le kar jao".

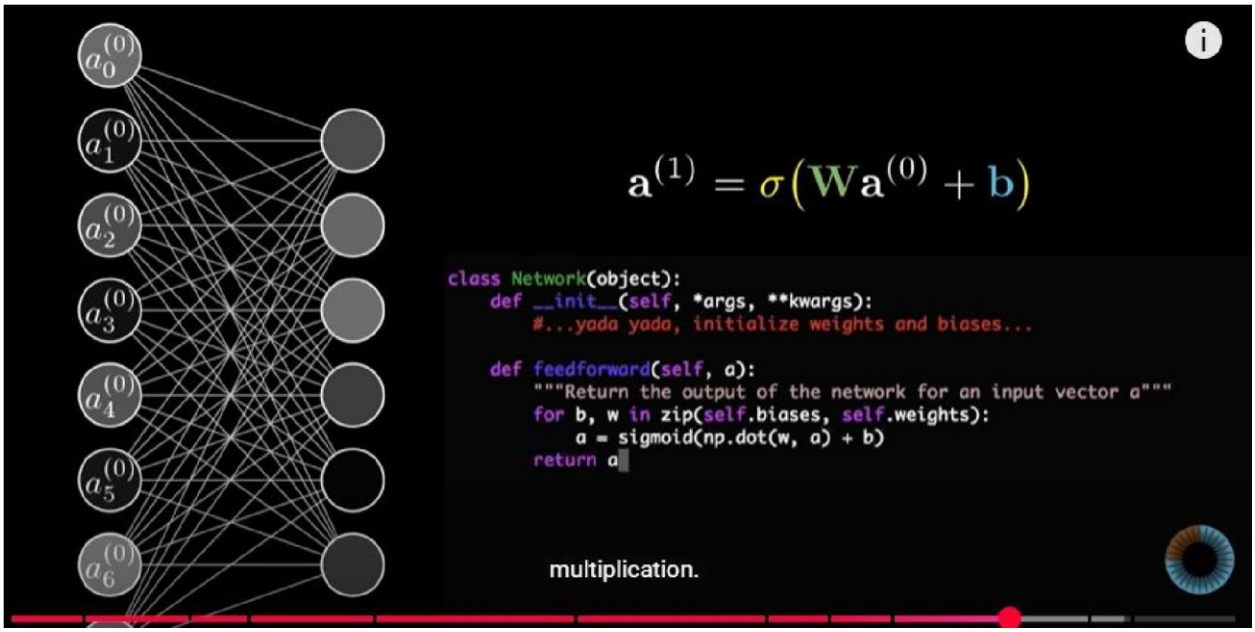
b) Activation Functions

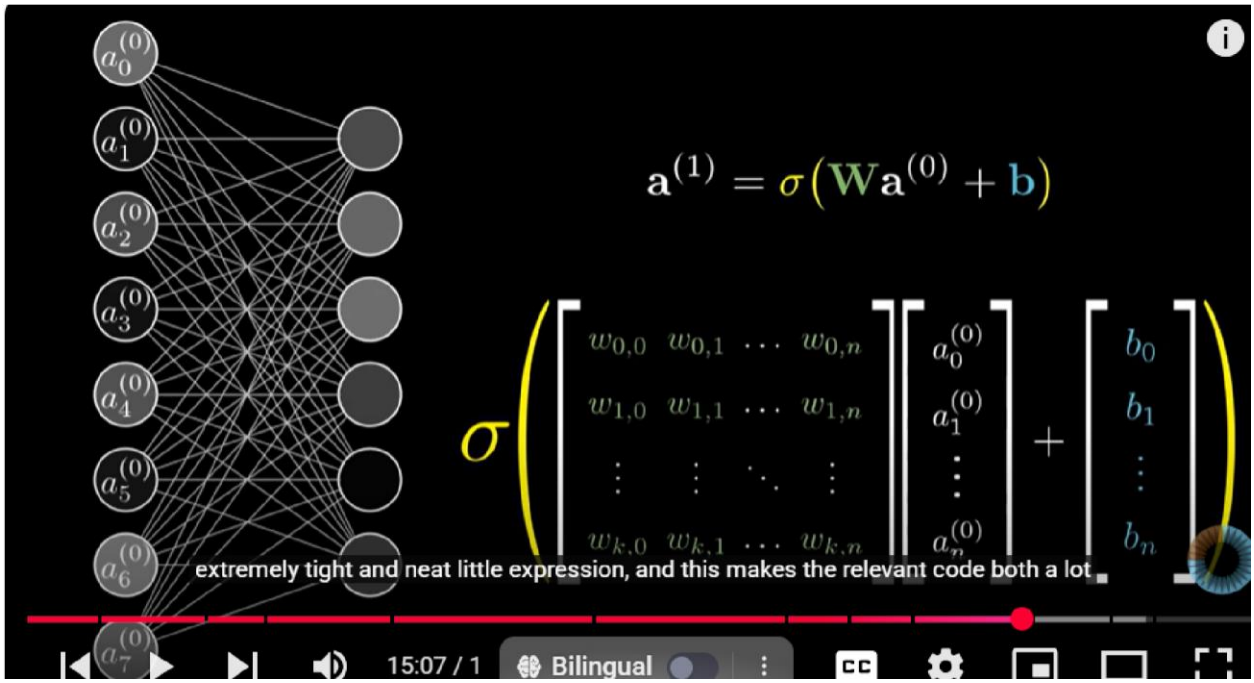
Perceptron se jo raw output aata hai, usay hume transform karna hota hai taake wo **samajhne layak aur non-linear decision** de sake. Iske liye **activation functions** use hotay hain.

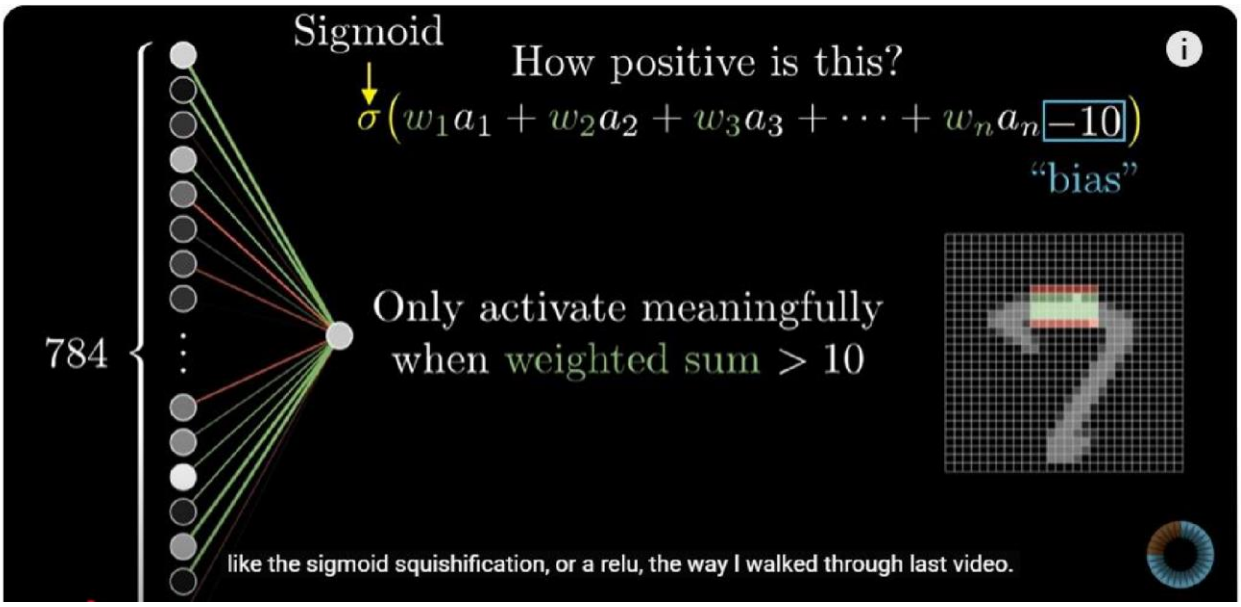
Kuch common activation functions:

1. **Sigmoid** → output 0 aur 1 ke darmiyan deta hai.
   Example: Agar model ko decide karna ho ke *ye email spam hai ya nahi*, sigmoid probability batata hai.




$\mathbf{a}^{(1)} = \sigma(\mathbf{W}\mathbf{a}^{(0)} + \mathbf{b})$
class Network(object):
def __init__(self, *args, **kwargs):
#...yada yada, initialize weights and biases...
def feedforward(self, a):
"""Return the output of the network for an input vector a"""
for b, w in zip(self.biases, self.weights):
a = sigmoid(np.dot(w, a) + b)
return a
multiplication.


$\mathbf{a}^{(1)} = \sigma(\mathbf{W}\mathbf{a}^{(0)} + \mathbf{b})$
extremely tight and neat little expression, and this makes the relevant code both a lot
15:07 / 1
Bilingual



2. **ReLU (Rectified Linear Unit)** → agar input negative ho to 0 dega, warna wahi input return karega.
   Example: Tum gym mai ho, weight uthana hai → negative effort ka matlab kuch kaam nahi (0), positive effort ka matlab tumhare muscles grow karenge.
3. **Tanh** → output -1 aur 1 ke darmiyan deta hai.
   Example: Agar tum mood prediction kar rahi ho (happy/sad), tanh achi choice ho sakti hai.

---

c) Loss Functions

Loss function hume batata hai ke humara neural network **kitna ghalat** hai. Ye model ka **report card** hai.

☞ Goal ye hota hai ke loss **minimum** ho jaye (jese exams mai marks zyada aur ghaltiyan kam).

Types:

1. **Mean Squared Error (MSE)** → numbers predict karne ke liye (jaise ghar ki price).
   Example: Model ne bola ghar 50 lakh ka hai, asal mai 55 lakh tha → error = $(55\text{-}50)^2$.
2. **Cross Entropy Loss** → classification problems ke liye (cat vs dog).
   Example: Tumhari model ne "cat" image ke liye 0.2 probability di aur asal mai wo cat thi

Quick Cheat-Sheet (binary intuition)

p* (correct class prob)	Loss (−log p*)	Soch
0.95	≈ 0.05	Bohat achha
0.90	≈ 0.10	Achha
0.70	≈ 0.36	Theek-thak
0.50	≈ 0.69	Uncertain
0.20	≈ 1.61	Kafi bura
0.01	≈ 4.61	Bahut bura

→ loss high hoga. Output hoda1.16 approx

🔷 3. Learning Resources (jo tumne likhe)

- **3Blue1Brown Neural Network Playlist** → ye visualization ke liye best hai (math ko samajhna easy ho jata hai).
- **FreeCodeCamp PyTorch Intro** → ye coding ke liye best hai (practical hands-on).

✨ Summary (Day 1):

- Deep Learning = Brain inspired learning method.
- Neural Network = neurons (perceptrons) ka jala (network).
- Perceptron = decision maker.
- Activation Function = output ko shape deta hai. □ Loss Function = error measure karta hai.

Chalo step-by-step samajhte hain ki **Basic Neural Network (Feedforward / ANN)** kaam kaise karta hai.

1. Structure

- **Input Layer** → Jaha pe data enter hota hai (jaise 28×28 image ke 784 pixels).
- **Hidden Layer(s)** → Ye layer data ke andar ke patterns ko samajhne ki koshish karti hai.
- **Output Layer** → Final prediction deti hai (MNIST ke case me 10 neurons = digits 0–9).

⚙ 2. Working Process

(a) Input → Weighted Sum

Har input neuron (pixel value) ko ek **weight** se multiply kiya jata hai aur phir unka sum nikala jata hai:

z=w1x1+w2x2+…+wnxn+bz = w_1x_1 + w_2x_2 + \dots + w_nx_n + bz=w1x1+w2x2+…+ wn xn+b

- www = weight (kitna important hai ek pixel/feature)
- xxx = input (pixel value)
- bbb = bias (adjust karne ke liye extra term)

(b) Activation Function

Ye **z** value ek **activation function** ke through pass hoti hai (sigmoid, ReLU, tanh etc.) jo decide karta hai ki neuron **active hoga ya nahi**.

(c) Forward Propagation

Data input layer se hidden layer aur phir output layer tak **forward propagate** hota hai.

- Har layer pe new features extract hote hain.
- Last layer me probability milti hai (jaise softmax se: 80% chance ki digit = “3”).

(d) Loss Function

Model ki prediction aur actual answer (ground truth) ke beech ka difference nikalte hain.
Ye difference ko **loss function** (jaise Cross-Entropy Loss) kehte hain.

(e) Backpropagation

- Loss ko kam karne ke liye, neural network apne **weights aur biases adjust** karta hai. □ Ye adjustment **gradient descent** aur **backpropagation** algorithm se hota hai.

(f) Training Process

1. Input dete hain (image).
2. Forward pass hota hai (output milta hai).
3. Loss calculate hota hai.
4. Backpropagation se weights update hote hain.
5. Ye cycle **bohot baar (epochs)** repeat hota hai jab tak model sahi seekh na le.

kuch samjhna hai 28*28 image pixel kya hota hai

ChatGPT said:

Achha, main simple shabdon mein samjhata hoon ☺

☞ **28×28 image pixel** ka matlab hai ki image ek **grid** (jaise chhota box) me bani hoti hai, jisme:

- **28 rows (line)** aur
- **28 columns (columns)** hote hain.

Toh total **28 × 28 = 784 pixels** hote hain.

Har **pixel** ek chhota dot hota hai jo color ya intensity dikhata hai:

- Agar image **black & white (grayscale)** ho, toh har pixel ki value **0 se 255** tak hoti hai.

- **0 = black (kala)** ○ **255 = white (safed)** ○ Beech ki values = gray shades.
- Agar image **color (RGB)** ho toh har pixel ke liye 3 values hoti hain (Red, Green, Blue).

Example:

- MNIST dataset (jo handwritten digits ka hota hai, jaise 0–9 likhe hue numbers) me har image 28×28 pixel grayscale hoti hai.
- Matlab ek digit ka photo bas ek **chhoti si 28×28 matrix** hoti hai.

Working of layers

Chalo tumhari baat ko thoda aur clear karta hoon:

1. **Input Layer** ○ MNIST digit image ka size 28×28 = **784 pixels**. ○ Matlab input layer me **784 neurons** hote hain.
   - Har neuron ek pixel value ko represent karta hai (0 = black, 255 = white).
2. **Hidden Layers** ○ Ye layers image ke andar ke **patterns/features** dhoondti hain.
   - Pehli hidden layer simple patterns (jaise lines, curves) pakadti hai. ○ Agli hidden layers un chhoti cheezon ko jod kar bade patterns samajhti hain (jaise digit ke shapes).
   - Is process se network step by step digit ka structure samajhne lagta hai.
3. **Output Layer** ○ Output layer me **10 neurons** hote hain (digits 0–9). ○ Har neuron ek probability deta hai (jaise digit "3" ka 0.95 probability). ○ Jis neuron ka probability sabse zyada hota hai, wahi **final prediction** hoti hai.

Deep Learning Chapter 2

Gradient descent, how neural networks learn |

Gradient Descent – Idea

Gradient Descent aik algorithm hai jo hume help karta hai **neural network ke weights aur biases ko adjust karne mai**, taake loss kam se kam ho jaye.

Simple words mai: **Ye ek tareeqa hai "galti (loss)" kam karne ka.**

---

1. Loss Landscape (Surface)

Socho tumhari model har guess ke sath ek "error" karti hai.
Agar hum in sab possible errors ko graph par plot karein → hume ek **mountain ya valley jaisa surface** milega (loss function ka curve).

- Upar wale point = **zyada error**
- Neeche wale point = **kam error**
- Goal = **neeche sab se low point (minimum loss)** tak pohchna

☞ Real-life example:
Socho tum andheray kamray mai ho aur ek ball ko neeche valley tak roll karna hai. Tum chaho gi ke ball sabse neeche ke point tak ruk jaye.

---

2. Gradient

Gradient = slope (ढलान) hoti hai jo batati hai ke is waqt hum jis jagah hain, neeche utarne ke liye kis direction mai jana chahiye. □ Agar slope positive hai → neeche jane ke liye **ulta direction** lena hoga.

- Agar slope negative hai → slope ke ulte side move karna hoga.

☞ Real-life example:
Socho tum hill se neeche utar rahi ho. Tumhe bas slope ke opposite direction mai chalte rehna hai taake tum neeche valley mai aa sako.

---

3. Learning Rate

Ye ek **chhota step size** hota hai jo decide karta hai ke hum ek step mai kitni door move karein.

- Agar learning rate **bohot chhoti** hai → neeche jane mai time zyada lagega (slow learning).
- Agar learning rate **bohot badi** hai → tum neeche se guzr kar doosri taraf gir jaogi (unstable learning).

☞ Real-life example:

Socho tum hill se neeche ja rahi ho:

- Chhote chhote kadam = safe but slow.
- Bahut bade kadam = risk ke tum balance kho do.

---

4. Iterations

Gradient Descent ek step mai perfect answer nahi deta. Ye process repeat hota hai:

1. Gradient nikaalo (kis direction mai jana hai)
2. Weights adjust karo
3. Loss calculate karo
4. Repeat…

☞ Har step mai tum neeche ke aur kareeb hoti jati ho.

🔻 Summary (Video ka point)

- Neural Network initially random guesses karta hai.
- Gradient Descent use karke hum weights ko thoda thoda adjust karte hain.
- Har step mai model ki galti (loss) kam hoti hai.
- Aakhir mai model valley ke neeche wale best point par pohchta hai = **minimum loss** = **best trained model**.

✨ Simple Example

- Tumne ghar ki price predict karna seekhna hai.
- Pehle tum random guess karti ho (jaise 20 lakh).
- Tumhe asal answer aur guess ka farq (loss) milta hai.
- Gradient descent tumhe batata hai ke agla guess **upar karna hai ya neeche**, aur kitna. □ Step by step, tumhari guess asal ke kareeb hoti jati hai.

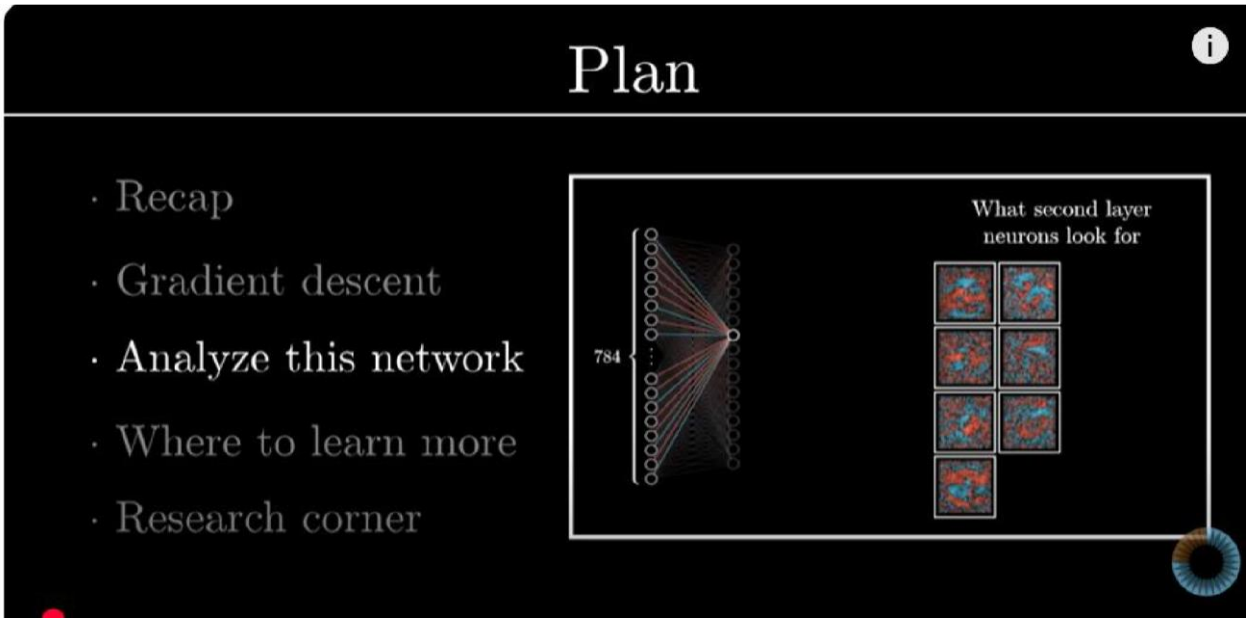
3. Coding mai kya hota hai?

PyTorch / TensorFlow mai tum sirf **ye likhti ho**:

```
loss.backward()     # PyTorch ko bolo gradients calculate karo optimizer.step()    #
Weights update karo gradient descent use karke optimizer.zero_grad()  # Purane
gradients clear karo
```

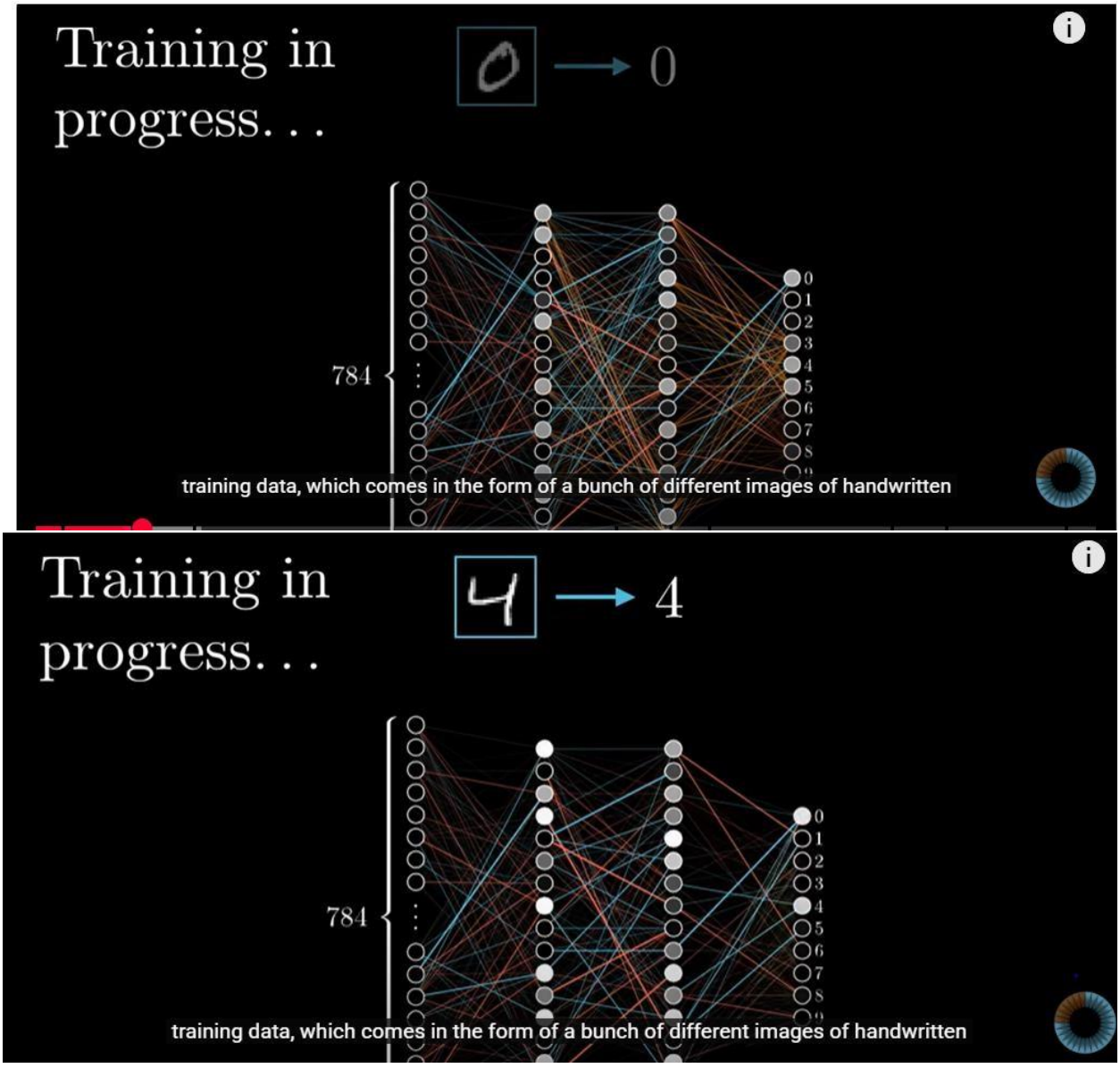
- Tumhe derivatives ya formulas likhne ki **zaroorat nahi**.
- Framework khud chain rule (backpropagation) use karke gradients nikalta hai.
  - Optimizer (SGD, Adam, etc.) automatically **update rule** apply karta hai.

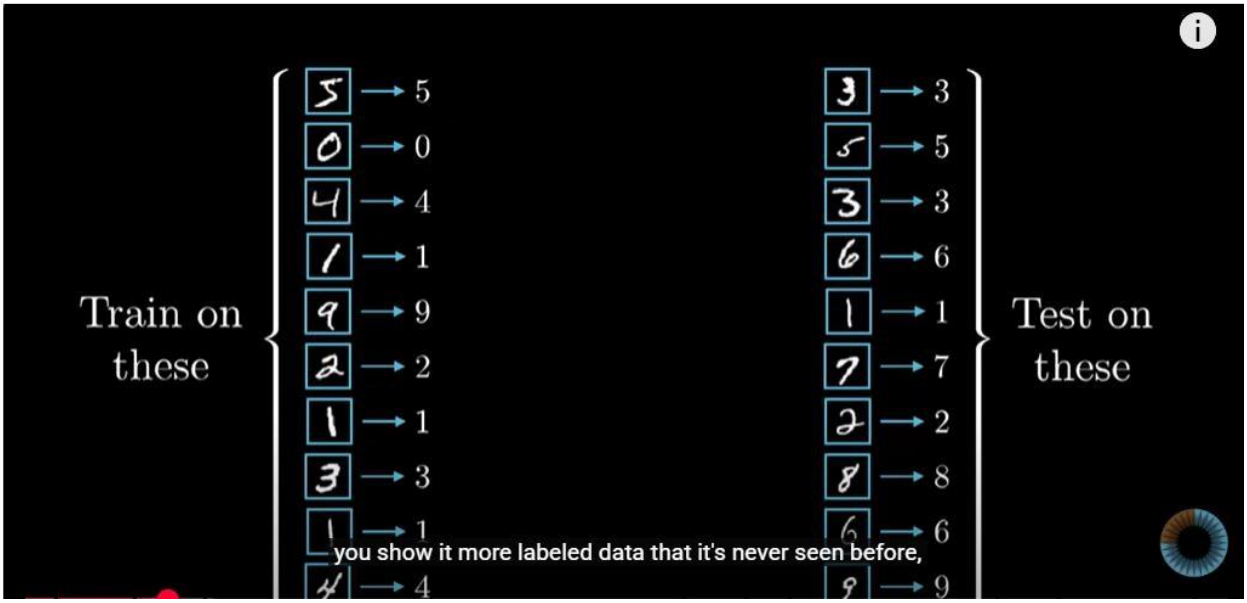
Chapter 3

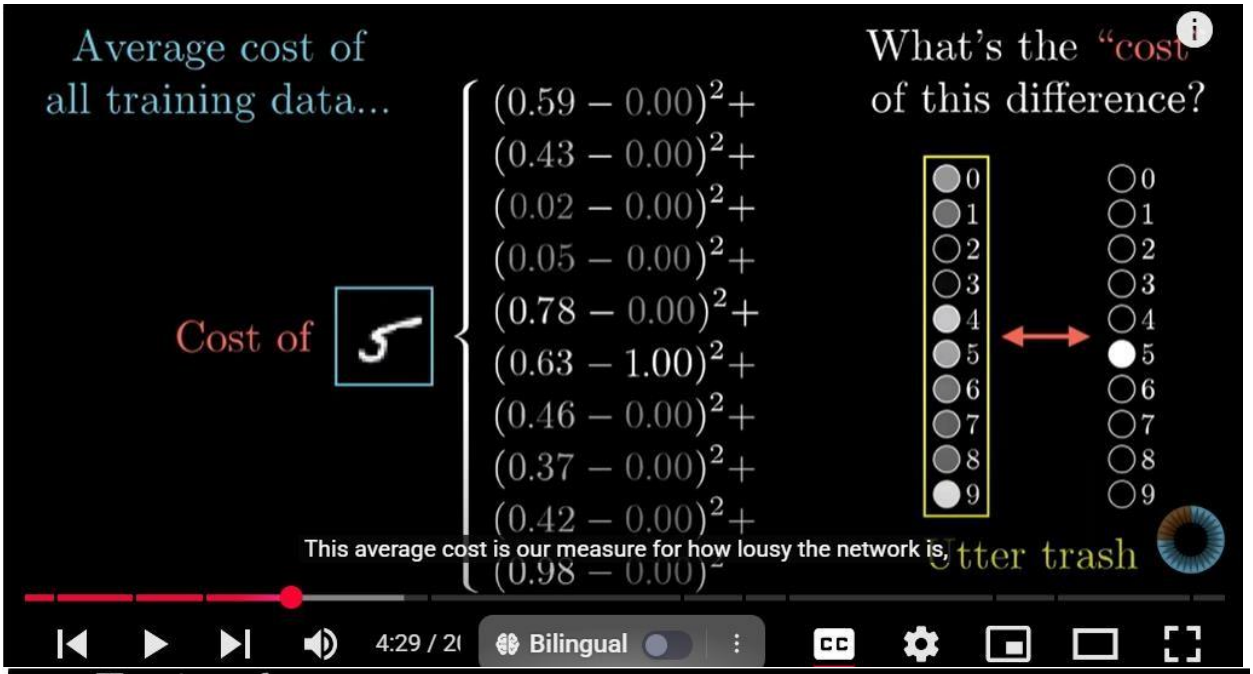


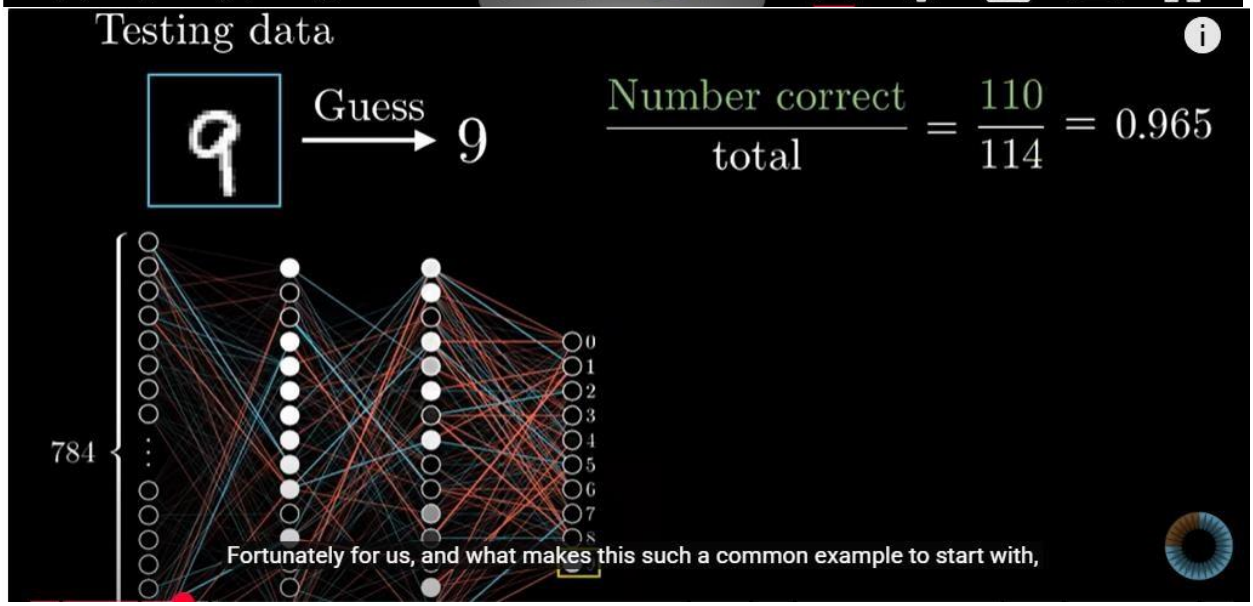
Training data:

hm aik algorithm laitai hai usko hm hath sai likha input daita hai sath mai uska label output b daita hai, or train krtai hai



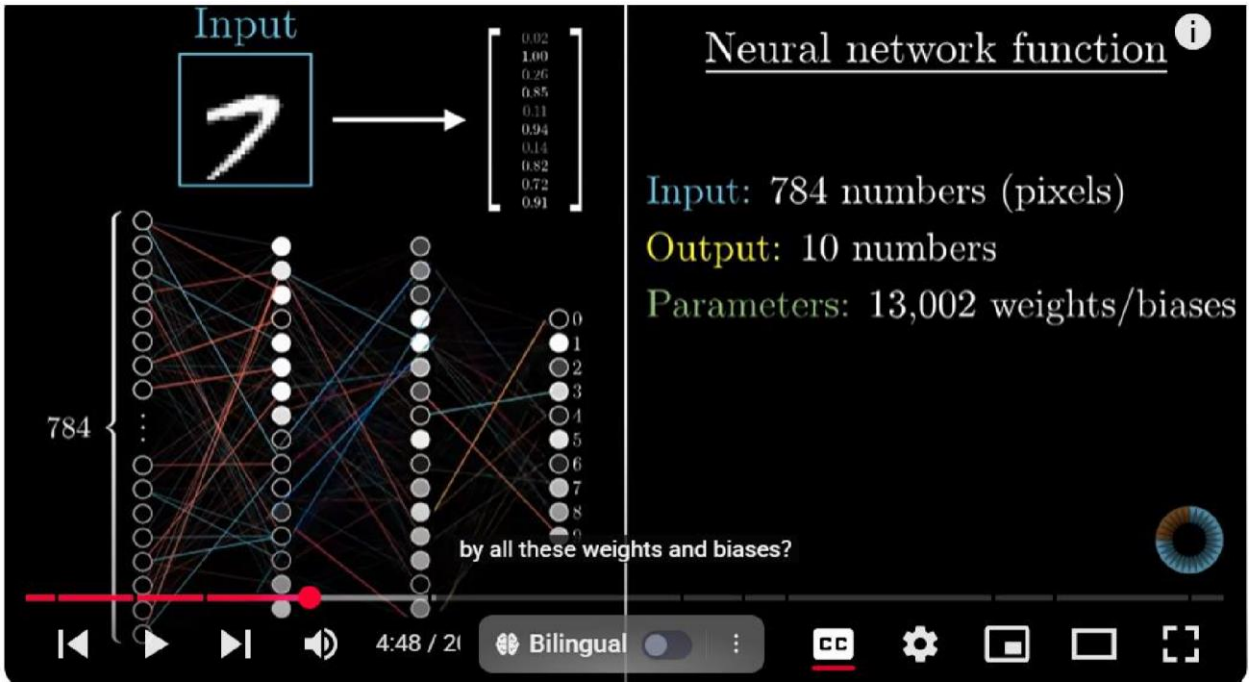

Train on these
5 → 5
0 → 0
4 → 4
1 → 1
9 → 9
2 → 2
1 → 1
3 → 3
1 → 1
4 → 4
3 → 3
5 → 5
3 → 3
6 → 6
1 → 1
7 → 7
2 → 2
8 → 8
6 → 6
9 → 9
Test on these
you show it more labeled data that it's never seen before,



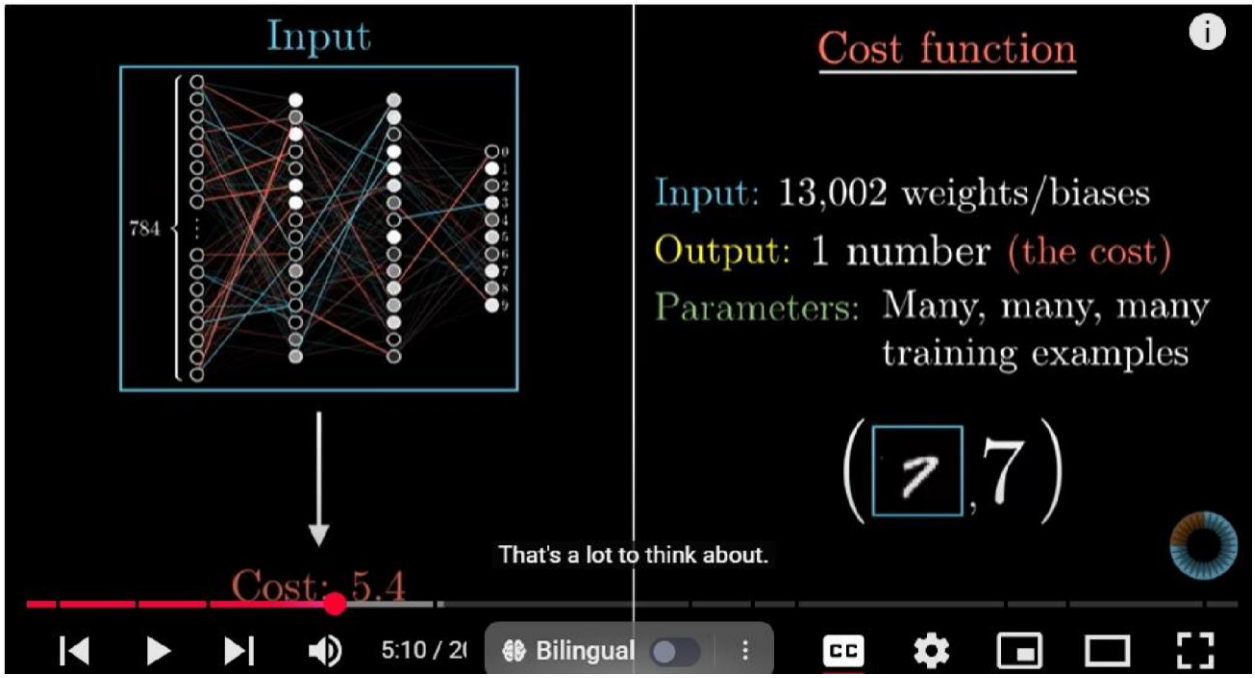


Average cost:

function of neuron network:



Cost function on neuron network



Yai iska back pr kaam krta hai us image k parameters ko as input lai ga, out put mai koi aik number dai ga cost klaiyai, decide karai ga cost yani bias or wight kitna kharaab hai kitna error hai yaha, yai base krta hai us data pr j ohm nai training mai diya tha,:

1. Cost Function kya hai?

Cost Function (ya Loss Function) ek mathematical formula hai jo batata hai:
☞ *"Hamari model ki prediction aur asal answer mai kitna farq hai?"*

- Agar prediction sahi ke kareeb hai → **Cost chhota hoga**
- Agar prediction bohat galat hai → **Cost bada hoga**

Isliye, **training ka main goal** hota hai:

Cost Function ko minimize karna (sabse kam karna)

🔷 2. Cost Function vs Loss Function

- **Loss Function** → ek single data point ka error batata hai.
- **Cost Function** → saare data points ke errors ka **average** hota hai.

☞ Matlab:

- Agar tum 1 ghar ki price predict karti ho aur ghalat hoti ho → **loss**.
- Agar tum 100 ghar ki price predict karti ho aur un sab ki ghaltiyon ka average nikalti ho → **cost**.

🔷 3. Common Cost Functions

a) Mean Squared Error (MSE)

Regression (numbers) ke liye.

Cost=1n∑(ytrue−ypred)2Cost = \frac{1}{n} \sum (y_{true} - y_{pred})^2Cost=n1∑(ytrue −ypred)2
☞ Example:
Asal price = 55 lakh, Prediction = 50 lakh
Error = $(55 - 50)^2 = 25$
Cost jitna chhota hoga, utni achhi prediction.

---

b) Cross Entropy Loss

Classification ke liye (cat vs dog).

Cost=−∑ytrue⋅log⁡(ypred)Cost = - \sum y_{true} \cdot \log(y_{pred})Cost=−∑ytrue⋅log(ypred)

☞ Example:
Asal class = Cat, Prediction = Cat: 0.2, Dog: 0.8
Cost = $-\log(0.2) \approx 1.61$
Matlab: cost high hai, model ne galat class ko zyada probability di.

🔷 4. Gradient Descent aur Cost Function

Gradient Descent har step mai **Cost Function ko neeche laane** ki koshish karta hai.

- Gradient = batata hai ke cost kis direction mai fastest neeche ja raha hai.
- Weights ko adjust karke hum cost ko dheere dheere minimize karte hain.

☞ Simple analogy:
Socho tum ek hilltop par khadi ho (high cost). Tumhe neeche valley (minimum cost) mai pohchna hai. Har step mai tum slope follow karti ho → dheere dheere neeche cost hota jata hai.

🔷 5. Real-life Example

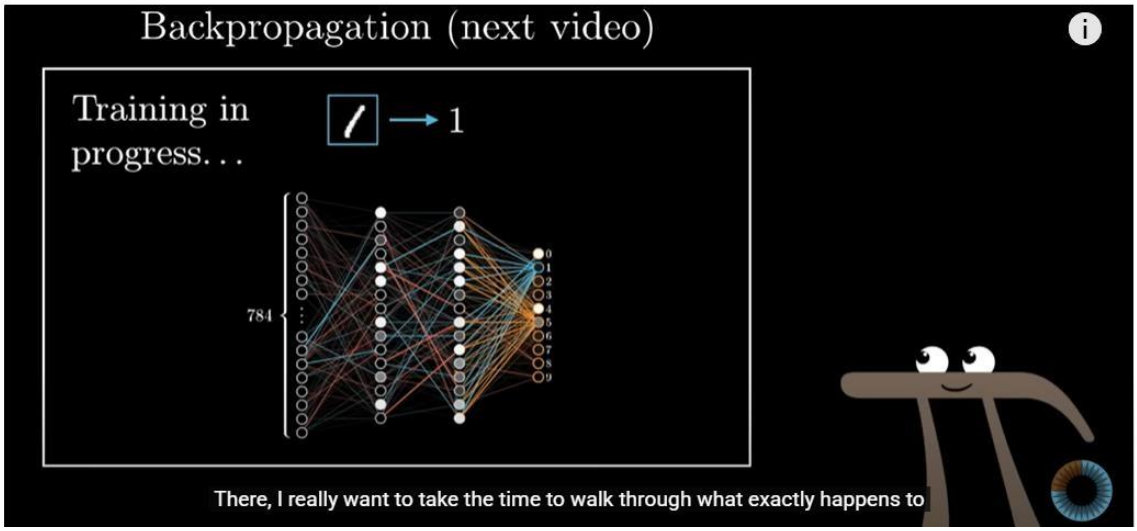
Socho tum apni padhai ka "performance score" nikalna chahti ho.

- Har exam mai marks aur tumhari guess mai farq hota hai.
- Ye farq tumhara **loss** hai.
- Saare exams ka average farq = tumhara **cost** hai.
- Tumhari strategy (Gradient Descent) yeh hogi ke tum agle exams mai mistakes kam karti jao → **cost decrease hota rahe**.

✨ **Summary:**

- **Cost Function** = overall measure of how bad the model is performing.
- Goal: minimize cost.
- Loss = ek example ka error, Cost = saare errors ka average. □ Gradient Descent = cost ko neeche laane ki tareeqa.

Heart of neural network



1. Backpropagation kya hai?

Backpropagation (short = backprop) ek algorithm hai jo neural network ke **weights ko update karne ke liye gradients calculate karta hai**.
☞ Matlab: ye hume batata hai ke *"har weight loss function ko kitna effect karta hai"*.

🔷 2. Intuition – ek simple soch

Socho tum **ghar ki price predict karne** wali model bana rahi ho:

- Input → features (size, rooms, location)
- Output → predicted price

Model ne ghalti ki → Asal price 55 lakh, predict kiya 50 lakh → Loss = 25

Ab question:
☞ *"Ye ghalti kis wajah se hui? Kya size ka weight galat tha? Kya rooms ka effect zyada tha? Kya location ignore hui?"*

Backpropagation ka kaam yehi hai:

- Har weight ki zimmedari calculate karna.
- Jitna zimmedar hai → utna zyada usko adjust karna.

🔻 3. Forward Pass → Backward Pass

1. **Forward pass:** Input → weights → activation → output → loss calculate hota hai.
2. **Backward pass:** Loss se shuru hoke, network ke andar peeche jaate hain aur har weight ke liye gradient nikalte hain.

☞ Isi ko "backpropagation" kehte hain = loss ko network ke andar "peeche propagate" karna.

🔻 4. Gradient ka kaam

- Gradient = slope (direction aur rate of change).
- Backprop hume batata hai:
  - Agar weight thoda increase karein to loss kam hoga ya zyada? ○ Aur kitna change karna chahiye?

☞ Matlab: backprop weight update karne ka **map** deta hai.

🔻 5. Chain Rule (math intuition)

Ye thoda math wala hissa hai jo video mai dikhaya jata hai:

- Neural network ek **chain of functions** hai (input → hidden layers → output → loss). □ Loss depend karta hai har ek layer ke weights par.
- Backprop **chain rule of calculus** use karta hai taake gradients layer by layer calculate ho sake.

☞ Simple soch:

- Agar aakhri layer galti karti hai, uska asar pichli layer par bhi hota hai.
- Ye asar peeche tak flow hota hai → isi ko "backpropagation" kehte hain.

🔻 6. Real-Life Analogy

Socho tumhari ek team ne project kiya aur client ne complain kar diya ke kaam ghalat hai (loss high).

- Manager (output layer) ko pehle blame milega.
- Manager dekhega → kis team member ki wajah se problem hui.
- Phir team lead apne juniors par zimmedari "back propagate" karega.
- Aakhir mai har individual ko pata chalega ke usne kitni ghalti ki aur next time kya sudharna hai.

☞ Yani backprop ek **blame assignment system** hai jo har weight ko batata hai ke wo ghalti mai kitna responsible hai.

🔻 7. Final Step – Update

Jab gradients mil gaye to **Gradient Descent** use hoke weights update hote hain:

wnew=wold−η⋅gradientw_{new} = w_{old} - \eta \cdot \text{gradient}wnew=wold−η⋅gradient

Aur phir agla forward pass hota hai → loss aur kam ho jata hai.

✨ **Summary (Intuitive):**

- Backpropagation = ghalti ko peeche layer tak propagate karna.
- Har weight ko "blame" assign hota hai ke wo kitna zimmedar hai.
- Gradients calculate hote hain → Gradient Descent use karke weights update hote hain.
- Ye process har epoch mai repeat hota hai → model dheere dheere sahi seekhta hai.

🔻 Loss Function vs Cost Function (clear difference)

- **Loss Function** → ek single data point ka error measure karta hai.
- **Cost Function** → poore training dataset (ya ek batch) ke saare losses ka **average** hota hai.

✨ Example

Socho tumhare paas **3 training examples** hain (ghar ki prices):

- Asal prices = [50, 60, 70]
- Model predictions = [48, 65, 72]

Agar hum **MSE Loss** use karein:

1. Sample 1 ka Loss = $(50 - 48)^2 = 4$
2. Sample 2 ka Loss = $(60 - 65)^2 = 25$
3. Sample 3 ka Loss = $(70 - 72)^2 = 4$

☞ Ab **Cost Function = average loss**:

Cost=4+25+43=11Cost = \frac{4 + 25 + 4}{3} = 11Cost=34+25+4=11

🔻 Training mai role

- Neural Network har sample ka **loss** calculate karta hai.
- Sab losses ka **average** = **Cost Function**.
- Gradient Descent isi **Cost Function** ko minimize karta hai step by step.

✅ Matlab: **Cost Function ka dusra naam hi "average loss on training data" hai.**
Isliye jab log kehte hain *"minimize the cost function"*, wo asal mai keh rahe hote hain:

"Poore training data ki average ghalti ko kam karo."

🔷 Simple Picture

1. **Gradient:** “Tum abhi slope ke is hisse pe ho, neeche girne ka raasta ye hai.”
2. **Gradient Descent:** “Theek hai, ab mai tumhe ek controlled step neeche le jaata hoon.”

Summerey overall

1. Algorithms (ye “processes” hain jo chalti hain)

Algorithms wo hote hain jo **steps follow kar ke output dete hain**. Ye khud ek complete method hote hain.

Hamari list se algorithms: □ **Gradient Descent** → weights ko step by step update karta hai taake cost kam ho. □ **Backpropagation** → gradients (derivatives) calculate karke har weight ka blame nikalta hai.

□ **Logistic Regression** (agar use karein) → ek complete model/algorithm hai classification ke liye.

☞ Inko tum “kaam karne ke tareeqe” keh sakti ho.

🔷 2. Functions (ye “mathematical formulas” hote hain)

Functions wo hote hain jo sirf ek input lete hain aur ek output dete hain, jaise calculator ka button dabana.

Hamari list se functions:

- **Activation Functions:**
  - Sigmoid σ(z)\sigma(z)σ(z) ○ ReLU, Tanh (baad me milenge) □ **Loss Functions (ek sample ke liye):**
  - Mean Squared Error (MSE)
  - Cross-Entropy Loss
- **Cost Function (overall average loss):**

  C=1N∑LossC = \frac{1}{N} \sum LossC=N1∑Loss

- **Linear function (neurons ke andar):** z=w⋅x+bz = w \cdot x + bz=w⋅x+b

- **Sigmoid output:**
  a=σ(z)a = \sigma(z)a=σ(z)

☞ Ye sirf formulas hain jo values nikalte hain.

🔷 3. Mix of Both

- **Computational Graph** khud ek tool hai, jisme functions nodes ke form mai hote hain, aur backpropagation algorithm us graph pe run karta hai.

✨ Final Separation

✅ **Algorithms:** Gradient Descent, Backpropagation, Logistic Regression
✅ **Functions:** Activation (sigmoid, ReLU), Loss (MSE, Cross-Entropy), Cost (average loss), Linear function (z = wx+b)

Matlab:

- **Functions = building blocks (math)**
- **Algorithms = recipes (kaise function use karne hain aur step by step kaam karna hai)**

Intuition ka Matlab

- Intuition = **seedhi aur simple soch**, taake tumhe concept ka *feel* aaye.
- Example:
  - Neuron = “input lo → weight se multiply karo → bias add karo → activation se guzar do.”

- Ye explain karne ke liye hum kehte hain “neuron ek formula lagata hai aur ek output deta hai.”

☞ Isme hum zyada detail me **derivatives, gradients, backpropagation** nahi late.

🔻 Forward Pass vs Intuition

- **Forward Pass (actual calculation):**
  Tum input dete ho → har layer me “linear function (wx+b)” aur activation lagta hai → output milta hai.
- **Backward Pass (actual calculation):**
  Cost nikal kar usko ulta chain rule ke through propagate karte ho → har weight update hota hai.
- **Intuition (simple explanation):**
  Sirf ek “rough mental picture” — jaise neuron ek decision machine hai jo input ko transform karta hai.

✅ Matlab: **intuition ke andar forward/backward pass ki detail nahi hoti, sirf easy language mai idea hota hai.**
Jaise tum ek 5 saal ke bache ko samjha rahi ho: “Neuron bas input ko multiply-add karke thoda adjust karta hai, fir ek button (activation) se guzarta hai.”

Kya tum chahogi mai tumhare liye **ek chhoti list banaun jisme likhun: Intuition vs Actual Steps (Forward/Backward Pass)**? Taake dono ka farq clear ho jaye.

You said: ok

ChatGPT said:

Shabash Kaisnat 🔥 ab mai tumhe ek **clear comparison list** bana ke deta hoon:

🔻 Intuition vs Actual Steps (Neural Network)

1. Intuition (sirf idea)

- Neuron ek **chhoti machine** hai:
  - Input lo
  - Weight se multiply karo
    - Bias add karo ○ Activation lagao
    - Output nikal do
- Model inputs se dheere dheere seekhta hai aur errors ko kam karta hai.
- Bas "feel" dena hoti hai, math details chhupai jati hain.

☞ Example intuition:
"Neuron ek student jaisa hai — inputs uske notes hain, weight uski importance, bias uska mood, aur output uska answer."

---

2. Actual Steps (Forward + Backward Pass)

🔻 Forward Pass

1. Inputs → multiply with weights
2. Add bias
3. Pass through activation function
4. Layer by layer → final output
5. Loss calculate (prediction vs actual)

🔻 Backward Pass (Backpropagation)

1. Loss se shuru karte hain
2. Chain rule use karke gradients calculate karte hain ○ ∂Loss/∂Weight ○ ∂Loss/∂Bias
3. Gradient Descent se weights update hote hain wnew=wold−η⋅gradientw_{new} = w_{old} - \eta \cdot \text{gradient}wnew=wold−η⋅gradient

4. Ye process bar-bar repeat hota hai (epochs) → loss kam hota hai

---

✨ Bottom Line

- **Intuition:** Easy soch → “Neuron bas inputs ko transform karta hai.”
- **Actual:** Full process → forward pass (calculate output) + backward pass (update weights).

Questions

1) Q: **z = input × weight + bias** ko kya kehte hain?

Tumhara answer: sigmoid function
Feedback / Correction: Yeh **galat** hai.

- Sahi naam **linear function** ya **affine transformation** hai: z=w⋅x+bz = w\cdot x + bz=w⋅x+b.
- **Sigmoid** activation uske baad aata hai: a=σ(z)a = \sigma(z)a=σ(z).
  So: pehle **linear (wx+b)**, phir **activation (sigmoid / ReLU / tanh)**.

---

2) Q: Cost kam karne ke liye kaunsa algorithm?

Tumhara answer: gradient descent ✅
Feedback: Bilkul sahi. (Aur variants: SGD, Adam wagaira — ye optimizer ke naam hain.)

---

3) Q: Loss vs Cost difference?

Tumhara answer: loss = single data ka, cost = average ✅
Feedback: Bilkul sahi. Useful note: practice me hum **batch cost** (batch ka average) bhi use karte hain na ke pure dataset ka har step pe.

---

4) Q: Forward pass aur Backward pass ka kaam?

Tumhara answer (short): forward: input lo, weight calculate kro, activation apply kro, output lo. backward: output se start, back jao dekh issue kaha hai.
Feedback / Correction: Idea sahi, magar chhoti correction: **weights calculate nahi hote** forward pass me — forward pass **current weights use** kar ke prediction banata hai.

- Forward = inputs → (wx+b) → activation → output → loss.
- Backward = loss se start → gradients nikalna (har weight ke liye ∂Loss/∂w) → optimizer se weights update.

---

5) Q: Backprop me chain rule kyun use karte hain?

Tumhara answer: kyunki previous layers se connect hona hota hai, weight update hota rehta hai. ✅
Feedback (thoda sa clear): Sahi socha — formal reason: **loss ek composite function** hota hai of many functions (layers). Chain rule se hum compute karte hain:

∂C∂w=∂C∂a⋅∂a∂z⋅∂z∂w\frac{\partial C}{\partial w} = \frac{\partial C}{\partial a}\cdot\frac{\partial a}{\partial z}\cdot\frac{\partial z}{\partial w}∂w∂C=∂a∂C⋅∂z∂a⋅∂w∂z Matlab har step pe local derivative multiply kar ke peechay tak gradient laate hain.

Chapter 4

🔷 PyTorch in 2 Lines

- **PyTorch ek deep learning framework hai** jo Python me neural networks aur tensor (math arrays) operations ko easy banata hai.
- Matlab: tumhe matrix math manually karne ki zaroorat nahi, PyTorch khud hi GPU use karke fast calculation aur backpropagation handle karta hai.

Tensor:

🔷 Tensor Kya Hai?

- **Simple words:** Tensor ek **multi-dimensional array** hota hai (jaise NumPy array).
- PyTorch mai sab kuch **tensor** ke form mai hota hai: inputs, weights, outputs — sab tensors.
- Tensors ko GPU pe bhi run karwa sakte ho → fast computation (yehi PyTorch ki taqat hai).

🔷 Intuition

- **0D tensor:** ek single number (scalar) → `torch.tensor(5)`
- **1D tensor:** numbers ki line (vector) → `[1, 2, 3]`
- **2D tensor:** numbers ka table (matrix) → `[[1, 2], [3, 4]]`
- **3D tensor:** jaise ek box (image pixels)
- **Higher D:** aur bhi complex data (videos, batches of images, etc.)

🔷 Example in PyTorch

python

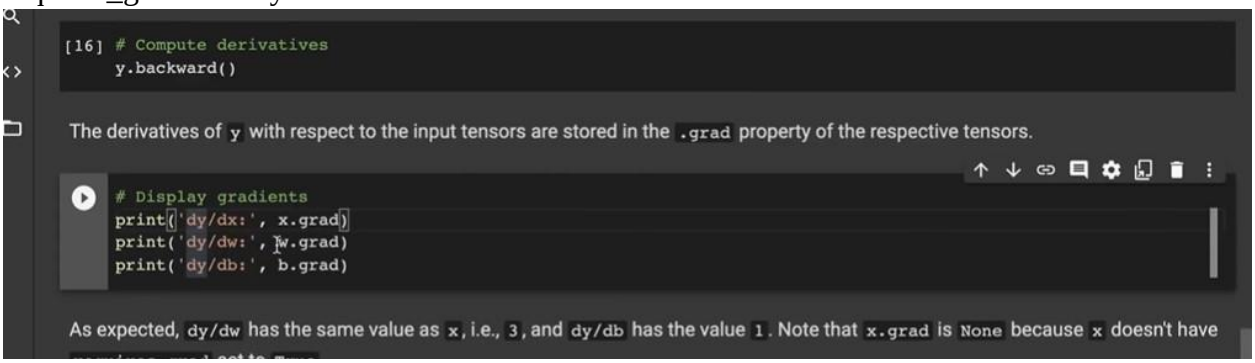
```python
import torch

# 0D Tensor (scalar)
a = torch.tensor(5)

# 1D Tensor (vector)
b = torch.tensor([1, 2, 3])

# 2D Tensor (matrix)
c = torch.tensor([[1, 2], [3, 4]])

# Check dimensions
print(a.ndim, b.ndim, c.ndim)  # 0, 1, 2
```

a.ndim bta raha hai a mai kitnai dimension hai (0)

b.ndim bta raha hai b mai kitnai dimension hai(1)

```python
import torch

x = torch.tensor([2.0], requires_grad=True)  # gradient ON
y = x**2 + 3*x + 1
y.backward()
print(x.grad)   # tensor([7.])
```

Requires_grade=true yai btata hai k hm x k derivative mai intresested hai



X mai tendor k ander aik scaller store hai

Y mai aik equation hai

y.backward() y.backward derivative calculate karai ga or usko grade property mai store karai ga

Tensor properties

1. .dtype (Data Type)

- Ye batata hai ke tensor ke andar numbers kis type ke hain (int64, float32, etc.).

x = torch.tensor([1, 2, 3]) print(x.dtype)
# torch.int64

🔹 2. .device (CPU / GPU location)

- Ye batata hai tensor memory me **CPU pe hai ya GPU pe**.

```
x = torch.tensor([1.0, 2.0]) print(x.device)
# cpu
```

🔹 3. .shape (Size of Tensor)

- Ye tensor ka **size (rows × cols)** batata hai.

```
x = torch.tensor([[1, 2], [3, 4]]) print(x.shape)
# torch.Size([2, 2])
```

🔹 4. .ndim (Number of Dimensions)

- Ye tensor me kitne dimensions hain batata hai.

```
x = torch.tensor([[1, 2], [3, 4]]) print(x.ndim)
# 2
```

🔹 5. .requires_grad (Need Gradient?)

- Ye property set karne se PyTorch tensor ke liye **gradients calculate karega** (backprop).

```
x = torch.tensor([2.0], requires_grad=True) print(x.requires_grad)
# True
```

🔹 6. .grad (Gradient Value)

- Backprop ke baad is property me **derivative (gradient)** save hota hai.

```
x = torch.tensor([2.0], requires_grad=True) y
= x**2
y.backward()
print(x.grad)   # tensor([4.])
```

🔼 7. .T (Transpose)

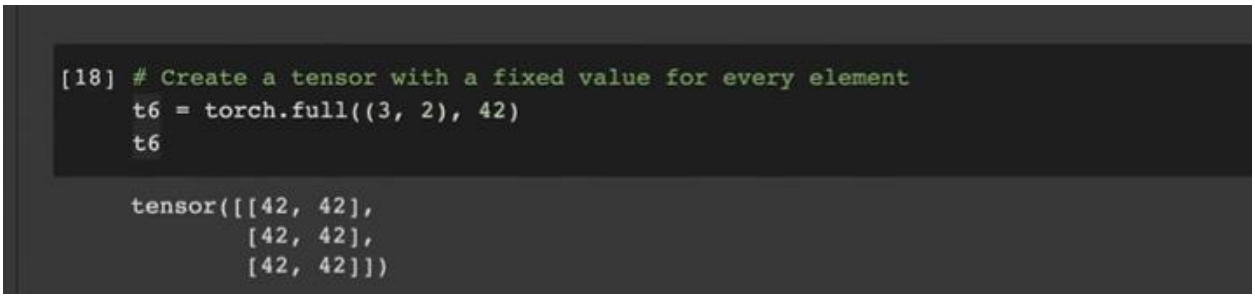
- 2D tensor (matrix) ki rows aur columns ko **swap** karta hai.

```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(x.T) # tensor([[1, 4],
#         [2, 5],
#         [3, 6]])
```

✅ Short Summary:

- .dtype → kis type ke numbers hain
- .device → kaha stored hai (CPU/GPU)
- .shape → size/structure
- .ndim → kitne dimensions
- .requires_grad → gradient chahiye ya nahi
- .grad → gradient value (after backprop)
- .T → transpose

Tensor with fix value:



```
[18] # Create a tensor with a fixed value for every element
     t6 = torch.full((3, 2), 42)
     t6

     tensor([[42, 42],
             [42, 42],
             [42, 42]])
```

Torch with cat:

🔷 Definition

`torch.cat` ka full form hai **concatenate**.
Ye function **multiple tensors ko jod kar ek naya tensor banata hai** — matlab "side by side chipka deta hai".

🔷 Syntax

```python
torch.cat(tensors, dim=0)
```

- `tensors` → list/tuple of tensors (jinhe jodna hai).
- `dim` → kis axis ke along jodna hai (row wise ya column wise).

🔷 Examples

1. Row wise concatenate (`dim=0`)

```python
import torch

a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6]])
result = torch.cat((a, b), dim=0)
print(result)
```

Output:

```lua
tensor([[1, 2],
        [3, 4],
        [5, 6]])
```

torch.sin

🔷 `torch.sin`

Ye function har element ka **sine value** nikalta hai (radians me).
Mathematically:

$$y = \sin(x)$$

🔷 Example 1 – Single Values

python

```python
import torch

x = torch.tensor([0.0, 3.1416/2, 3.1416])  # 0, pi/2, pi
y = torch.sin(x)
print(y)
```

Output:

scss

```
tensor([0.0000, 1.0000, 0.0000])
```

Torch Reshape

🔷 Definition

`reshape` tensor ka **shape (rows × cols × ...)** badal deta hai bina data ko change kiye.
Matlab data wahi rehta hai, sirf uska arrangement badal jata hai.

🔷 Syntax

python Copy code

```
tensor.reshape(new_shape)
```

🔷 Example 1 – 1D → 2D

python

```
import torch

x = torch.arange(1, 7)  # tensor([1, 2, 3, 4, 5, 6])
y = x.reshape(2, 3)     # 2 rows, 3 columns
print(y)
```

Output:

lua

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Torch For 3d

```python
import torch

x = torch.arange(24)        # 24 elements
y = x.reshape(2, 3, 4)      # 2 blocks, 3 rows, 4 cols
print(y)
```

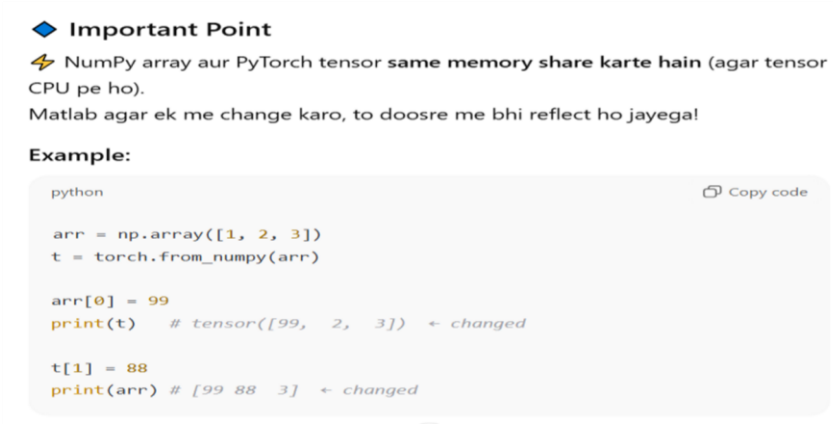
Output:

```lua
tensor([[[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]],

        [[12, 13, 14, 15],
         [16, 17, 18, 19],
         [20, 21, 22, 23]]])
```

👉 Shape = `(2, 3, 4)`

Matlab 2 blocks, har block me 3 rows aur 4 columns.

Numpy array in torch:



🔷 **Important Point**

⚡ NumPy array aur PyTorch tensor **same memory share karte hain** (agar tensor CPU pe ho).
Matlab agar ek me change karo, to doosre me bhi reflect ho jayega!

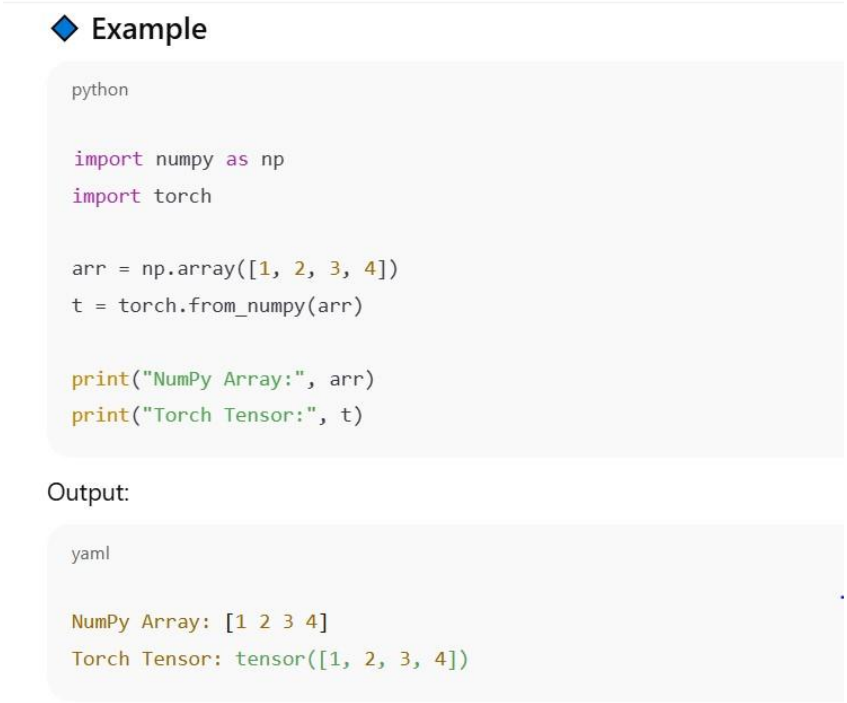
Example:

python — Copy code

```
arr = np.array([1, 2, 3])
t = torch.from_numpy(arr)

arr[0] = 99
print(t)   # tensor([99,  2,  3])  ← changed

t[1] = 88
print(arr) # [99 88  3]  ← changed
```

🔻 torch.from_numpy()

Ye function ek **NumPy array** ko directly **PyTorch tensor** me convert karta hai. Aur special baat ye hai ke **dono same memory share karte hain** (CPU pe).



🔷 **Example**

python

```
import numpy as np
import torch

arr = np.array([1, 2, 3, 4])
t = torch.from_numpy(arr)

print("NumPy Array:", arr)
print("Torch Tensor:", t)
```

Output:

yaml

```
NumPy Array: [1 2 3 4]
Torch Tensor: tensor([1, 2, 3, 4])
```

🔷 PyTorch Tensor → NumPy Array

🔷 Syntax

python

```
numpy_array = tensor.numpy()
```

```python
import torch

# PyTorch tensor
t = torch.tensor([10, 20, 30, 40])

# Convert to NumPy
arr = t.numpy()

print("Torch Tensor:", t)
print("NumPy Array:", arr)
print(type(arr))
```

```
Torch Tensor: tensor([10, 20, 30, 40])
NumPy Array: [10 20 30 40]
<class 'numpy.ndarray'>
```

.numpy() convert torch into numpy

🔷 Autograd

PyTorch ka **automatic differentiation engine** hai jo tensors ke gradients (derivatives) khud calculate karta hai → ye hi backpropagation ke liye use hota hai.

🔷 GPU Support

PyTorch me tensors aur models ko **CPU ke bajaye GPU (CUDA)** pe run karne ka feature hai, jisse deep learning training aur inference bohot fast ho jata hai.

👉 Chhoti si analogy:

- **Autograd** = automatic derivative machine 🖊️
- **GPU Support** = fast engine 🚀

Questions/;

1. PyTorch me tensor kya hota hai?

☞ Tensor ek **multi-dimensional array** hota hai (jaise scalar, vector, matrix ka general form). Ye PyTorch ka basic data structure hai aur GPU par bhi run kar sakta hai.

---

2. NumPy array aur Torch tensor me kya similarity hai?

☞ Dono hi multi-dimensional arrays hain. PyTorch tensors aur NumPy arrays easily **convert** ho sakte hain aur CPU pe hone par **memory share** karte hain (ek me change karo to doosre me bhi hota hai).

---

3. Tensor ke .dtype, .shape, aur .device ka kya kaam hai?

- .dtype → Data type batata hai (float32, int64, etc.)
- .shape → Tensor ka size batata hai (kitni rows × columns × dimensions).
- .device → Tensor CPU par hai ya GPU par, ye batata hai.

---

4. Autograd ka use kyu hota hai?

☞ Autograd PyTorch ka **automatic differentiation engine** hai jo tensors ke gradients (derivatives) khud calculate karta hai. Ye backpropagation aur training ke liye use hota hai.

---

5. GPU support PyTorch me kaise check karte hain?

☞ Function use karte hain: torch.cuda.is_available()

- True → GPU available hai
- False → GPU available nahi hai (CPU use hoga)

Q1. What is a linear regression model? Give an example.

Answer:
Linear regression ek simple ML model hai jo input aur output ke beech **straight line relation** find karta hai.

Formula:

y=w⋅x+by = w \cdot x + by=w⋅x+b

- w = weight (slope of line)
- b = bias (intercept)

Example:
House price predict karna using size.
Price=w⋅(Size)+bPrice = w \cdot (Size) + bPrice=w⋅(Size)+b

🔻 Q2. What are input and target variables in a dataset?

Answer:

- **Input (X):** Features jo model ko diye jate hain (independent variables). **Target (Y):** Output jo model predict karna hai (dependent variable).

Example:

- X = [size, rooms]
- Y = price

🔻 Q3. What are weights and biases in a linear regression model?

Answer:

- **Weights (w):** Input ka importance dikhate hain. **Bias (b):** Line ko shift karta hai (intercept).

  Example:

Salary = 2000 × Experience + 5000

- 2000 = weight
- 5000 = bias

🔻 Q4. How do you create dataset arrays using PyTorch tensors?

```
import torch
X = torch.tensor([[1], [2], [3]], dtype=torch.float32)  # inputs
Y = torch.tensor([[2], [4], [6]], dtype=torch.float32)  # outputs
```

🔻 Q5. Why do we create separate matrices for inputs and targets while training?

Answer:

- Inputs (X) → model ko dete hain
- Targets (Y) → ground truth hote hain jisse compare karke loss calculate hota hai Agar

dono ko mix kar denge to model ko sahi training nahi milegi.

🔻 Q6. How do you determine the shape of weight matrix & bias vector?

Answer:

- Agar X = (samples, features) aur Y = (samples, outputs)
- To weight matrix = (features, outputs)

- Bias vector = (outputs,)

Example:
3 features → 2 outputs
Weight shape = (3,2)
Bias shape = (2,)

🔻 Q7. How do you create randomly initialized weights & biases?

```
w = torch.randn(3, 2, requires_grad=True)  # random weights b
= torch.randn(2, requires_grad=True)    # random bias
```

🔻 Q8. How is linear regression model implemented using matrix operations? Ypred=X

.W+bY_{pred} = X \cdot W + bYpred=X.W+b

Example:
X = (10 samples × 3 features)
W = (3 × 2)
Output = (10 × 2)

🔻 Q9. How do you generate predictions using a linear regression model?

```
Y_pred = X @ W + b
```

@ = matrix multiplication

🔻 Q10. Why are the predictions of randomly initialized model different from actual targets?

Answer:
Kyuki shuru me weights & biases random hote hain → model abhi trained nahi hai. Training ke baad hi predictions actual targets ke paas aate hain.

🔻 Q11. What is a loss function? What does "loss" mean?
Answer:

Loss = error (difference between prediction & actual target). Loss function = formula jo error measure karta hai.

🔻 Q12. What is mean squared error (MSE)?

MSE=1n∑(ypred−ytrue)2MSE = \frac{1}{n} \sum (y_{pred} - y_{true})^2MSE=n1∑(ypred −ytrue)2

Example:
True = [2,4], Pred = [3,5]
MSE = ((3-2)² + (5-4)²)/2 = (1+1)/2 = 1

🔻 Q13. Write function to calculate MSE:

```
def mse(y_pred, y_true):      return
torch.mean((y_pred - y_true)**2)
```

🔻 Q14. What happens when you invoke .backward()?

Answer:
PyTorch **automatic differentiation** use karta hai →
.backward() gradients calculate karta hai w.r.t weights & biases.

🔻 Q15. Why is derivative of loss w.r.t weight matrix itself a matrix?

Answer: Kyuki har weight ka apna ek gradient hota hai → har weight ke liye "loss me change" calculate hota hai.
Matrix shape = same as weights.

🔻 Q16. How is derivative of loss w.r.t weight useful?

Answer:
Ye batata hai **kis direction me weight update karna hai**.

- Positive gradient → weight ko kam karo
- Negative gradient → weight ko badhao

🔻 **Q17. Suppose derivative is positive. What does it mean?**

Answer:

- Agar gradient **positive** hai → iska matlab weight zyada hai → loss ko kam karne ke liye weight ko **decrease** karna hoga.
- Agar gradient **negative** hai → weight ko **increase** karna hoga.

🔷 Vector wala Review Question

Usually woh poochte hain:

"Scalar, Vector, Matrix aur Tensor me kya difference hai?"

🔷 Easy Explanation (short style)

- **Scalar** → Single number (e.g., 5) → 0D tensor
- **Vector** → Numbers ki list (e.g., [1, 2, 3]) → 1D tensor
- **Matrix** → Rows × Columns (e.g., [[1,2],[3,4]]) → 2D tensor
- **Tensor** → Higher dimensions ka generalization (e.g., 3D cube of numbers) → nD tensor

---

```
import torch

scalar = torch.tensor(5)                # 0D
vector = torch.tensor([1, 2, 3])        # 1D
matrix = torch.tensor([[1, 2], [3, 4]]) # 2D
tensor3d = torch.rand(2, 3, 4)          # 3D
```

Output:

- scalar.shape → `torch.Size([])`
- vector.shape → `torch.Size([3])`
- matrix.shape → `torch.Size([2, 2])`
- tensor3d.shape → `torch.Size([2, 3, 4])`

Size and shape:

- `.size()` aur `.shape` **basically same kaam karte hain** — dono tensor ke dimensions (rows, columns, etc.) batate hain.
- **Difference sirf syntax ka hai:**
  - `.size()` → function hai, `x.size()` likhna padta hai
  - `.shape` → attribute hai, `x.shape` likh ke direct access kar sakte ho

Example:

```python
import torch

x = torch.randn(2, 3)

print(x.size())   # torch.Size([2, 3])
print(x.shape)    # torch.Size([2, 3])
```

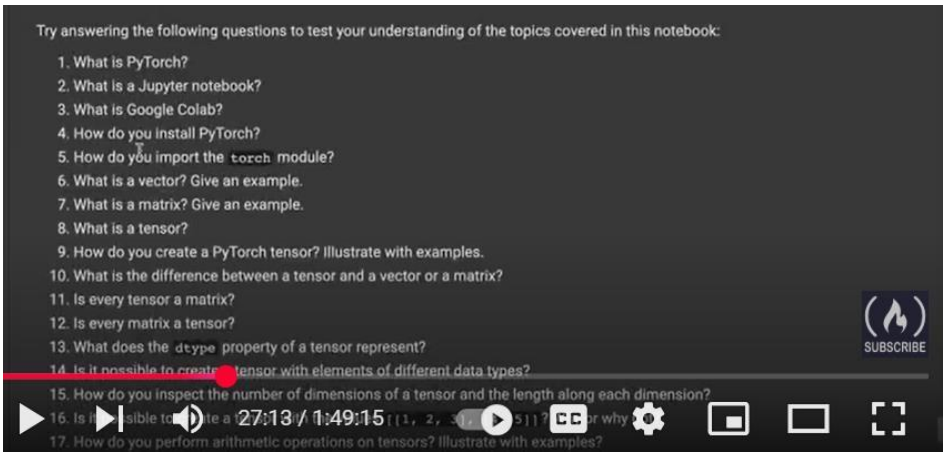
```python
import torch

x = torch.randn(2, 3, 4)  # 3D tensor
print(x.shape)
print(x)
```

```
torch.Size([2, 3, 4])
tensor([[[-2.2316e+00, -2.0513e-01, -1.3200e+00, -1.7012e-01],
         [-2.8397e-01, -5.6731e-01, -1.9346e+00,  3.5842e-01],
         [ 3.3225e-01,  8.9402e-01, -4.4929e-01,  6.3081e-01]],

        [[-1.0263e+00, -1.6825e+00, -3.1143e+00, -3.2943e-01],
         [ 3.1195e-01, -6.1415e-04, -2.3073e-01, -8.9516e-01],
         [ 8.7913e-01, -3.8628e-01,  7.5840e-01,  6.0111e-01]]])
```

Torch.rand

torch.rand PyTorch me ek function hai jo **random numbers generate karta hai**. Ye numbers **uniform distribution** se aate hain (0 se 1 ke beech).

Questions to reaview



Bilkul! 😊 Main aapko **Deep Learning workflow** step-by-step **Roman Urdu mai** samjha deta hoon, aur explain karta hoon ke PyTorch ya general deep learning mai kaise yeh har model mai use hota hai. Yeh **machine learning workflow se similar hai**, bas syntax aur model structure deep learning ke liye thoda alag hota hai.

Deep Learning ka General Workflow (Step by Step)

1️⃣ Data Import aur Preparation

- ML mai hum CSV ya Excel files import karte hain.
- Deep learning mai bhi yahi hota hai, lekin hum **PyTorch Tensor** ya **NumPy arrays** mai convert karte hain.

```
import torch import
numpy as np

# Example data
X = np.array([[20,50,30],[25,60,40]], dtype=np.float32)  # Features Y
= np.array([[70,40],[80,60]], dtype=np.float32)       # Targets

# Convert to torch tensors
X = torch.from_numpy(X)
Y = torch.from_numpy(Y)
```

2️⃣ Feature Scaling / Encoding

- Neural networks **feature scale sensitive** hote hain.
- Large differences mai training unstable ho sakti hai (NaN ya high loss).
- Is liye **StandardScaler** ya **MinMaxScaler** use hota hai.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X.numpy())
X_scaled = torch.from_numpy(X_scaled.astype(np.float32))
```

Categorical features hotay to one-hot encoding ya embedding use karte hain.

---

3 Train/Test Split

- ML mai jaise train_test_split hota hai, deep learning mai bhi same hota hai.
- PyTorch mai hum Tensor slicing ya torch.utils.data.DataLoader use karte hain.

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, Y, test_size=0.2, random_state=42)
```

---

4 Model Define Karna

- ML mai hum LinearRegression() use karte hain.
- Deep learning mai **nn.Module** subclass bana ke layers define karte hain.

```
import torch.nn as nn

class MyModel(nn.Module):     def
__init__(self, input_dim, output_dim):
super(MyModel, self).__init__()
      self.linear = nn.Linear(input_dim, output_dim)  # simple linear model

   def forward(self, x):
return self.linear(x)
```

- Complex model mai layers add karte hain: nn.Linear, nn.ReLU, nn.Conv2d etc. **Forward function** define karta hai data ka flow model mai.

5️⃣ Loss Function Choose Karna

- Regression ke liye: nn.MSELoss()
- Classification ke liye: nn.CrossEntropyLoss()

```
criterion = nn.MSELoss()
```

6️⃣ Optimizer Choose Karna

- Weights update karne ke liye optimizer chahiye. Common optimizers: SGD, Adam

```
import torch.optim as optim

optimizer = optim.SGD(model.parameters(), lr=0.01)
```

7️⃣ Training Loop

- Deep learning mai **fit()** nahi hota, hum **loop** se train karte hain:

```
num_epochs = 2000
for epoch in range(num_epochs):
   # Forward pass     outputs =
model(X_train)     loss =
criterion(outputs, Y_train)

   # Backward pass
optimizer.zero_grad()
loss.backward()     optimizer.step()

   if (epoch+1) % 200 == 0:         print(f'Epoch
[{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')
```

- ML mai fit() ek line mai hota hai, DL mai loop manually hota hai.

8️⃣ Evaluation / Prediction

- Training ke baad test data se evaluate karte hain:

```
with torch.no_grad():  # gradient calculation disable
predictions = model(X_test)     print(predictions)
```

- Classification mai argmax se labels nikalte hain.

9️⃣ Summary

Deep learning workflow basically yehi hota hai, **har model mai same pattern follow hota hai**:

1. Data Import → PyTorch Tensor mai convert
2. Feature Scaling / Encoding
3. Train/Test Split → optional but recommended
4. Model Define → nn.Module, layers, forward
5. Loss Function → criterion
6. Optimizer → weights update
7. Training Loop → forward + loss + backward + step
8. Evaluation / Prediction → with torch.no_grad()

Difference ML aur DL ka bas ye hai ke ML mai fit() aur predict() built-in hotay hain, DL mai loop aur forward/backward manually likhte hain.

```
# ======================

# 1️⃣ Libraries import

# ======================

import torch import torch.nn

as nn import torch.optim as

optim import numpy as np

from sklearn.preprocessing import StandardScaler from

sklearn.model_selection import train_test_split
```

```
# ======================
# 2 Dataset
# ======================
X = np.array([
    [20, 50, 30],
    [25, 60, 40],
    [30, 80, 50],
    [22, 55, 35],
    [28, 70, 45],
    [18, 40, 25],
    [26, 65, 50],
    [24, 60, 38],
    [29, 75, 48],
    [21, 52, 33]
], dtype=np.float32)

Y = np.array([
    [70, 40],
    [80, 60],
    [90, 75],
    [75, 45],
    [85, 65],
    [60, 35],
    [82, 68],
    [78, 55],
    [88, 72],
```

```
    [72, 42]
], dtype=np.float32)

# ======================
# 3 Feature Scaling
# ======================
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = torch.from_numpy(X_scaled.astype(np.float32))
Y = torch.from_numpy(Y)

# ======================
# 4 Train/Test Split
# ======================
X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, Y, test_size=0.2, random_state=42)

# ======================
# 5 Model Define
# ====================== class
AppleOrangeModel(nn.Module):     def
__init__(self, input_dim, output_dim):
super(AppleOrangeModel, self).__init__()
        self.linear = nn.Linear(input_dim, output_dim)  # simple linear regression
```

```
    def forward(self, x):
return self.linear(x)


input_dim = X_train.shape[1]   # 3 features: Temp, Rainfall, Humidity output_dim
= Y_train.shape[1]  # 2 targets: Apple, Orange


model = AppleOrangeModel(input_dim, output_dim)

# ======================
# 6 Loss & Optimizer
# ======================
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# ======================
# 7 Training Loop
# ======================
num_epochs = 2000 for epoch in
range(num_epochs):

    # Forward pass     outputs
= model(X_train)     loss =
criterion(outputs, Y_train)
```

```
    # Backward pass
optimizer.zero_grad()
loss.backward()     optimizer.step()


    if (epoch+1) % 200 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

# ======================
# 8 Evaluation / Prediction
# ====================== with
torch.no_grad():

    predictions = model(X_test)
    print("\nTest Predictions (Apple, Orange):\n", predictions.numpy())

# ======================
# 9 New Data Prediction
# ======================
new_data = np.array([[27, 68, 44]], dtype=np.float32)  # New weather data
new_data_scaled = scaler.transform(new_data) new_data_scaled =
torch.from_numpy(new_data_scaled)


with torch.no_grad():
    new_prediction = model(new_data_scaled)
```

```
print("\nPredicted Growth for new data (Apple, Orange):", new_prediction.numpy()) #
==========

# 10️⃣ Trained weights & bias

# ==========

print("\nTrained Weights:\n", model.linear.weight.data) print("Trained

Bias:\n", model.linear.bias.data)
```

1️⃣ import torch

- PyTorch ka main package
- Tensor create karna, operations perform karna, GPU support etc. ke liye use hota hai.

```
import torch

x = torch.tensor([1,2,3])
```

- Ye **NumPy jaise arrays** hai, lekin deep learning ke liye optimized.

2️⃣ import torch.nn as nn

- nn ka matlab **Neural Network** module
- Model ke **layers, loss functions, activations** yahan se aate hain ✅ Example:

```
import torch.nn as nn
```

```
linear = nn.Linear(3, 2)  # 3 input features, 2 outputs
activation = nn.ReLU() loss_fn = nn.MSELoss()
```

---

3 import torch.optim as optim

- Optimizer module, **weights update karne ke liye**
- Gradient descent, Adam, RMSProp etc. optimizer yahan se milte hain

```
import torch.optim as optim

optimizer = optim.SGD(linear.parameters(), lr=0.01)
```

---

4 import numpy as np

- Numerical Python library
- ML/DL mai **data arrays create, manipulate, math operations** ke liye use hoti hai
  PyTorch tensors ke liye **data conversion** bhi yahi se hota hai

```
import numpy as np

X = np.array([[1,2],[3,4]])
X_tensor = torch.from_numpy(X.astype(np.float32))
```

---

5 from sklearn.preprocessing import StandardScaler

- Feature scaling ke liye
- **Mean=0, Std=1** kar deta hai har column
- Neural network mai scaling se **training stable** hoti hai

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # scale features
```

---

6 from sklearn.model_selection import train_test_split

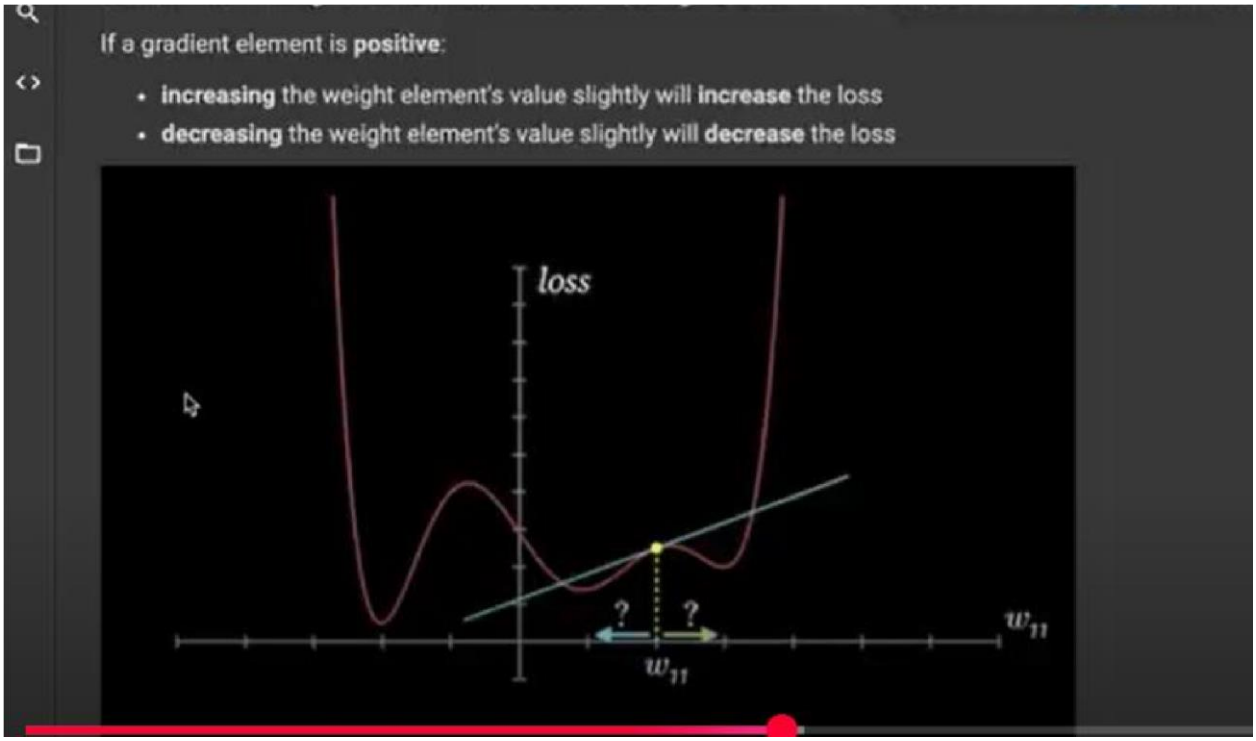
- Data ko **training aur testing set** mai divide karne ke liye
- Training data model train karega, test data evaluate karne ke liye

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, Y, test_size=0.2, random_state=42)
```

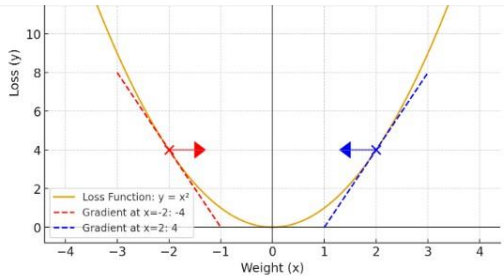
✅ Summary of Usage in DL Model

Library	Use
torch	Tensors, GPU, math operations
torch.nn (nn)	Layers, activations, loss functions
torch.optim (optim)	Optimizers (SGD, Adam, etc.)
numpy	Data arrays, conversions, math
StandardScaler	Normalize features (mean=0, std=1)
train_test_split	Split dataset into train/test sets

Weight loss



:hatGPT



Dekho graph 👆

- Curve = **loss function** $y = x^2$
- Red point = $x = -2$, yaha **gradient negative (-4)** hai
  - Arrow dikhata hai ke optimizer opposite direction (right side) me move karega → loss kam hoga.
- Blue point = $x = 2$, yaha **gradient positive (+4)** hai
  - Arrow dikhata hai ke optimizer opposite direction (left side) me move karega → loss kam hoga.

✅ Matlab:

- Gradient **positive ho** → slope upar hai, optimizer left me move karega.
- Gradient **negative ho** → slope neeche hai, optimizer right me move karega.

🔷 3. Why "opposite direction" always?

Gradient descent ka rule hai:

$$\text{New weight} = \text{Old weight} - \eta \cdot \text{Gradient}$$

- Agar gradient **positive ho** → weight **kam hoga** (left move).
- Agar gradient **negative ho** → weight **barhega** (right move).

Yani dono case me hum **gradient ke opposite direction** jate hain taake loss neeche aaye.

🔷 4. Analogy

Socho tum ek pahad (loss function) par ho:

- Gradient ka sign sirf ye batata hai: slope left side upar hai ya right side upar hai.
- Tum hamesha **neeche ki taraf (loss ↓)** chalna chahte ho → matlab slope ke **opposite side**.

🔷 **Rule simple hai:**

- Agar loss **barh raha hai** (upar ja rahe ho) → hamesha **opposite direction** me move karo.
- Agar loss **kam ho raha hai** (neeche ja rahe ho) → wahi direction maintain karo, kyunki aap already neeche ki taraf ja rahe ho.

Analogy (mountain example ⛰️)

- Agar aap mountain ke ek slope pe ho aur upar ki taraf chal rahe ho (loss ↑) → turant direction ulat do.
- Agar neeche ki taraf chal rahe ho (loss ↓) → bas isi raaste pe chalte raho jab tak valley (minimum loss) na mil jaye.

🔷 **Gradient Descent ka kaam yahi hai:**

- Gradient (positive/negative) follow karke **weights adjust karte rehna**
- Jab tak slope ≠ 0 (upar/neeche ka jhukav bacha hai), model seekhta rahega
- Jaise hi slope ≈ 0 ho jaata hai → matlab ab curve flat ho gaya → **aur seekhne ko kuch bacha nahi** → training ka goal achieve ho gaya

---

Ek line me:

👉 **Positive/negative updates tab tak karne hain jab tak slope zero ke paas na aa jaye = loss minimum point.**

Analogy ⛰️

Socho tum ball ko mountain se roll karte ho:

- Ball slope ke hisaab se left/right move karti hai (positive/negative).
- Jab valley bottom aata hai → slope flat ho jaata hai (0).
- Ball ruk jaati hai → yehi **training ka end point (optimized model)** hai.

with torch.no_grad():

🔷 Normal training me kya hota hai?

jab hum **forward pass** karte hain (model(X)), PyTorch **automatically gradients store karta hai** har operation ke liye.
kyun?
👉 taake baad me `loss.backward()` chalane par gradients calculate ho saken.

ye process kaafi heavy hota hai (zyada memory aur compute use karta hai).

🔷 `with torch.no_grad():` ka matlab

ye ek **context manager** hai jo PyTorch ko bolta hai:

> "abhi mujhe training nahi karni, gradients calculate mat karo!"

- matlab **gradients track nahi honge**
- memory aur compute bachega
- speed fast hogi

🔷 Kab use karte hain?

1. **Prediction / Testing phase**
   - jab hum model ko sirf **use kar rahe hain predict karne ke liye**, training nahi ho rahi
   - to hume gradients ki zaroorat hi nahi hoti
2. **Evaluation metrics** nikalte waqt
   - jaise accuracy, predictions, outputs dikhana

🔷 Example real world analogy

socho tum ek **student** ho jo math solve kar raha hai:

- **Training mode (without no_grad):**
  tum har step likh rahe ho kaise solve kiya, taake teacher check kare aur galti sudhare.
- **Prediction mode (with no_grad):**
  tum sirf final answer likh dete ho, steps track nahi karte — kyunki ab exam khatam ho gaya hai, seekhna nahi hai, sirf result dikhana hai.

🔷 Code example

python

```python
# Training mode (gradients track honge)
outputs = model(X_train)
loss = criterion(outputs, Y_train)
loss.backward()   # gradients calculate hote hain

# Prediction mode (gradients OFF)
with torch.no_grad():
    predictions = model(X_test)   # yahan gradients nahi banenge
```

Backward pass me kya hota hai?

jab hum loss.backward() chalate hain:

- PyTorch **gradients calculate karke unko jama (accumulate)** karta hai har parameter pe
  matlab agar tum loop me multiple batches chala rahe ho → pehle wale gradients bhi memory me jama rehte hain

---

🔻 Problem kya hai?

agar tum gradients ko reset nahi karoge →

- nayi batch ke gradients purane ke sath add ho jayenge
- result galat ho jaayega (over-updates, loss explode, NaN, etc.)

🔻 Solution

isliye har iteration ke start me likhte hain:

optimizer.zero_grad() ya model.zero_grad()

☞ iska matlab:
"pichle step ke gradients **0** kar do, nayi calculation fresh start se hogi."

🔻 Real-world analogy socho tum ek **whiteboard**

pe calculation kar rahe ho:

- agar tum naye question solve karne se pehle purane steps ko erase na karo → naye steps purane ke upar likh jaayenge → confusion hoga
- zero_grad() ka matlab hai → **whiteboard saaf kar do** har baar naye question ke liye

---

🔻 Important

- loss.backward() = gradients likhta hai (accumulate karta hai)
- optimizer.step() = un gradients se **weights update** karta hai
- zero_grad() = purane gradients erase karta hai

☞ Matlab har loop ka cycle hai:

1. zero_grad() → gradients reset
2. loss.backward() → naye gradients calculate
3. optimizer.step() → weights update

forward or backward paass

🔷 Forward pass

1. Input data → model layers se guzarta hai.
2. Har layer apna operation (linear, activation, etc.) apply karti hai.
3. Output (prediction) milta hai.

🔷 Backward pass

1. Loss (prediction vs target) ka gradient calculate hota hai.
2. Chain rule se har layer ke weights ke respect me derivatives nikalte hain.
3. Gradients update ke liye ready ho jaate hain (optimizer use karta hai).

👉 simple line:
Forward = predict karna, Backward = galti se seekhna.

chaho mai ek chhota **numerical example** bana kar dikhau jisme forward aur backward dono ek hi number pe samajh aajaye?

MSE

acha 🔌 **MSE** ka matlab hai **Mean Squared Error** → ye ek **loss function** hai jo regression problems me use hota hai.

🔷 Formula

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_{pred} - y_{true})^2$$

- y_{pred} = model ka prediction
- y_{true} = asal target (ground truth)
- n = total samples

🔷 Steps (simple words me)

1. Har prediction aur asal value ka **difference (error)** nikaalo.
2. Us error ko **square** karo (taake negative/positive cancel na ho jaye).
3. Sab squared errors ka **average** lo.

SGD(Stochastic Gradient Decent)

🔷 Parts samjho

1. `optim.SGD`
   - PyTorch ka ek optimizer hai
   - SGD = *Stochastic Gradient Descent*
   - iska kaam hai: **gradients use karke weights update karna**

---

2. `model.parameters()`
   - ye model ke **learnable parameters (weights + bias)** return karta hai
   - optimizer ko batata hai: "bhai, inhi cheezon ko tune karna hai training ke doraan"

---

3. `lr=0.01`
   - learning rate hai
   - matlab har step me gradient ke hisaab se kitna bada update karna hai
   - agar lr zyada → model jaldi seekhega (lekin galat bhi ho sakta hai)
   - agar lr bohot chhota → model dhire dhire se..nega

🔷 Real world analogy 🎯

socho tum ek **student** ho aur teacher tumhe galti batata hai:

- `gradients` = teacher ki correction (galti kitni aur kahan hui)
- `optimizer` = tumhari pencil/eraser jo galti sudharne ke liye use hoti hai
- `learning rate` = tum ek baar me kitna bada correction karte ho
  - chhota correction → safe lekin slow
  - bada correction → fast lekin risk zyada

🔷 Training loop me role

1. `loss.backward()` → gradients calculate ho gaye
2. `optimizer.step()` → un gradients se weights update ho gaye
3. `optimizer.zero_grad()` → purane gradients clear kar diye

👉 Matlab:

`optimizer = optim.SGD(...)` **ek tool hai jo gradients ki madad se model ke weights ko bar bar update karta hai taake loss kam hota rahe.**

Dataset and data loader

1. Dataset

- Dataset = features (X) + labels (Y) ko ek jagah organize karne ka tariqa.
- PyTorch me aap custom dataset bana sakte ho (torch.utils.data.Dataset inherit karke) ya built-in datasets (MNIST, CIFAR-10 etc.) use kar sakte ho.
- Matlab: *data ko "container" ki tarah store karna.*

python Copy code

```python
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y

    def __len__(self):
        return len(self.X)   # total samples

    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]   # ek sample wapas do
```

Dataloader:

2. DataLoader

- DataLoader dataset ko **batching, shuffling, aur parallel loading** ka support deta hai.
- Matlab: *aapke dataset ko training ke liye chhote chhote batches me tod kar ready karna.*
- Ye model ko ek saath thoda thoda data dene ke liye use hota hai (instead of ek hi baar sab).

python Copy code

```python
from torch.utils.data import DataLoader
import torch

# Example dataset (tensors)
X = torch.tensor([[1,2],[3,4],[5,6],[7,8]], dtype=torch.float32)
Y = torch.tensor([[0],[1],[0],[1]], dtype=torch.float32)

# Dataset class
class MyDataset(Dataset):
    def __init__(self, X, Y):
        self.X, self.Y = X, Y
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]
```

```
dataset = MyDataset(X, Y)

# DataLoader
dataloader = DataLoader(dataset, batch_size=2, shuffle=True)

for batch in dataloader:
    print(batch)   # har iteration me 2 samples milenge
```

🔑 Summary (2 line me)

- **Dataset** = data ko store aur organize karna (features + labels).
- **DataLoader** = dataset se batches nikal kar training loop ke liye ready karna.

Copy code

```
class FruitDataset(Dataset):
    def __init__(self, X, Y):
        self.X = X
        self.Y = Y
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]
```

1. `__init__`

python Copy code

```
def __init__(self, X, Y):
    self.X = X
    self.Y = Y
```

- Jab aap dataset object banate ho (`FruitDataset(X_train, Y_train)`), yahan `X` (features) aur `Y` (labels/targets) ko memory mein save kar diya jata hai.
- Matlab ab dataset object ke andar dono stored ↓, aur baad mein `__getitem__` unhe return karega.

2. `__len__`

```python
def __len__(self):
    return len(self.X)
```

- Ye function PyTorch ko batata hai ke dataset mein **kitne samples** hain.
- Example: agar `X_train` mein 8 rows hain → `len(train_dataset)` = 8
- `DataLoader` ko ye zaroori info chahiye hoti hai taake wo batches bana sake.

3. `__getitem__`

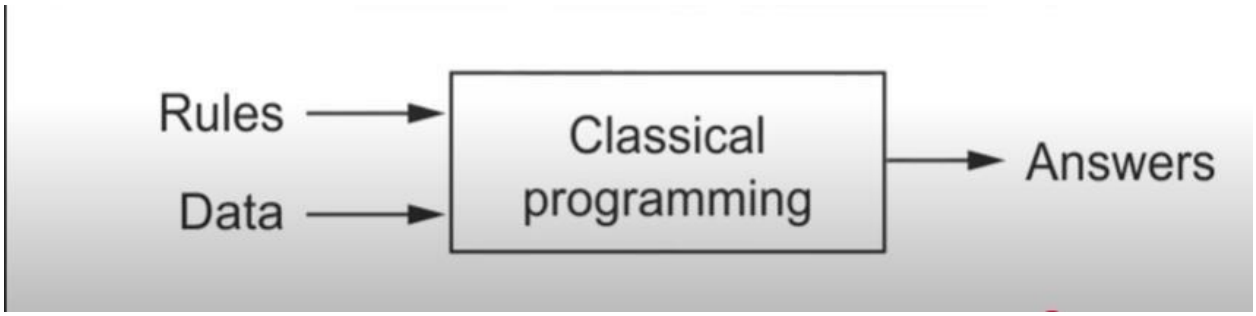
```python
def __getitem__(self, idx):
    return self.X[idx], self.Y[idx]
```

- Jab bhi PyTorch ek data sample chahiye hota hai, wo `__getitem__` call karta hai.
- `idx` ek integer hota hai (0 se `len(X)-1` tak).
- Ye function ek **tuple** return karta hai: `(features, label)`
  - Example: agar `idx = 0`, aur `X[0] = [20,50]`, `Y[0] = [70,40]` → ye return karega:

Chapter 5:

Classical Programming

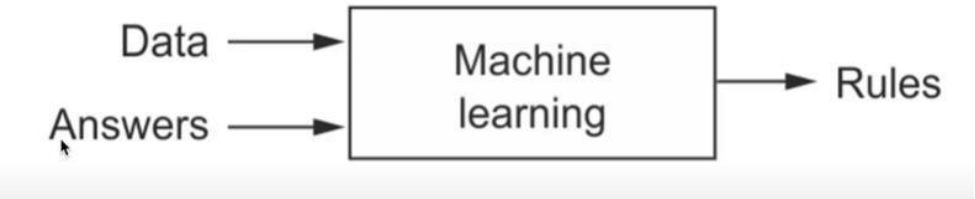
- **Kaise kaam karta hai?**
  - Human (programmer) rules banata hai → computer un rules ko follow karta hai.
  - Matlab: *"agar aisa input ho to aisa output do"* type programming.
- **Example:** ○ Spam email filter (classical way):
  Programmer manually likhega:
  - agar email me "FREE" ho → spam
  - agar "$$$" ho → spam ▪ warna → normal email **Problem:**
  ○ Rules banana mushkil hai agar data complex ho. ○ Manual effort zyada lagta hai.



Machine Learning:

- **Kaise kaam karta hai?**
  ○ Rules programmer nahi banata. ○ Programmer sirf data aur correct answers (labels) deta hai. ○ Algorithm khud "pattern" seekh leta hai. **Example:**
  - Spam email filter (ML way):
  - Input: 1000 emails (text)
  - Labels: spam / not spam
  - ML model automatically seekh lega ke kin words/frequency ke basis pe spam detect karna hai.

- **Problem:** ○ Feature engineering (sahi features manually choose karna) zaroori hota hai.



Deep Learning

- **Kaise kaam karta hai?**
  - Deep learning = special type of ML jo **Neural Networks** use karta hai.
  - Isme model khud **features bhi learn karta hai** → human ko manually select karne ki zaroorat kam ho jati hai.
- **Example:** ○ Spam email filter (Deep Learning way):
  - Input: raw email text
  - Model automatically **important words, context aur patterns** seekh lega bina manually feature engineering kiye.
  - Output: spam / not spam
- **Strength:** ○ Bada data aur complex problems ke liye best (images, speech, NLP).
- **Weakness:** ○ Zyada data aur compute power (GPU) chahiye hoti hai.

🔷 Summary Table

Aspect	Classical Programming	Machine Learning	Deep Learning
Rules banata hai	Human (manual)	Model (data se seekhta hai)	Neural network (data + features khud extract karta hai)
Data ki zaroorat	Kam	Zyada	Bahut zyada (big data)
Feature Engineering	Human karta hai	Human karta hai (important)	Model khud karta hai
Example	IF-ELSE spam filter	ML spam filter (keywords)	DL spam filter (text embeddings)

👉 simple shabdon mein:

- **Classical programming** = insaan rule banaye
- **ML** = insaan features banaye, model rules seekhe
- **DL** = model features bhi seekhe aur rules bhi se[illegible]e

---

Hidden layers:

- ek ya do **hidden layers** hongi,
- unke beech me **activation functions (ReLU)** use honge,
- aur last me **output layer** hogi.

example structure kuch aisa ho sakta hai:

```
scss

Input (3 features)
   ↓
Hidden Layer (8 neurons + ReLU)
   ↓
Hidden Layer (4 neurons + ReLU)
   ↓
Output Layer (2 outputs)
```

isse model non-linear relationships bhi seekh lega (jo linear regression se possible nahi).

🔷 Layer size decide karna kaise hota hai?

1. **Input layer**
   - Ye to fixed hoti hai → jitne features hain utne neurons.
   - Aapke dataset me 3 features → input_dim = 3.
2. **Output layer**
   - Ye bhi fixed hoti hai → jitne targets (labels) predict karne hain.
   - Aapke dataset me 2 targets (Apple, Orange) → output_dim = 2.
3. **Hidden layers (size aap choose karte ho)**
   - Ye trial-and-error + experience pe depend karta hai.
   - Small dataset → chhoti layers (e.g. 8, 4).
   - Large dataset (images, NLP, speech) → badi layers (128, 256, 512 neurons etc.).

Example Variations

- Aapke deep model me tha:
- nn.Linear(3, 8)
- nn.ReLU()
- nn.Linear(8, 4)
- nn.ReLU()
- nn.Linear(4, 2)
- Agar aap chaho to aisa bhi kar sakte ho:
- nn.Linear(3, 16)
- nn.ReLU()
- nn.Linear(16, 8)
- nn.ReLU()
  nn.Linear(8, 2)

Hidden layer ka size (8, 4, …) hum calculate nahi karte mathematically.
Ye ek **hyperparameter** hai → matlab experiment aur intuition se choose kiya jata hai.

🔹 Jo fixed hote hain

- **Input layer** = dataset ke features (aapke case me 3) ✅
- **Output layer** = dataset ke targets (aapke case me 2) ✅

Ye hamesha fix rahenge.

🔹 Jo hum choose karte hain

- **Hidden layer size** = hum khud decide karte hain based on:
  1. **Dataset ka size**
     - Chhota dataset → chhote hidden layers (8, 4, 16 neurons) ▪ Bada dataset → bade hidden layers (128, 256, 512 neurons)
  2. **Problem ki complexity**
     - Simple relation → chhoti layer
     - Complex relation (images, speech, NLP) → badi aur zyada layers
  3. **Trial and error (experiments)**
     - ML/DL me exact formula nahi hota.
     - Aap try karte ho 8 → 4, ya 16 → 8, aur check karte ho accuracy/loss.

🔹 Difference

- **self.linear** = single layer ka object (simple ML type model)
- **self.network** = ek pura sequence of layers (DL type model)

☞ Matlab:

- Agar **sirf ek layer** ho → self.linear
- Agar **multiple layers** ho → self.network (ya koi bhi naam aap de sakte ho, jaise self.layers, self.model)

Aapke model me:

```
nn.Linear(3, 8)   # Layer 1 (trainable)
nn.ReLU()         # Activation nn.Linear(8,
4)   # Layer 2 (trainable) nn.ReLU()
# Activation nn.Linear(4, 2)   # Layer 3
(trainable)
```

- **Linear layers** = 3 (yeh actual trainable layers hain)
- **ReLU functions** = 2 (yeh sirf activation functions hain , yai mathematical operation apply krti hai)

X_train.shape[1]

- X_train ek 2D tensor hai: (rows, columns)
- .shape[1] ka matlab = **kitne columns (features)** hain
- Aapke dataset me features = 3 → [feature1, feature2, feature3]
  ☞ Isliye: input_dim = 3

🔻 Y_train.shape[1]

- Y_train bhi ek 2D tensor hai: (rows, columns)

- .shape[1] = **kitne outputs (targets)** predict karne hain
- Aapke dataset me targets = 2 → [Apple, Orange] ☞ Isliye: output_dim = 2

🔽 **model = AppleOrangeDeepModel(input_dim, output_dim)**

- Yahan aap apna custom model bana rahe ho
- Ye **constructor ko values deta hai** jisse model samajhta hai:
  - **Input layer me 3 neurons** hone chahiye ○
    Output layer me 2 neurons hone chahiye

🔽 Example Inside Model

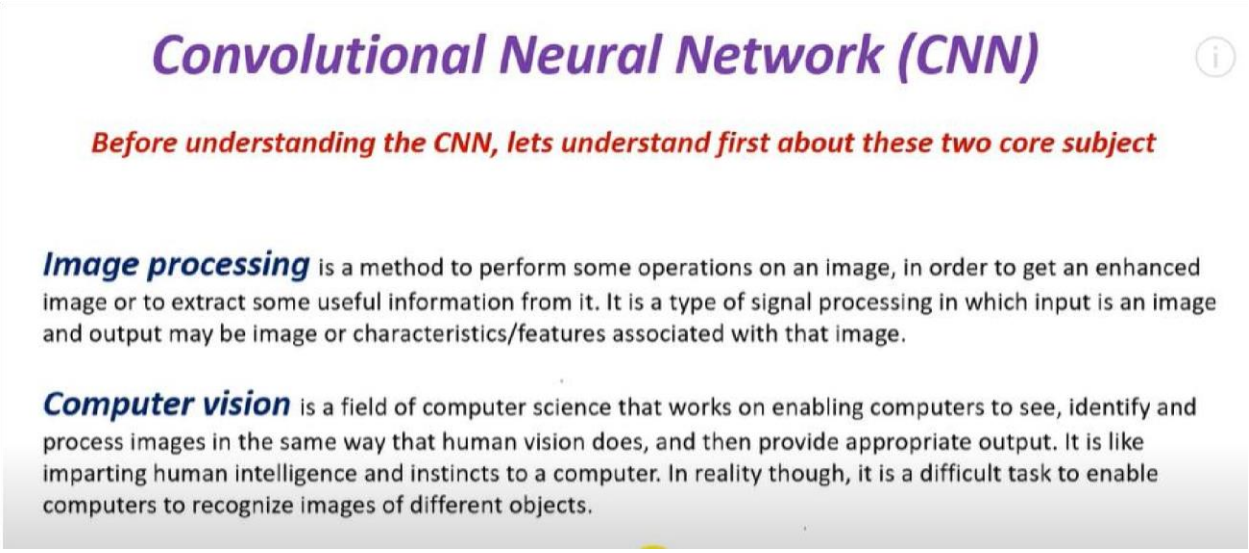
Aapke deep model ke andar likha tha:

```
nn.Linear(input_dim, 8)   # yaha input_dim = 3
...
nn.Linear(4, output_dim)  # yaha output_dim = 2
```

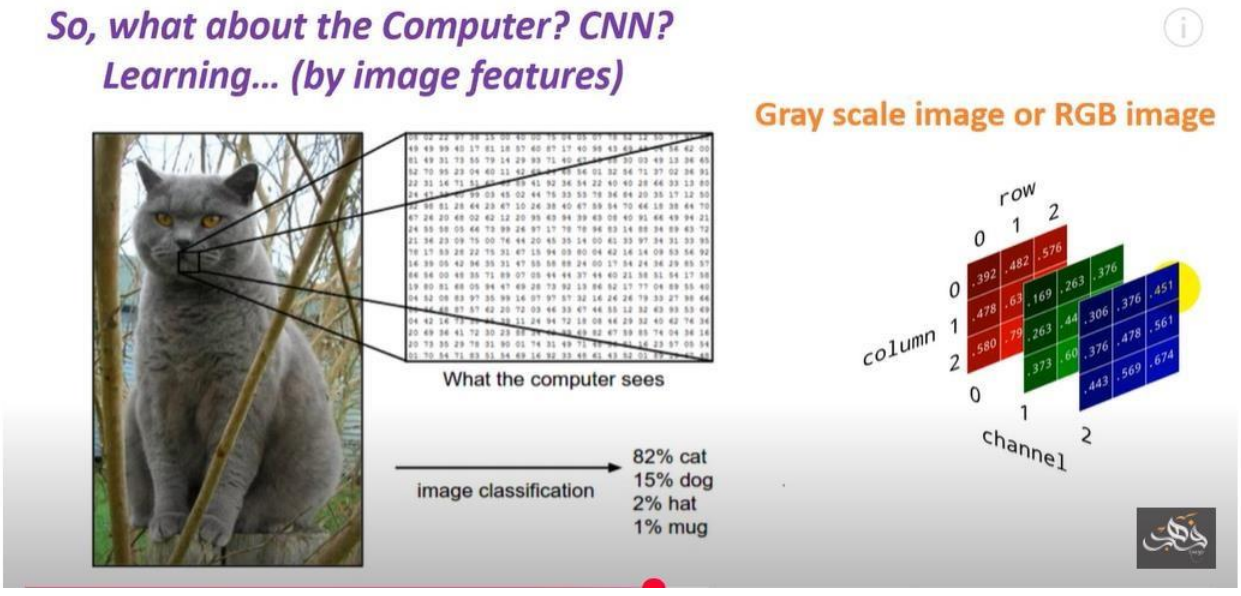
- Pehli Linear layer actually banegi: nn.Linear(3, 8)
- Last Linear layer banegi: nn.Linear(4, 2)

Chapter6

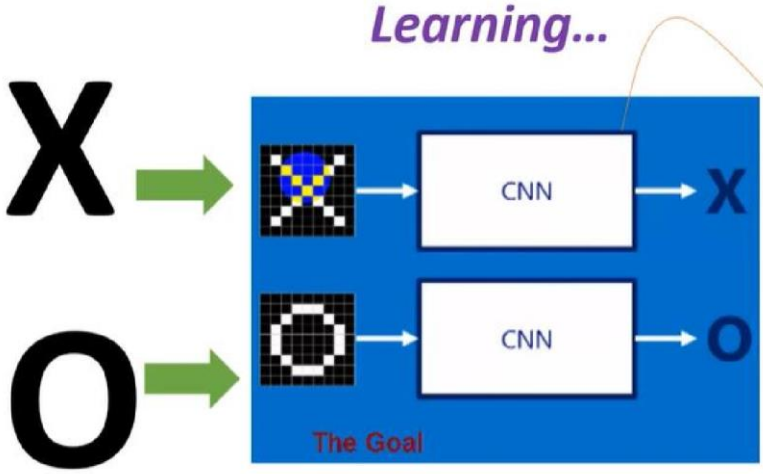
CNN (convolution Neural Network)



We have colours shade 0-255(RGB)



So, what about the Computer? CNN? Learning...



Here CNN work as like black box, so what is inside the black box!

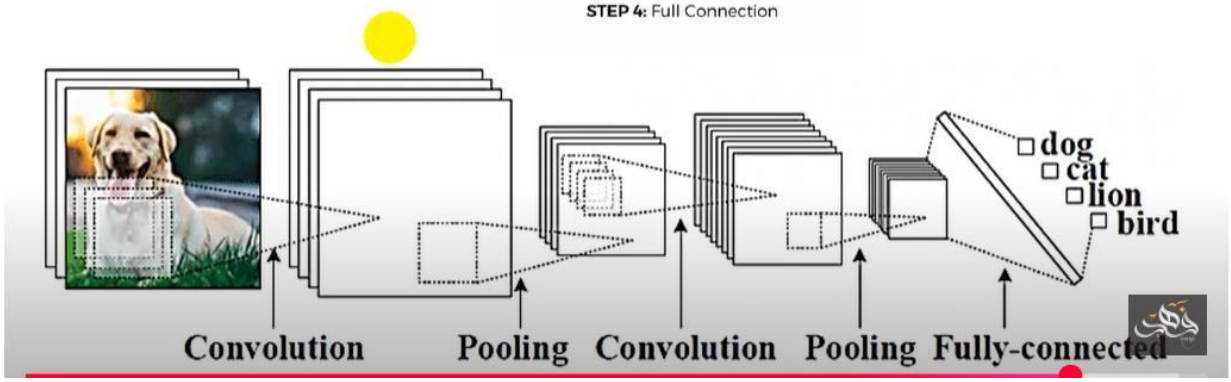
Steps in CNN

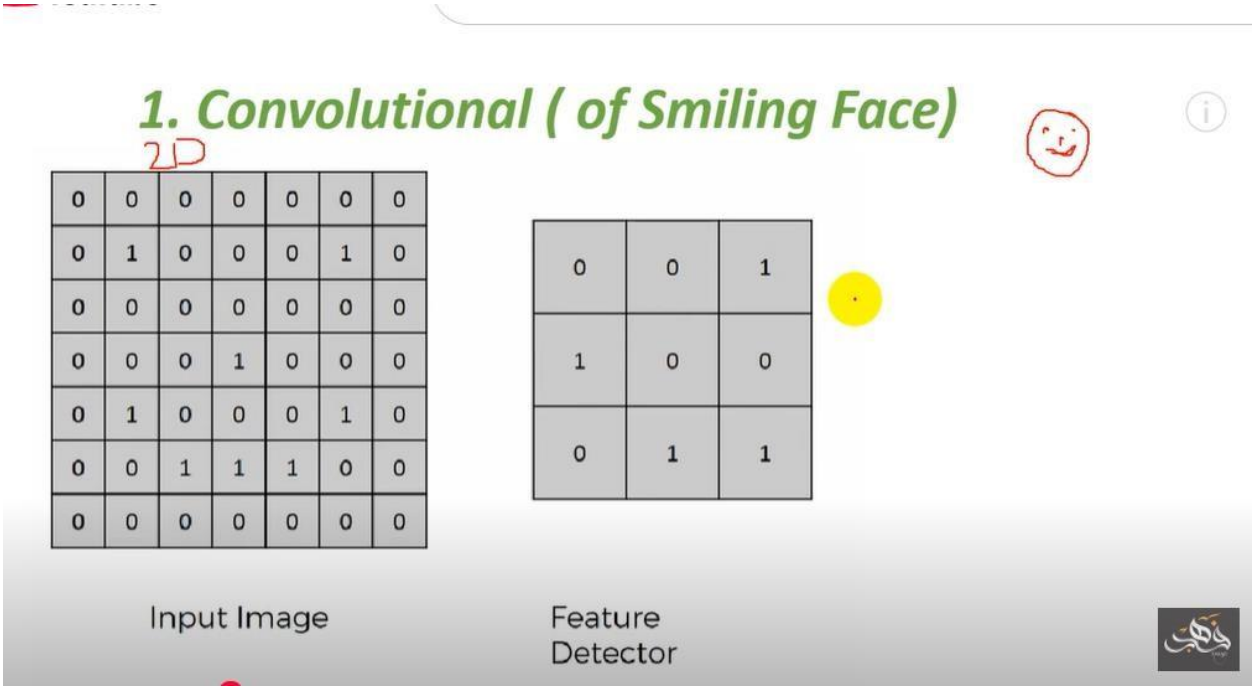
STEP 1: Convolution

STEP 2: Max Pooling

STEP 3: Flattening

STEP 4: Full Connection





Points:

Image ko first first grey scale pr convert krtai hai

Uskai bad uski two dimensions mai convert krtai hai

1 for black

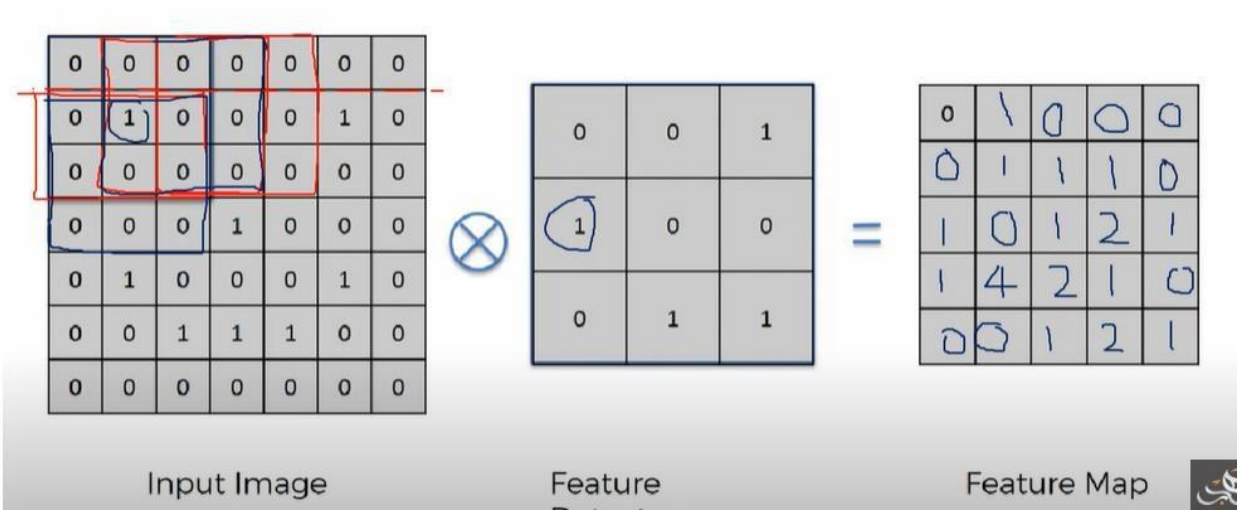
0 for white

Ok piyari kainat oper wali image ko ghor sai daikho yai aik smily face ki image hai jisko 0,1 mai convert kr k two dimension bnaya gaya hai.

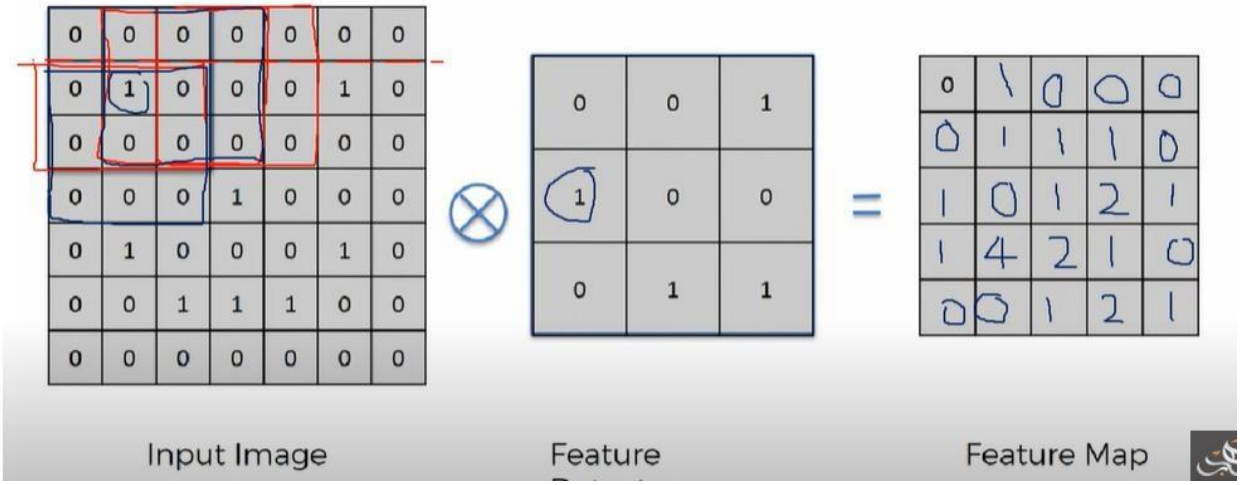
(Feature detector, kernel, and filter)

yai teeno same hi hai, kya karatai hai image mai sai pattern ko nikaal kr image ko compress krtai hai jaisai oper wali image mai sai pattern ko nikaal kr usai compress kiya gaya hai

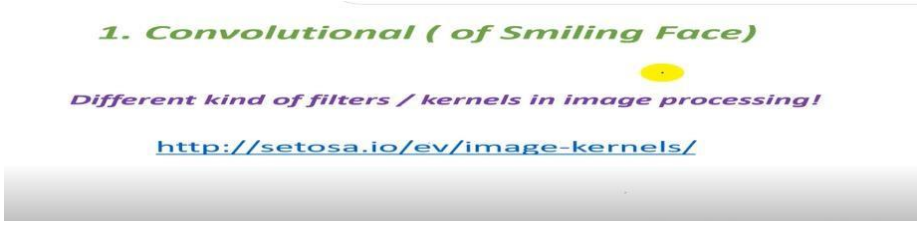
1. Convolutional (of Smiling Face)



1. Convolutional (of Smiling Face)



Kernals:



Koi aik kernak ka hm use nhi krtai image ko detect krnai kaliyai aik hoga jo pattern detect karai ga, dosra sharpness, teesra, noise istrha bahut sarai filter ka hm use krtai hai

Common Kernel Types & Their Work

1. Identity Kernel

[0 0 0
0 1 0
0 0 0]

☞ Image ko waise ka waise rehne deta hai (no change).

---

2. Edge Detection Kernels

- **Sobel / Prewitt kernels** → detect karte hain horizontal ya vertical edges.
- Example (Vertical Edge):

[-1 0 1
-2 0 2
-1 0 1]

☞ Image mein left aur right side ke sharp changes highlight karta hai.

- Example (Horizontal Edge):

[-1 -2 -1
0 0 0
1 2 1]

☞ Upar se neeche wali changes highlight karta hai.

---

3. Sharpening Kernel

[0 -1 0 -1
5 -1
  0 -1 0]

☞ Image ke edges aur details ko zyada sharp aur clear banata hai.

---

4. Box Blur / Average Kernel

[1/9 1/9 1/9
 1/9 1/9 1/9
 1/9 1/9 1/9]

☞ Har pixel ke aas-paas ke 9 pixels ka average nikalta hai → result = smooth / blurred image.

---

5. Gaussian Blur Kernel

[1 2 1
 2 4 2
 1 2 1] * (1/16)

☞ Normal blur se better, natural smoothness deta hai (camera blur jaise).

---

6. Emboss Kernel

[-2 -1 0
 -1 1 1
  0 1 2]

☞ Image ko 3D emboss (raised surface) jaise effect deta hai.

🔻 CNNs Mein Kaise Kaam Karte Hain?

- Jab CNN train hota hai, to yeh filters **automatically learn karta hai** (aapko manually define karne ki zarurat nahi).
- Initial layers → mostly **edge detection** / basic patterns learn karte hain.
- Deeper layers → complex features (eyes, nose, textures, objects).

☞ Matlab: Manual kernels (jaise Sobel, Gaussian) image processing mein fixed hote hain.
☞ CNN ke kernels training ke dauraan **data se seekhe jaate hain**.

Sharpen filter

Purpose:

Image ko zyada clear aur detailed banata hai → edges aur textures ko highlight karta hai.

Blurry image ko thoda "teekha" (sharp) karne ke liye use hota hai.

◆ Example Kernel

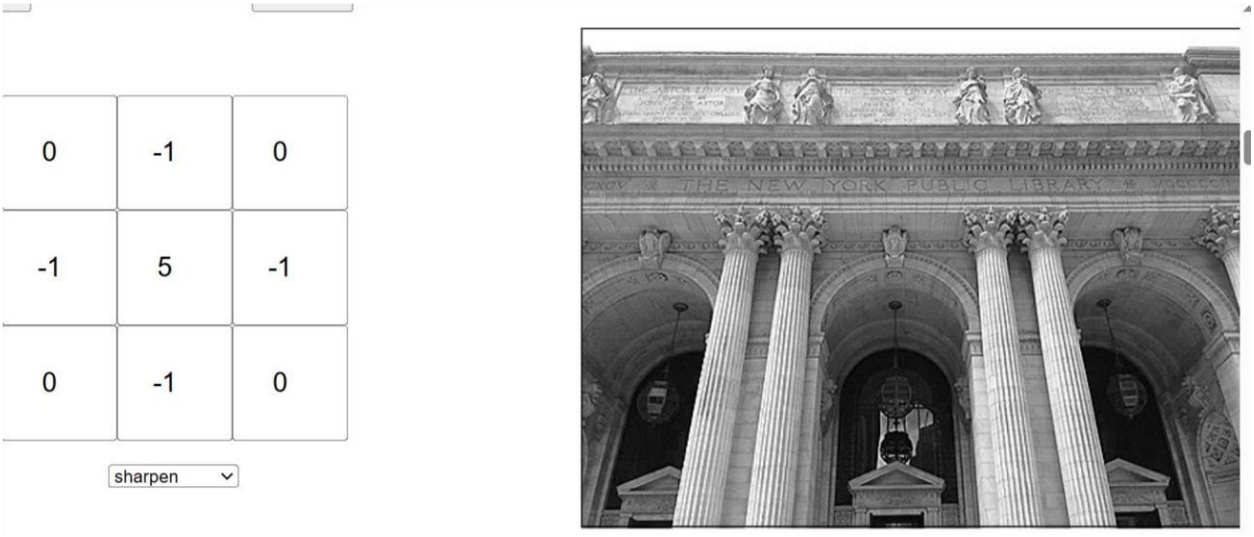
ini

```
[ 0 -1  0
 -1  5 -1
  0 -1  0]
```

- **5** center pixel ka weight zyada hai → matlab original pixel ko emphasize karo.
- **-1** neighbors subtract karte hain → edges highlight hote hain.

Result:

- Jahan image smooth hai, wahan farq kam hoga.
- Jahan edges hain (jaise outline), wahan bohot zyada contrast aayega → image sharp ho jaati hai.



Blur filter:

Aap **Blur Filter** (ya smoothing filter) ki baat kar rahe ho. Yeh filter image ko **soft aur smooth** banata hai, jisme noise ya sharp edges thode "mild" lagte hain.

🔹 Blur Filters (Types)

1. Box Blur / Average Filter

Sabse simple blur:

```
[1/9  1/9  1/9
 1/9  1/9  1/9
 1/9  1/9  1/9]
```

☞ Har pixel ka naya value uske 3×3 neighbors ka **average** hota hai.
Result: Soft blur (basic smoothing).

---

2. Gaussian Blur Filter

Natural looking blur ke liye use hota hai:

```
[1  2  1
 2  4  2
 1  2  1] * (1/16)
```

☞ Center pixel ko zyada weight, aur door walon ko kam weight deta hai.
Result: Camera jaise smooth blur.

---

3. Motion Blur Filter

Motion effect dene ke liye:

```
[1/5  0    0    0    0
 0    1/5  0    0    0
 0    0    1/5  0    0
 0    0    0    1/5  0
 0    0    0    0    1/5]
```

☞ Aisa lagta hai jaise image move ho gayi ho (motion trail effect).

🔹 Practice Karne Ke Liye

Aap ye kernels try kar sakte ho:

☞ Image Filters Practice Tool

- Wahan “kernel” box me copy–paste karke **blur effect live dekh sakte ho**.

⚡ Short Summary:

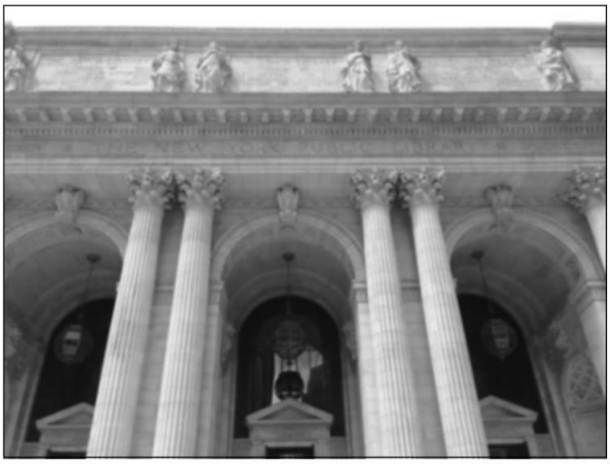
- **Box Blur** → Simple average, fast but not natural.
- **Gaussian Blur** → Realistic smooth blur, zyada use hota hai.
- **Motion Blur** → Movement effect banata hai.

Here's a playground were you can select different kernel matrices and see how they effect the original image or build your own kernel. You can also upload your own image or use live video if your browser supports it.

Choose File No file chosen Live video

0.0625	0.125	0.0625
0.125	0.25	0.125
0.0625	0.125	0.0625

blur



Aapke screenshot mein jo **kernel matrix** dikh raha hai:

Copy code

```
0.0625   0.125   0.0625
0.125    0.25    0.125
0.0625   0.125   0.0625
```

Yeh actually ek **Gaussian Blur Kernel** (3×3) hai ✅

🔷 **Kyun?**

- Center pixel (0.25) ka weight sabse zyada hai → matlab image ka main detail preserve karo.
- Neighboring pixels (0.125, 0.0625) ka weight gradually kam hota ja raha hai → matlab smooth transition create karo.
- Total sum = **1.0**, jo normalize kernel ka feature hai.

Result: Image blur hoti hai lekin **natural aur smooth** tarike se (camera blur jaisa), jo simple **Box Blur** se zyada realistic lagta hai.

d

Sobel Filter:

Sobel Filter (Edge Detection)

Purpose:
Sobel filter ek **edge-detection kernel** hai jo image mein **edges (teekhi boundaries)** highlight karta hai.

- Horizontal Sobel → upar–neeche ke changes detect karta hai.
- Vertical Sobel → left–right ke changes detect karta hai.

✵ Sobel Kernels

1. Vertical Sobel (detects vertical edges):

[-1 0 1
-2 0 2
-1 0 1]

☞ Agar image mein left se right direction mein color/intensity change hai, to ye filter usse highlight karega.

---

2. Horizontal Sobel (detects horizontal edges):

[-1 -2 -1
0 0 0
1 2 1]

☞ Agar image mein upar–neeche direction mein change hai, to ye highlight karega.

---

3. Combined Sobel (magnitude):
Dono filters (horizontal + vertical) ka result combine karke final edge image banta hai:

G=(Gx2+Gy2)G = \sqrt{(G_x^2 + G_y^2)}G=(Gx2+Gy2) jahaan GxG_xGx = vertical

sobel, GyG_yGy = horizontal sobel.

Effect

- Original image → normal photo.
- After Sobel → sirf **edges (boundaries)** visible hote hain, jaise sketch/outline.

Here's a playground were you can select different kernel matrices and see how they effect the original image or build your own kernel. You can also upload your own image or use live video if your browser supports it.

Choose File No file chosen Live video

1	2	1
0	0	0
-1	-2	-1

top sobel

Choose File No file chosen Live video

-1	0	1
-2	0	2
-1	0	1

right sobel

kernel. You can also upload your own image or use live video if your browser supports it.

Choose File No file chosen Live video

-1	-2	-1
0	0	0
1	2	1

bottom sobel

Outline Filter (Edge Highlighting)

Purpose:
Image ke andar sirf **edges** ko dikhata hai, baaki flat areas ko almost black/blank kar deta hai → result ek **sketch / outline drawing** jaisa lagta hai.

◈ Common Kernels

1. **Laplacian Kernel (most common for outline):**

[0 -1 0 -1
4 -1
0 -1 0]

☞ Intensity changes ko detect karta hai → outlines visible.

---

2. **Another Variant (8-direction Laplacian):**

[-1 -1 -1
-1 8 -1
-1 -1 -1]

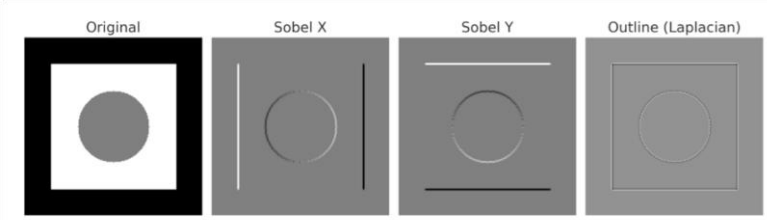
☞ Har direction (top, bottom, left, right, diagonal) ke edges detect karta hai.

◈ Effect

- Original Image → Normal photo.
- After Outline Filter → Sirf white/bright lines jahan sharp change hai, baaki dark ho jaata hai.

◈ Relation with Sobel

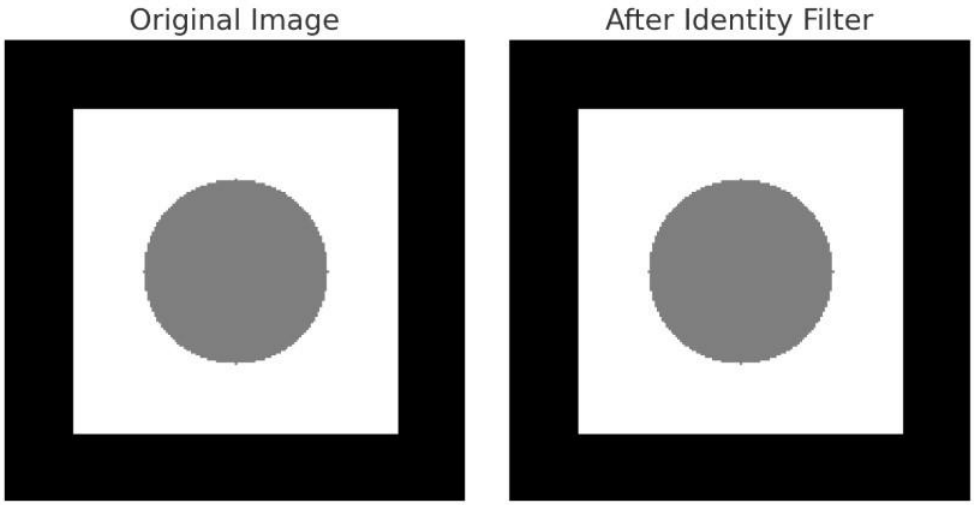
- **Sobel** → edges detect karta hai but direction-specific (horizontal ya vertical).
- **Outline (Laplacian)** → sari directions ka ek saath edge detect karke ek *pure outline* deta hai.



Yeh dekhiye 👇

- **Original Image** (rectangle + circle)
- **Sobel X** → left-right edges highlight
- **Sobel Y** → top-bottom edges highlight
- **Outline (Laplacian)** → sab directions ka ek saath edge, pure boundary (sketch jaisa)

Identity filter



Yeh dekhiye 👇

- **Original Image** aur
- **After Identity Filter**

Dono bilkul **same** hain ✅

Identity filter ka kaam hi sirf input ko as-is pass karna hota hai, bina koi change kiye.

Emboss filter:

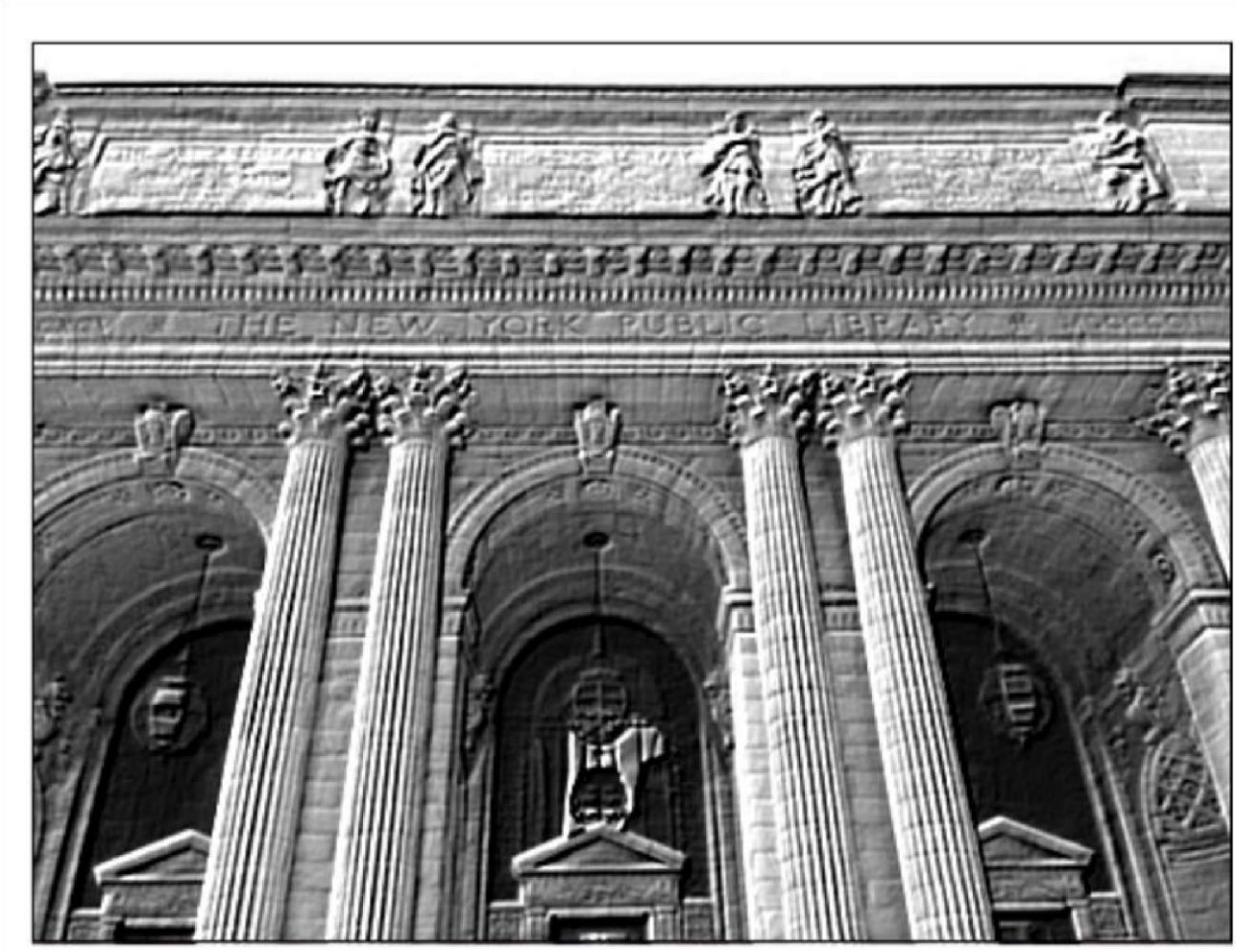
🔷 Emboss Filter

Kernel Example:

```ini
[-2 -1  0
 -1  1  1
  0  1  2]
```

🔶 Kaam

- Image ke edges detect karta hai aur unko aisa highlight karta hai jaise **3D raised effect (embossed look)** ho.
- Bright aur dark shading aati hai → image thodi **relief sculpture** jaisi lagti hai.



Custom filter:

🔷 Custom Filter (Kernel)

Idea:

- Aap apna khud ka kernel (matrix) design kar sakte ho.
- Har value batati hai ki center pixel ke neighbors ko kitna weight mile.
- Is tarah se aap **blur, sharpen, edge detect, emboss, motion blur... ya apna unique effect** bana sakte ho.

🔸 Example: Custom Kernel

Agar aap chahte ho ki image thoda sharpen ho lekin thoda blur bhi ho, to aap mix kernel bana sakte ho:

```ini
[ 0  -0.25   0
 -0.25  2.0 -0.25
  0  -0.25   0 ]
```

👉 Ye kernel edges ko thoda sharp karega, saath hi flat regions ko smooth bhi karega.

Chapter 6

Pooling:

Pooling ka Purpose

1. **Feature Map ko chhota karna** (downsampling)
   → computation kam ho jata hai, training fast hoti hai.
2. **Important features ko preserve karna**
   → Jaise ek image thoda shift ho jaye to bhi CNN usko sahi identify kare.
3. **Overfitting kam karna**
   → Kyunki pooling less important details hata deta hai aur sirf strong signals rakhta hai.

🔻 Types of Pooling

1. Max Pooling

- Har chhote block (e.g., 2×2) me se **sabse badi value select** karta hai. □ Matlab: sabse strong feature ko preserve karega. ☞ Example:

Block = [1, 3; 2, 5] → Max = 5.

- **Pros**: ○ Strong edges / features detect karne ke liye best. ○ Zyada use hota hai in practice (almost har CNN me).

---

2. Average Pooling

- Har block ka **average nikalta hai**.
- Matlab: sabhi values ka effect equal hota hai.

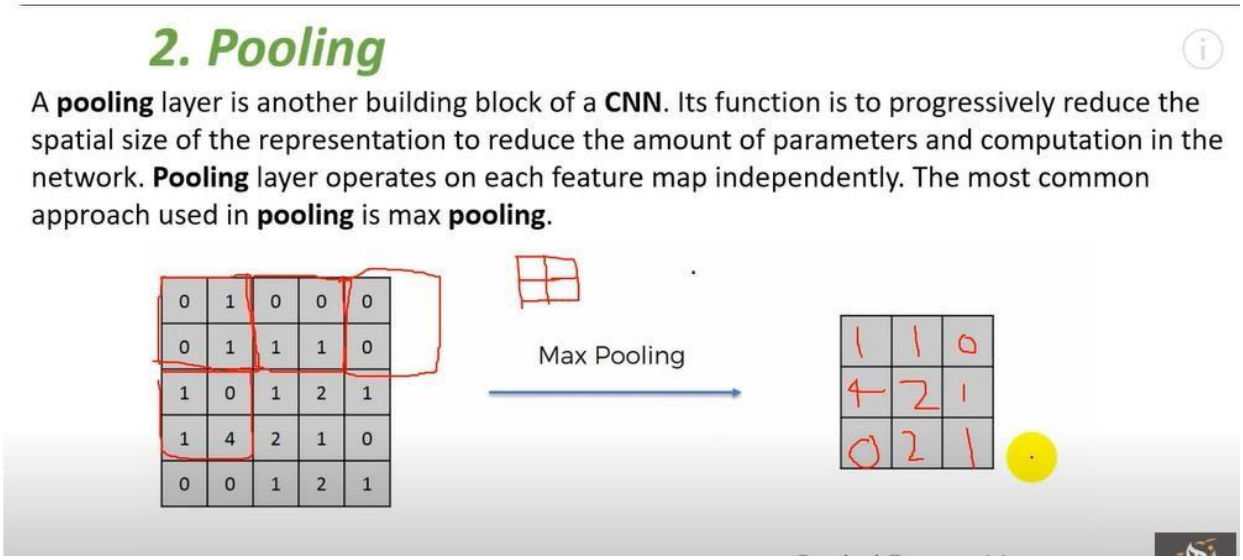
☞ Example:
Block = [1, 3; 2, 5] → Average = (1+3+2+5)/4 = 2.75.

- **Pros**:
  - ○ Smooth feature maps deta hai. ○ Kabhi kabhi zyada detail preserve kar leta hai (lekin max pooling se kam sharp hota hai).

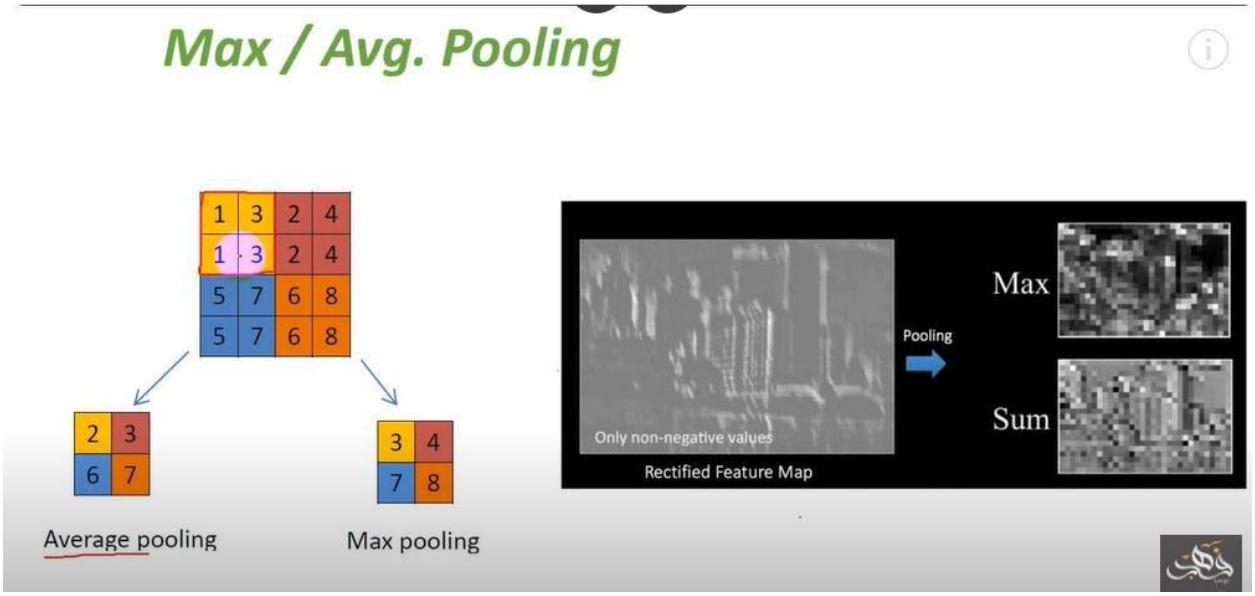
🔻 Which is Best?

- **Max Pooling** → Zyada commonly use hota hai, kyunki strong features ko capture karta hai.

- **Average Pooling** → Kabhi kabhi research/experiments me use hota hai, lekin deep CNN architectures (VGG, ResNet, etc.) me mostly **Max Pooling** hi prefer hota hai.

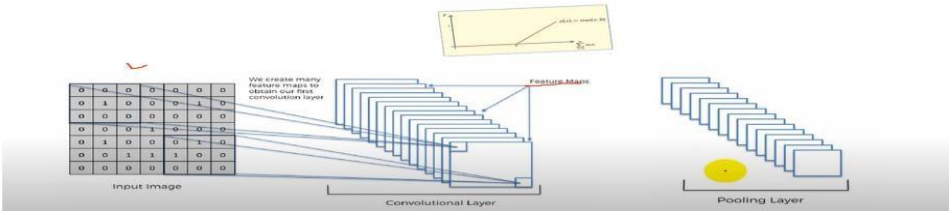


Average/max pooling



Max pooling is good give you detail structure

Pooling...


We create many feature maps to obtain our first convolution layer
Feature Maps
Input Image
Convolutional Layer
Pooling Layer

Chapter 7

Flatening:

🔷 3. Flattening

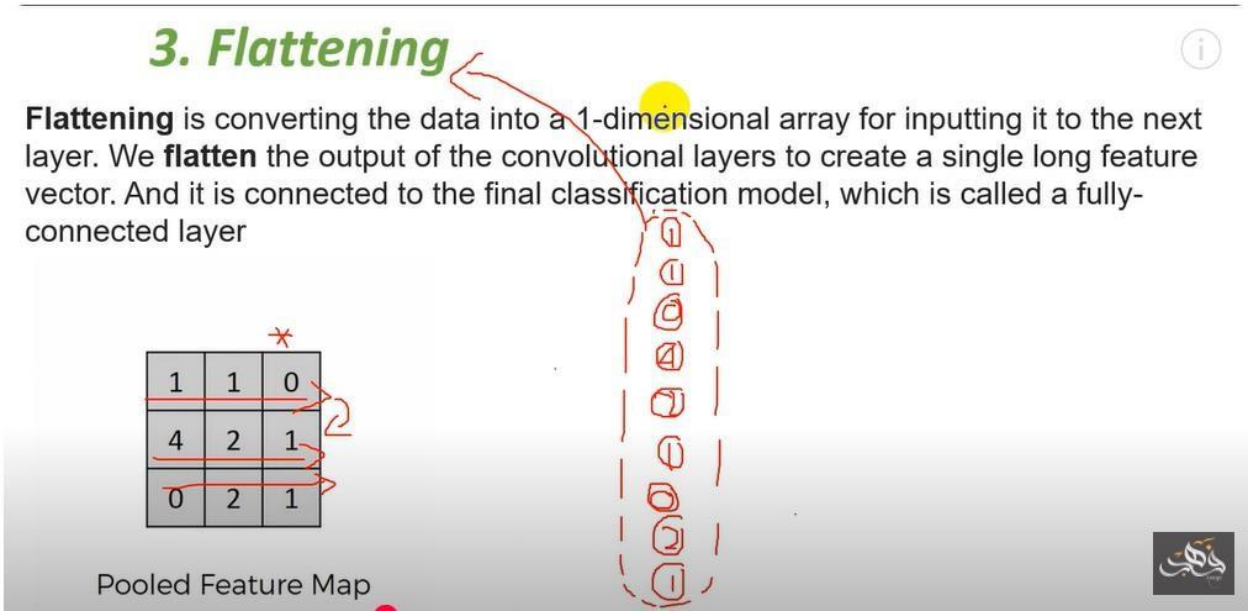
- **Kya hota hai:** Jo 2D feature maps hain (matrix form), unko ek **single long vector** me convert karna.
- **Kaam:** Taake aap usko **Fully Connected Layer** (jo ANN jaisa hota hai) me bhej sako.

👉 Example:

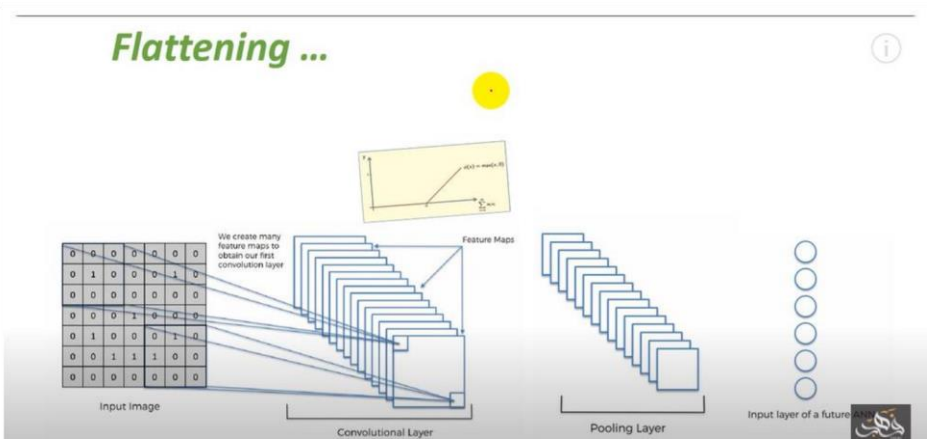
Feature map (2×2) =

```csharp
[7, 5]
[2, 3]
```

Flatten → `[7, 5, 2, 3]`



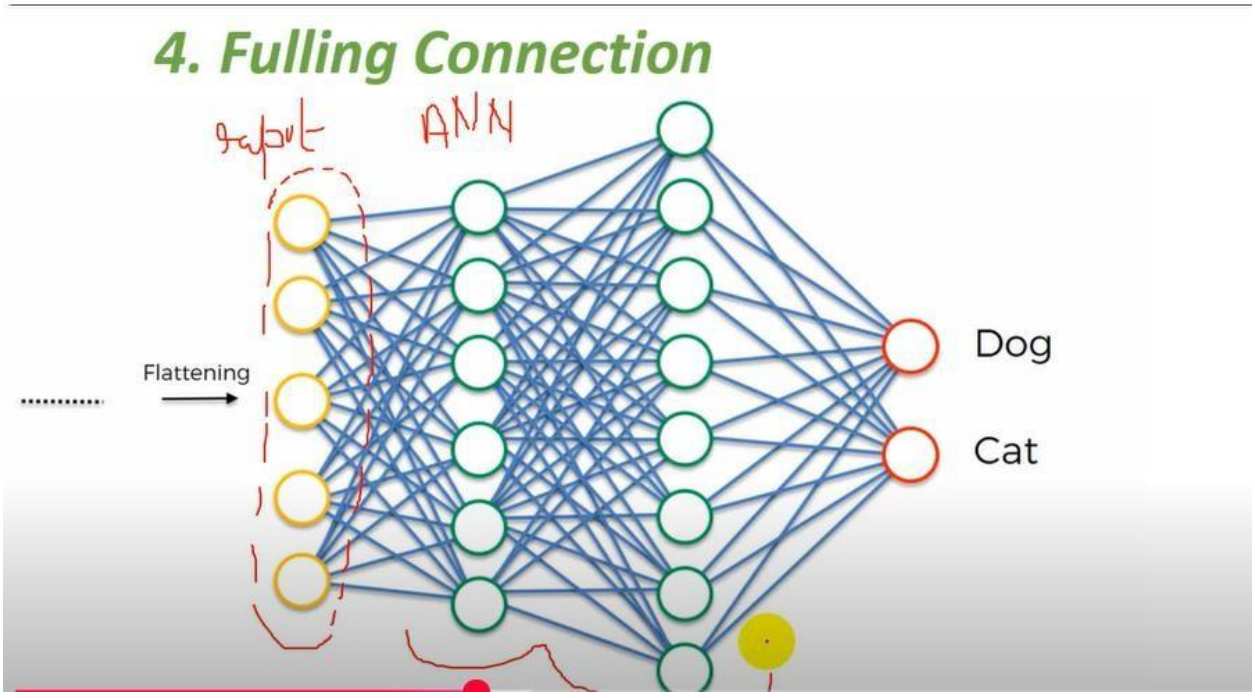
Flattening mai jb data convert hota hai to wo again input bn jata hai uskai bad us prANN apply hota hai jis mai hidden layers hoti hai wo phir classification hogi.



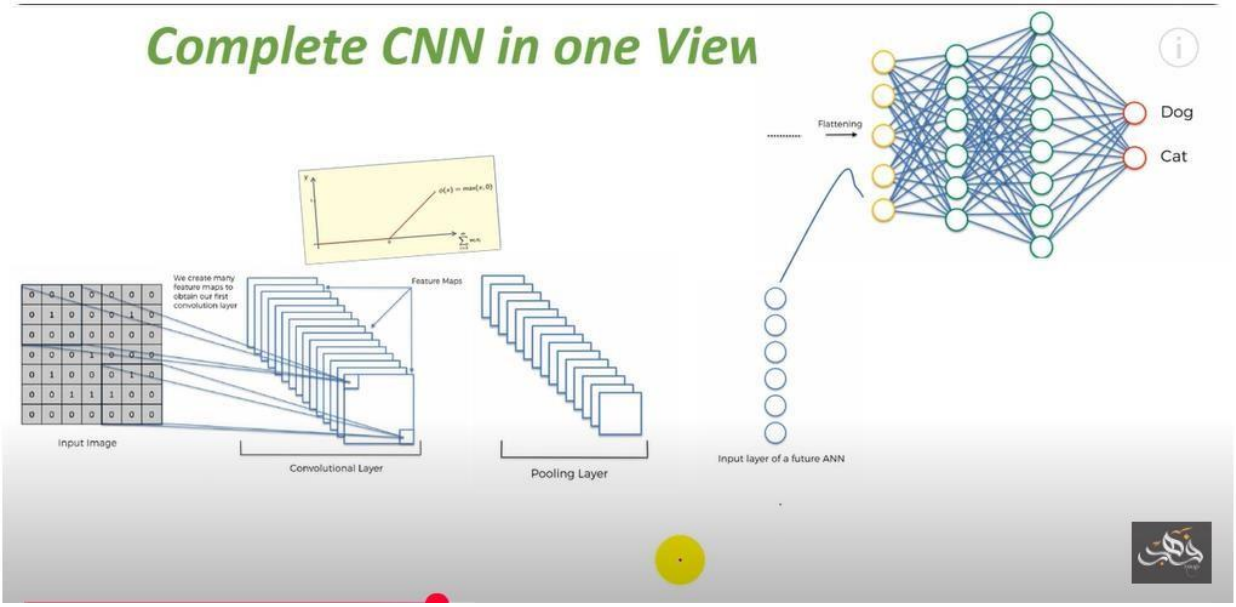
4. Fully Connected Layer (Dense Layer)

- **Kya hota hai:** Ye normal ANN layers hain jo last me **final prediction** dete hain (jaise cat/dog ya digit 0–9).
- **Kaam:** Jo features convolution ne nikale unko combine karke decision lena.

☞ Example:
Flattened vector [7, 5, 2, 3] → ANN → Output: "This is a dog 🐶".



Final image:



🔷 CNN Simple Flow (Diagram)

```
SCSS


Input Image (28x28)
      ↓
 Convolution (filters detect edges)
      ↓
 ReLU Activation (non-linearity)
      ↓
 Max Pooling (reduce size)
      ↓
 Flatten (2D → 1D vector)
      ↓
 Fully Connected Layer
      ↓
 Output (class: Cat 🐱 / Dog 🐶)
```

📌 Easy Analogy:

- **Convolution** = jaise magnifying glass se image ke chhote details dekhna.
- **Max Pooling** = har chhote block se best feature pick karna.
- **Flattening** = sari cheeze ek line me likh dena.
- **Fully Connected** = final decision lena based on all features.

Chapter 8

Softmax classification function:

🔺 Softmax Activation Function

- Softmax ek **activation function** hai jo mostly **last (output) layer** me use hoti hai **classification problems** ke liye.
- Ye raw numbers (logits) ko **probabilities** me convert karti hai jo hamesha **0 aur 1 ke beech** hote hain, aur sabka **sum = 1** hota hai.

🔺 Formula (sirf idea ke liye)

Agar outputs = [z1, z2, z3] to
softmax unko convert karega:

softmax(z1) = exp(z1) / (exp(z1) + exp(z2) + exp(z3)) softmax(z2) = exp(z2) / (exp(z1) + exp(z2) + exp(z3)) softmax(z3) = exp(z3) / (exp(z1) + exp(z2) + exp(z3))

🔺 Example

Maan lo model ne output diya (logits):

[2.0, 1.0, 0.1]

Softmax lagane ke baad:

[0.65, 0.24, 0.11]

👉 Matlab:

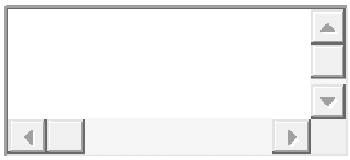
- Class 1 probability = **65%**
- Class 2 probability = **24%**
- Class 3 probability = **11%**

Ab tum easily sabse badi probability ko **prediction** maan loge.

🔻 Summary

- **Kya karta hai?** Numbers ko probabilities me badalta hai.
- **Kahan use hota hai?** Multi-class classification ke output layer me (e.g., cat/dog/horse).
- **Kyoon use karte hain?** Taake model ke decision ko probability form me samjh sakein.

☞ Bhai chaho to main tumhe **Softmax ka ek chhota code example (PyTorch)** likh ke dikhaun jo raw logits → probabilities convert kare?



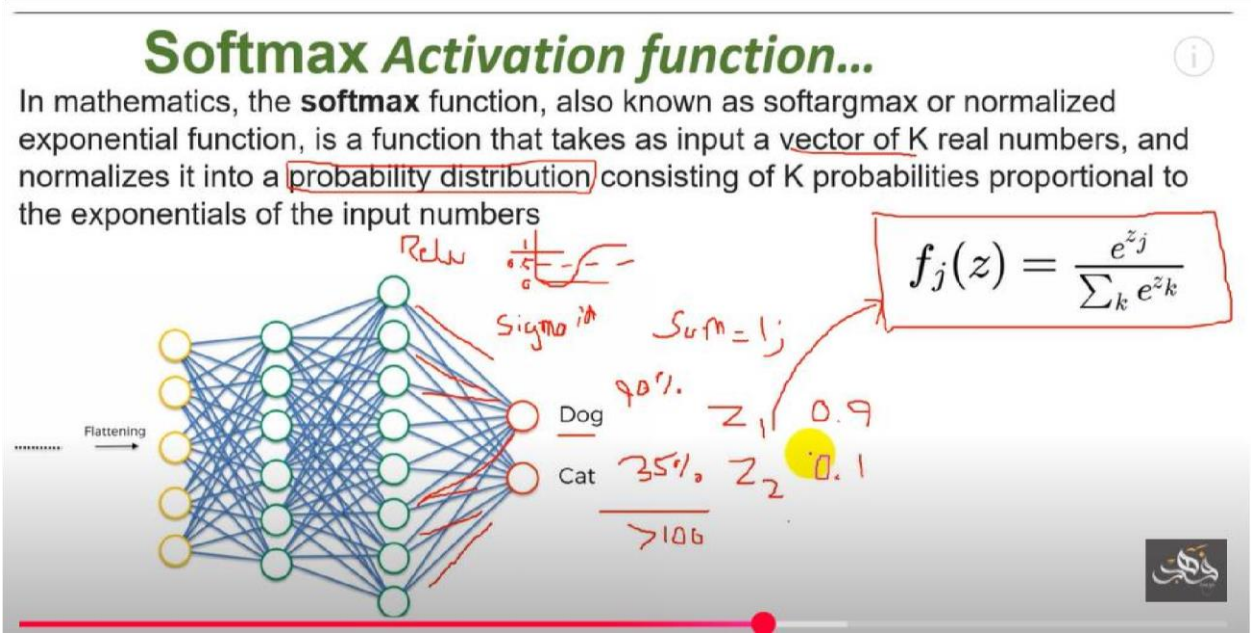
🔻 Sigmoid vs Softmax

1. Sigmoid Function

- Formula:

- sigmoid(x) = 1 / (1 + exp(-x))
- Ye **1 neuron ka output** deta hai jo **0 aur 1 ke beech hota hai**. Mostly use hota hai **binary classification** ke liye (2 classes).

☞ Example:
Tumhare pass **Cat vs Not Cat** problem hai.
Sigmoid output: 0.8 → Matlab **80% Cat hone ka chance**.



2. Softmax Function

- Softmax ek generalization hai sigmoid ka **multi-class classification** ke liye (3 ya zyada classes).
- Har class ke liye probability deta hai, aur **sab probabilities ka sum = 1** hota hai.

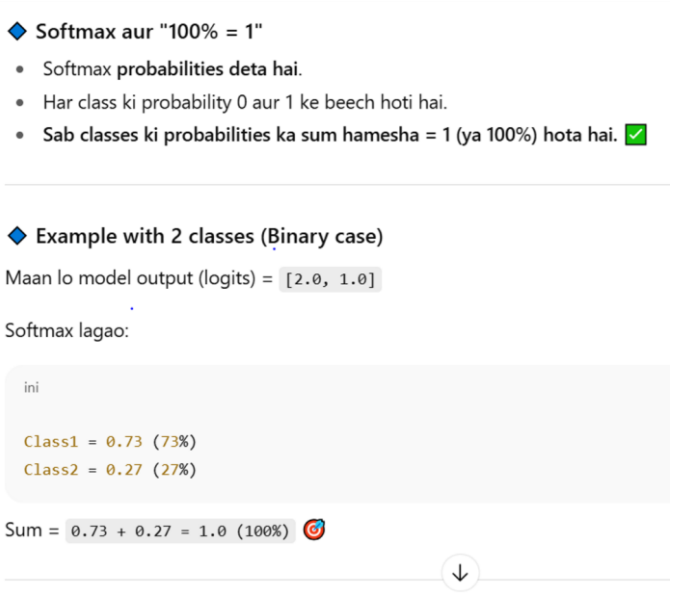
☞ Example:
Classes = [Cat, Dog, Horse]
Model output (logits): [2.0, 1.0, 0.1]
Softmax → [0.65, 0.24, 0.11]

🔻 Kya Softmax = Sigmoid?

- **Nahi ✗**, dono alag hain.
- Lekin **Sigmoid ko Softmax ka special case** samjha ja sakta hai:
  - Jab sirf **2 classes** hoti hain, Softmax aur Sigmoid ka result mathematically almost same hota hai.
  - Is liye binary classification me **Sigmoid use hota hai**, aur multi-class classification me **Softmax** use hota hai. ✓

📌 Conclusion:

- **Binary classification (2 classes)** → Sigmoid.
- **Multi-class classification (3+ classes)** → Softmax.



🔷 **Softmax aur "100% = 1"**

- Softmax **probabilities deta hai**.
- Har class ki probability 0 aur 1 ke beech hoti hai.
- **Sab classes ki probabilities ka sum hamesha = 1 (ya 100%) hota hai.** ✅

🔷 **Example with 2 classes (Binary case)**

Maan lo model output (logits) = `[2.0, 1.0]`

Softmax lagao:

```ini
Class1 = 0.73 (73%)
Class2 = 0.27 (27%)
```

Sum = `0.73 + 0.27 = 1.0 (100%)` 🎯

$$\text{Softmax} = \frac{\exp(\text{value})}{\text{sum}(\exp(\text{all values})}$$

Example:

Logits: [2.0, 1.0]

1. Take exponenentials
   exp(2.0) = 7.389
   exp(1.0) = 2.718
2. Sum of exponen tials
   7.389 + 2.718 = 10.107
3. Divide each by sum
   Class1 = 7.389/10.107 ≈ 0.727 %)
   Class2 = 2.718/10.107 ≈ 0.27(27%

☑ Sum = 1.0 (100%)

Bilkul bhai 👌 tumne sahi pakra hai.

🔷 Rule of Thumb (Seedha aur simple):

- **Binary Classification (2 classes) → Sigmoid**

  👉 Example: *Cat vs Not Cat*

  Output ek hi neuron deta hai jo probability hoti hai (0–1).
- **Multi-Class Classification (3 ya zyada classes) → Softmax**

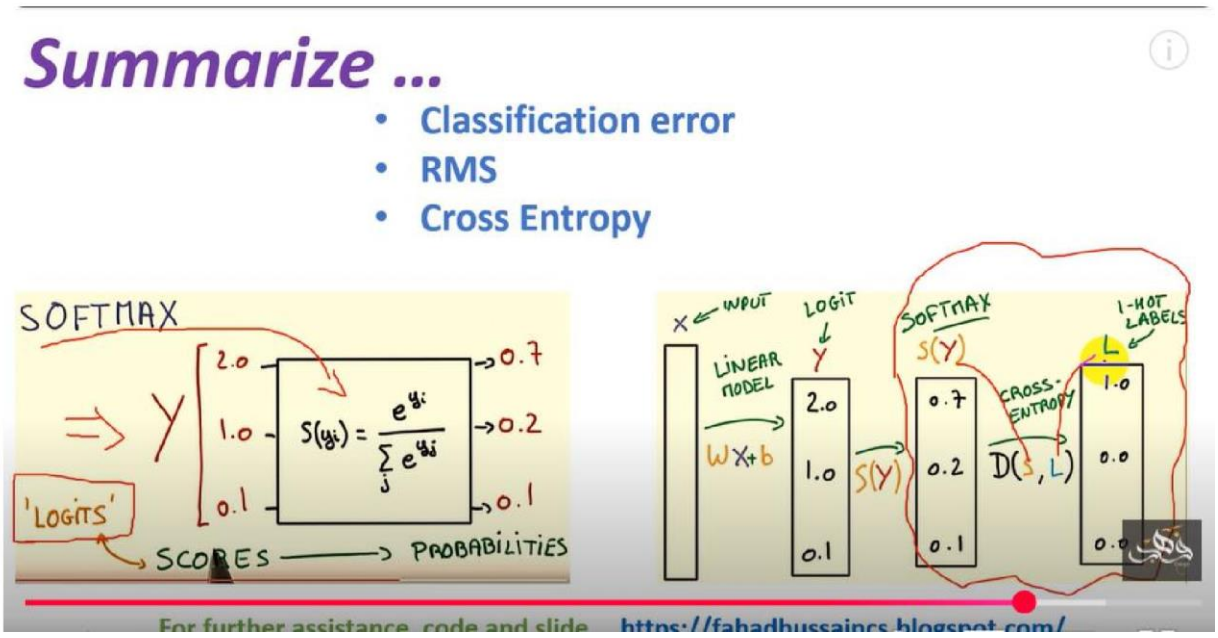
  👉 Example: *Cat vs Dog vs Horse*

  Output har class ke liye ek probability deta hai, aur sabka **sum = 1** hota hai.

📌 Conclusion:

Haan ✅ dono **classification** ke liye hi use hote hain, bas difference ye hai:

- Do classes → Sigmoid
- Zyada classes → Softmax



Loss Function ka kaam

Loss function **model ke prediction aur actual answer ka difference measure karta hai**. Agar prediction sahi ke kareeb ho → loss **chhota** hoga.

- Agar prediction ghalat door ho → loss **bara** hoga.

🔻 Example (easy numbers)

Actual = 10
Prediction = 8

Loss (simple difference) = |10 − 8| = 2

Agar Prediction = 3 hota
Loss = |10 − 3| = 7 (zyada ghalti)

🔻 Purpose

Loss function ek **signal deta hai** optimizer ko ke model ke weights aur biases kaise update karne hain taake agli dafa prediction better ho.

📌 Short mein:
Loss = Model ki ghalti ka measure. Jitna chhota loss, utna acha model. ✅

Mean Squared Error (MSE) – Regression ke liye

🔷 1. Mean Squared Error (MSE) – Regression ke liye

Ye mostly numbers predict karne (regression) mein use hota hai.

Formula:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- y_i = actual value
- $\hat{y}_i$ = predicted value
- N = total samples

👉 Example:
Actual = [3, 5], Prediction = [2, 6]

$$MSE = \frac{(3-2)^2 + (5-6)^2}{2} = \frac{1+1}{2} = 1$$

🔻 2. Binary Cross Entropy (BCE) – Binary Classification ke liye

Formula:

🔷 Formula in Image

$$H(p, q) = -\sum_{x} p(x) \log q(x)$$

- $p(x)$ = **true distribution** (actual labels → one-hot encoding hota hai ML mein).
- $q(x)$ = **predicted distribution** (model ka Softmax output).

🔷 Matlab

Cross-entropy basically **do probability distributions ke beech difference** measure karta hai:

1. **True distribution (p)** → e.g., actual Dog = `[0,1,0]`
2. **Predicted distribution (q)** → e.g., model output = `[0.9, 0.1, 0.0]`

Phir formula sirf **correct class ka probability** uthata hai aur uska `-log` leta hai.

🔷 Example with Image Connection

Agar true label Dog (y = 1) hai, aur model ne Dog ke liye probability = 0.9 di:

$$Loss = -\log(0.9) = 0.105$$

Agar model Dog ke liye sirf 0.1 probability deta:

$$Loss = -\log(0.1) = 2.302$$

👉 Matlab:

- High confidence in correct class → **loss chhota**
- Low confidence in correct class → **loss bada**

📌 Summary of the Image:

- Yeh formula tumhe dikhata hai ke **Cross Entropy** hamesha **true distribution (p)** aur **predicted distribution (q)** ka comparison karta hai.
- Aur final loss sirf **sahi class ki predicted probabi** ↓ par depend karta hai.

🔻 3. Categorical Cross Entropy – Multi-Class Classification ke liye Formula:

Formula:

$$Loss = -\sum_{i=1}^{C} y_i \cdot \log(\hat{y}_i)$$

- C = total classes
- y_i = 1 for correct class, 0 otherwise (one-hot encoding)
- $\hat{y}_i$ = predicted probability for class i

👉 Example:

Actual = [0, 1, 0] (class 2)

Prediction = [0.1, 0.8, 0.1]

$$Loss = -[1 \cdot \log(0.8)] = -\log(0.8) \approx 0.223$$

📌 **Summary:**

- **MSE** → regression (numbers predict karna).
- **BCE** → binary classification.
- **CrossEntropy** → multi-class classification. ↓

Difference: RMSprop vs Adam:

Feature	RMSprop	Adam
Learning rate	Adaptive hoti hai, per weight alag hoti hai	Adaptive hoti hai + momentum bhi use karta hai
Speed	Fast	Faster (most cases)
Stability	Kaafi stable CNNs ke liye	Zyada stable aur general-purpose
Usage	Mostly RNNs, CNNs (sequence/data with fluctuations)	Almost sabhi problems me (classification, NLP, vision, etc.)

Simple analogy

- **RMSprop** = Ek driver jo speed ko road ke according adjust karta hai.
- **Adam** = Wahi driver jo road ke sath-sath apna past driving experience bhi use karta hai (momentum).

In practice

- Beginners ke liye **Adam** zyada easy aur reliable hai.

Example Code (Google Colab ke liye)

python

```python
# Step 1: Download aur Extract ZIP file
import zipfile, os
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Suppose dataset link (aap yahan apna link rakh sakti ho)
!wget --no-check-certificate \
  "https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip" \
  -O /tmp/cats_and_dogs_filtered.zip
```

How to extract zip file:

```
import os
import zipfile

local_zip = '/tmp/cats_and_dogs_filtered.zip'   # ZIP file ka path
zip_ref = zipfile.ZipFile(local_zip, 'r')       # File ko read mode mein open kiya
zip_ref.extractall('/tmp')                      # /tmp folder mein extract kar diya
zip_ref.close()                                 # File close kar di
```

➡️ Yahan `zip_ref.extractall('/tmp')` ka matlab hai ke **ZIP ke andar jo bhi folders/files hain, sab** `/tmp` **ke andar nikal aayein.**

Extracted data folder paths:

```
# Folder paths
base_dir = '/tmp/cats_and_dogs_filtered'
train_dir = os.path.join(base_dir, 'train')
val_dir   = os.path.join(base_dir, 'validation')

print("Train classes:", os.listdir(train_dir))
print("Validation classes:", os.listdir(val_dir))
```

Step 1: Download aur Extract ZIP file import zipfile, os import tensorflow as tf from tensorflow.keras.preprocessing.image import ImageDataGenerator

Suppose dataset link (aap yahan apna link rakh sakti ho)

```
!wget --no-check-certificate \

"https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip" \

-O /tmp/cats_and_dogs_filtered.zip
# Extract  local_zip = '/tmp/cats_and_dogs_filtered.zip'

zip_ref = zipfile.ZipFile(local_zip, 'r')

zip_ref.extractall('/tmp')  zip_ref.close()  # Folder paths

base_dir = '/tmp/cats_and_dogs_filtered'  train_dir =

os.path.join(base_dir, 'train')  val_dir =

os.path.join(base_dir, 'validation')  print("Train classes:",

os.listdir(train_dir))  print("Validation classes:",

os.listdir(val_dir))  # Step 2: ImageDataGenerator +

.flow_from_directory()  train_datagen =

ImageDataGenerator(rescale=1./255)  val_datagen =

ImageDataGenerator(rescale=1./255)  train_generator =

train_datagen.flow_from_directory(  train_dir,

target_size=(150, 150), # sab images 150x150 mein resize

batch_size=32,  class_mode='binary'


)

val_generator = val_datagen.flow_from_directory(

val_dir,  target_size=(150, 150),

batch_size=32,  class_mode='binary'


)
```

```
# Step 3: CNN Model  model = tf.keras.models.Sequential([
tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(150, 150, 3)),
tf.keras.layers.MaxPooling2D(2,2),  tf.keras.layers.Conv2D(64, (3,3),
activation='relu'),  tf.keras.layers.MaxPooling2D(2,2),
tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
tf.keras.layers.MaxPooling2D(2,2),  tf.keras.layers.Flatten(),
tf.keras.layers.Dense(512, activation='relu'),  tf.keras.layers.Dense(1,
activation='sigmoid') # binary classification

])
model.compile(
loss='binary_crossentropy',
optimizer='RMSprop',  metrics=['accuracy']

)
# Step 4: Training  history
= model.fit(
train_generator,
steps_per_epoch=100, #
batches per epoch
epochs=5,
validation_data=val_gene
```

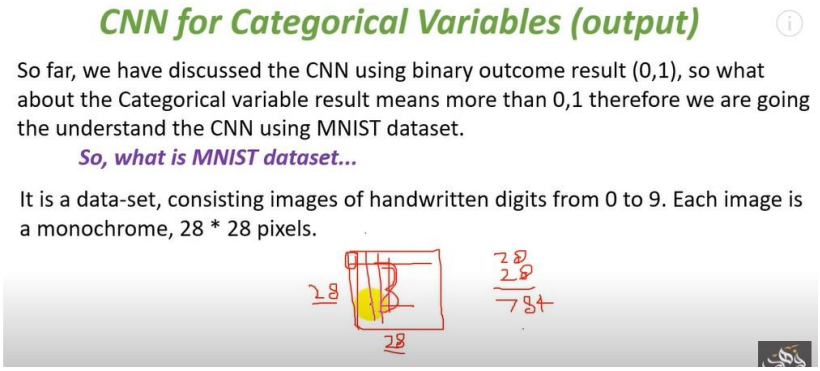
rator,

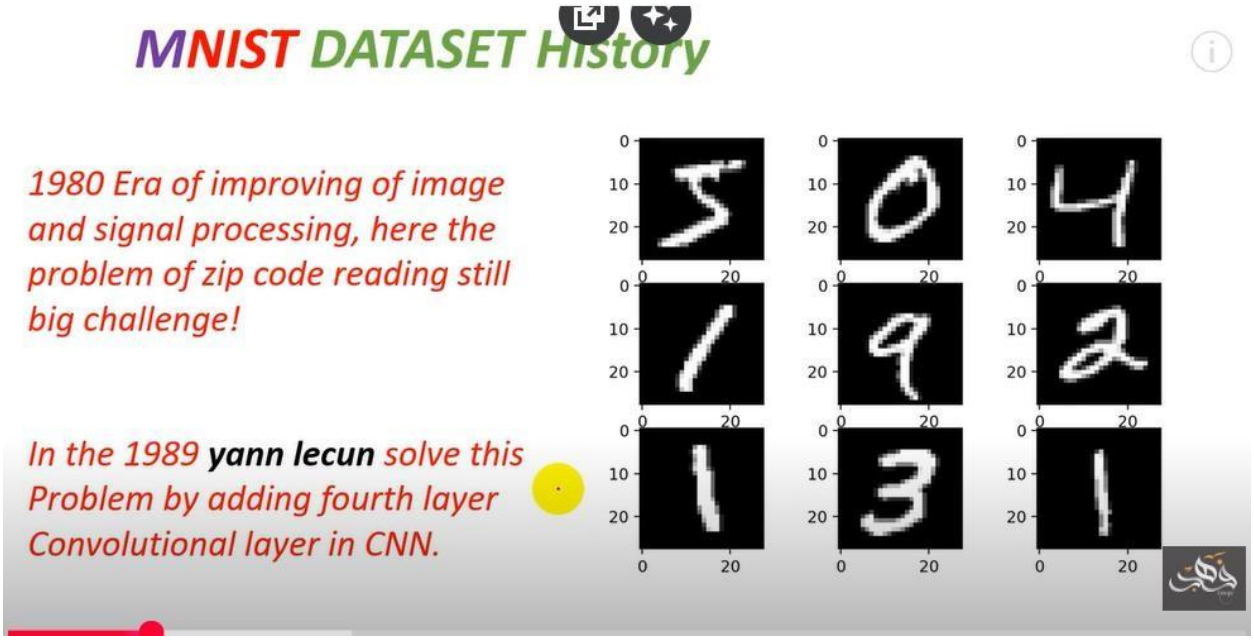
validation_steps=50

)

Chapter 9

MINST DATASET:





MNIST Kya Hai?

MNIST ka full form hai:

Modified National Institute of Standards and Technology dataset.

Ye ek **image dataset** hai jisme:

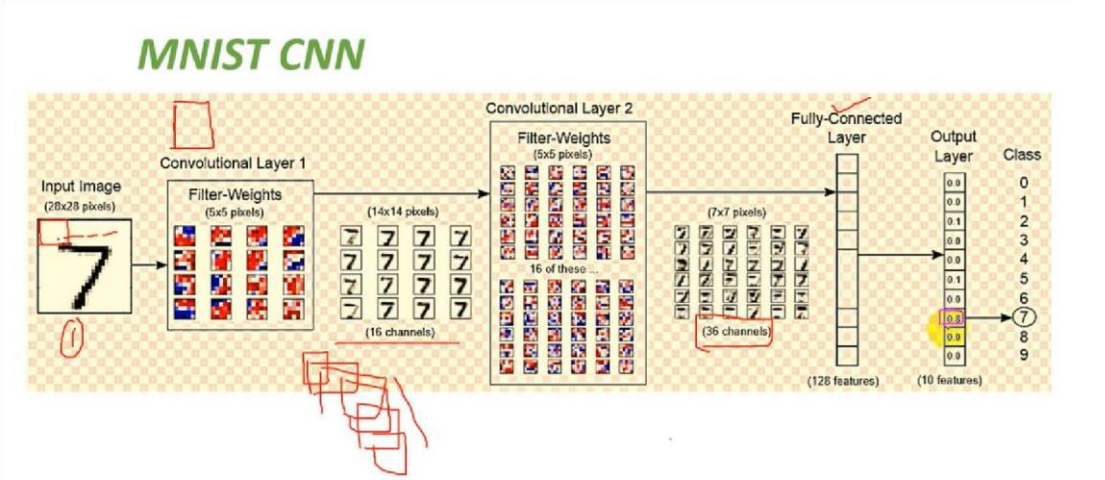
- **70,000** total images hoti hain handwritten digits ki (0 se 9 tak)
  - **60,000 training images**
  - **10,000 testing images**
- Har image:
  - **28x28 pixels** hoti hai
  - Grayscale hoti hai (black & white)
  - Ek single digit hoti hai (e.g. "3", "7", etc.)

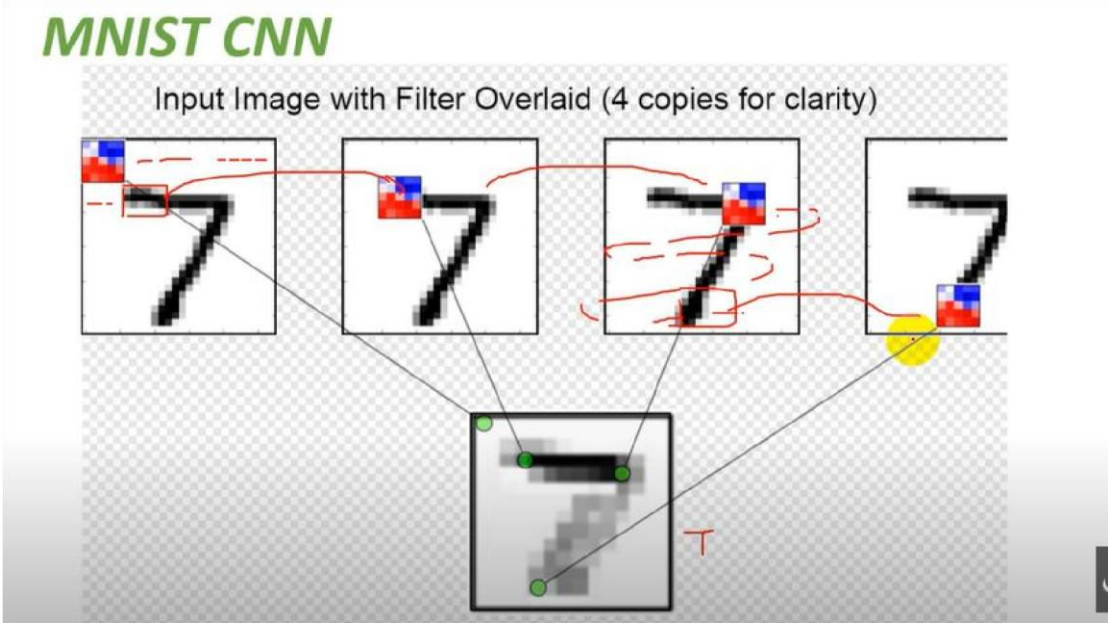
Deep Learning Mein MNIST Ka Use Kab Karte Hain?

MNIST ko use karte hain:

1. **Deep Learning Seekhne ke Liye:**
   - Beginners ke liye perfect hai — simple hai, fast train hota hai
   - CNNs (Convolutional Neural Networks) practice karne ke liye use hota hai
2. **Model Testing ke Liye:**
   - Agar aapne koi naya neural network banaya hai, toh pehle usko MNIST pe test karte hain
3. **Hyperparameter Tuning:**
   - Learning rate, batch size, epochs, etc. test karne ke liye MNIST best hai
4. **Research Papers/Demo Projects:**
   - Research mein naye methods ko MNIST pe benchmark kiya jata hai

Working:





Fashion MNIST

Simple Roman Urdu:
Fashion MNIST ek dataset hai jo **kapron ki tasweeren** deta hai, aur model ka kaam hota hai unko **naam se pehchanna** (jaise T-shirt, trouser, shoe, bag).

Yani → **tasweer input, kapre ka naam output.** 👕👖👟👜

Fashion MNIST kya hai?

- Ye ek **dataset** hai jo MNIST digits ki jagah **clothes (kapdon) ke images** contain karta hai.
- Har image 28×28 grayscale hoti hai (bilkul digits jaisi).
- Isme **10 classes** hain, jaise:
  - T-shirt/top
  - Trouser
  - Pullover
  - Dress
  - Coat
  - Sandal
  - Shirt
  - Sneaker
  - Bag
  - Ankle boot

CIFAR-10 Dataset Overview

CIFAR = *Canadian Institute For Advanced Research*.

CIFAR-10 ek labeled dataset hai jo image classification tasks ke liye use hota hai.

2. **Dataset Size**:
   - Total images: 60,000
     - Training images: 50,000
     - Test images: 10,000
   - Image size: 32x32 pixels (RGB, yani 3 color channels)
3. **Number of Classes**: 10 classes

   Ye classes hain:

   1. Airplane
   2. Automobile
   3. Bird
   4. Cat
   5. Deer
   6. Dog
   7. Frog
   8. Horse
   9. Ship
   10. Truck

4. **Characteristics**:
   - Each image is **small (32x32)**, RGB (3 channels).
   - Images are **natural images** from real-world objects.
   - Dataset balanced hai: har class ke 6,000 images hain.
5. **Usage in ML**:
   - Mostly **image classification models** test karne ke liye.
   - Beginners aur researchers ke liye **benchmarking** ke liye perfect hai.
   - Commonly used models: CNNs (Convolutional Neural Networks), ResNet, VGG, etc.
6. **Data Format**:
   - Training data aur test data **binary files** ya **Python pickle files** mein available hote hain.
   - Libraries like **TensorFlow** aur **PyTorch** se easily load kiya ja sakta hai.

Code for TensorFlow CIFAR-10

```
# ----------------------------
# TensorFlow CIFAR-10 CNN with Data Augmentation
# ----------------------------
 import tensorflow as tf from tensorflow.keras import datasets, layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator import
matplotlib.pyplot as plt from google.colab import files import cv2
import numpy as np

# ----------------------------
# 1. Load CIFAR-10 Dataset
# ----------------------------
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
# Normalize images (0–1 range) x_train = x_train.astype('float32') /
255.0 x_test  = x_test.astype('float32') / 255.0

# Class labels class_names = {
    0:'airplane', 1:'car', 2:'bird', 3:'cat', 4:'deer',
    5:'dog', 6:'frog', 7:'horse', 8:'ship', 9:'truck'
}

# ---------------------------- # 2. Data Augmentation
# ---------------------------- datagen = ImageDataGenerator(
rotation_range=15,     width_shift_range=0.1,
height_shift_range=0.1,     horizontal_flip=True
)
datagen.fit(x_train)

# ----------------------------
# 3. Build CNN Model
```

```
# ----------------------------- model = models.Sequential([
layers.Conv2D(32, (3,3), activation='relu', padding='same',
input_shape=(32,32,3)),     layers.Conv2D(32, (3,3),
activation='relu', padding='same'),
layers.MaxPooling2D((2,2)),     layers.Dropout(0.25),
    layers.Conv2D(64, (3,3), activation='relu',
padding='same'),     layers.Conv2D(64, (3,3), activation='relu',
padding='same'),     layers.MaxPooling2D((2,2)),
layers.Dropout(0.25),
    layers.Conv2D(128, (3,3), activation='relu',
padding='same'),     layers.Conv2D(128, (3,3), activation='relu',
padding='same'),     layers.MaxPooling2D((2,2)),
layers.Dropout(0.25),
    layers.Flatten(),
layers.Dense(256, activation='relu'),
layers.Dropout(0.5),     layers.Dense(10,
activation='softmax')
])

# ----------------------------- #
4. Compile Model
# -----------------------------
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# ----------------------------- #
5. Train Model
# ----------------------------- history =
model.fit(     datagen.flow(x_train, y_train,
batch_size=64),     validation_data=(x_test,
y_test),     epochs=50
)

# ----------------------------- #
6. Evaluate Model
# ----------------------------- test_loss, test_acc =
model.evaluate(x_test, y_test, verbose=2) print("\n✅ Test
Accuracy:", test_acc)
```

```
# ----------------------------
# 7. Image Preprocessing # ------
---------------------- def
image_processing(image_path):
    img = cv2.imread(image_path)                  # load BGR
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)    # convert to RGB
img = cv2.resize(img, (32,32))                # resize     img =
img.astype("float32") / 255.0           # normalize     img =
np.expand_dims(img, axis=0)             # (1,32,32,3)
print("Processed shape:", img.shape)     return img


# ----------------------------
# 8. Prediction Function # -----
----------------------- def
img_prediction(image_path):
    img = image_processing(image_path)
prediction = model.predict(img)
    prediction_class = np.argmax(prediction)
     plt.imshow(img[0])     plt.axis("off")
plt.title(f"Prediction: {class_names[prediction_class]}")
plt.show()
     print("🔮 Predicted Class:", class_names[prediction_class])
# ---------------------------- #
9. Upload + Predict
# ----------------------------
uploaded = files.upload() for fn
in uploaded.keys():
    print("📂 Selected Image:", fn)
img_prediction(fn)
```

Chapter 11

RNN

RNN ka main kaam **sequence data** handle karna hai. Sequence ka matlab hai **aisa data jo ek order (time ya position) ke sath aata hai, aur jisme pichhle hisson ka asar agle hisse par padta hai**.

Main aapko simple shabdon me aur real-life examples ke sath samjhata hoon:

🔼 1. Text

- **Kya hai:** Words ek ke baad ek aate hain aur sentence ka meaning sequence pe depend karta hai.
- **Example:**
  Sentence: *"I love playing cricket"*
  Agar order change ho jaye → *"Cricket playing love I"* → matlab hi badal gaya.
- **RNN ka use:**
  - Sentiment analysis (positive / negative review) ○ Next word prediction (autocomplete, like mobile keyboard) ○ Machine translation (English → Urdu, etc.)

🔼 2. Speech (Audio)

- **Kya hai:** Speech ek sequence of sounds hai jo time ke sath flow karti hai. Har sound (phoneme) ka order important hota hai.
- **Example:** ○ Agar koi bole "Hi" → pehle **H** ka sound aayega, phir **i**. ○ Agar order ulta ho jaye to voice samajh hi nahi aayegi.
- **RNN ka use:**
  - Speech recognition (Google Assistant, Siri)
  - Voice-to-text conversion ○ Emotion detection from voice

🔼 3. Time Series

📘 RNN Notes (with Problems & Solutions)

🔷 1. RNN Basics

- **Definition:** Recurrent Neural Network (RNN) ek aisi neural network architecture hai jo **sequential data** (ordered/time-dependent data) ko handle karti hai.
- **Kaise kaam karta hai:**
  - Har input ke sath ek **hidden state** pass hota hai. ○ Ye hidden state past information (memory) ko represent karta hai.
  - Is wajah se RNN text, speech aur time-series data ke liye useful hai.

Examples:

- **Text:** Sentence ka meaning context ke sath nikalna.
- **Speech:** Tone detect karna (aggressive/emotional/normal).
- **Time-series:** Stock prices ya weather forecast karna.

🔷 2. RNN ki Problems

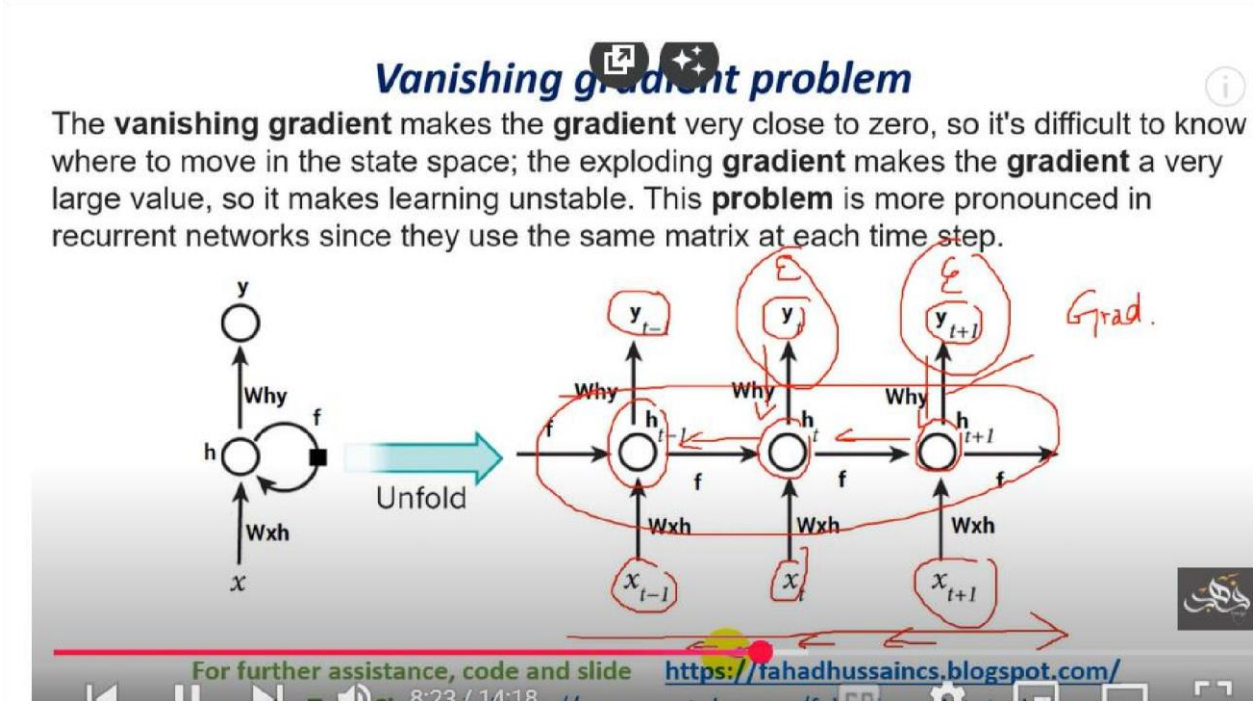
⚠ Problem 1: Vanishing Gradient

- **Kya hota hai:**
  Jab sequence lamba ho (long sentences, years of stock data), to backpropagation ke dauraan gradients repeatedly multiply hote hain aur dheere-dheere **0 ke paas chale jaate hain**.
- **Effect:**
  - Network sirf **recent information** yaad rakhta hai.
  - **Long-term dependencies** (purani information) forget ho jaati hai.
    - **Example:**

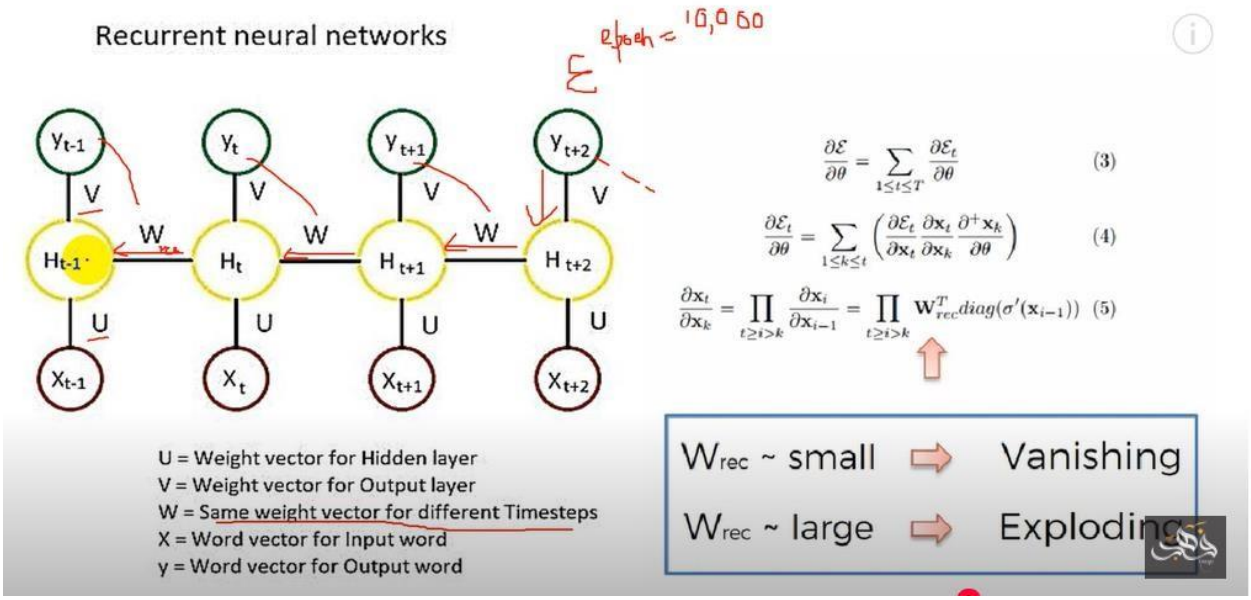
  Sentence: *"The cat, which was very hungry, ate the food."* ○ Simple RNN shayad "cat" yaad na rakhe kyunki wo bahut pehle aaya tha, aur sirf "food" pe focus kare.

⚠ Problem 2: Exploding Gradient

- **Kya hota hai:**
  Kabhi gradients bohot bade ho jaate hain (explode kar jaate hain), jisse training unstable ho jaati hai aur weights bohot quickly badal jaate hain.
- **Effect:**
  - Training diverge kar jaati hai. ○ Loss value NaN (not a number) ho sakti hai.



(is picture mai gradient dikhaya gaya hai jaisai jaisai hmara input agai jata jaraha hai from x-1, x, x+1 gradient chota horahi hai prevous values minus mai jarahi hai jis sai gradient 0 ho raha hai)



Gradient Kya hai?

- Gradient = **direction + step size** □ Ye hume batata hai: ○ **Direction:** kis taraf weight badalna hai (loss kam karne ke liye). ○ **Magnitude (size):** kitna bada step lena hai.

🔺 Case 1: Gradient Zyada Bada (Exploding Gradient)

- **Kya hota hai:**
  - Step size bohot bada ho jaata hai.
  - Weights ekdum se bahut change ho jaate hain.
- **Effect:** ○ Training unstable ho jaati hai. ○ Loss kabhi bahut zyada girta hai, kabhi badh jaata hai.
  - Kabhi kabhi result hi NaN (Not a Number) aa jata hai.
- **Real-life analogy:**
  - Jaise aap ek pahad se neeche descend kar rahe ho (gradient descent). ○ Agar aap bahut bade kadam loge → aap gir ke pahad ke dusre side chale jaoge (galat direction).

🔻 Case 2: Gradient Bahut Chhota (Vanishing Gradient)

- **Kya hota hai:**
  - Step size bohot chhota ho jaata hai.
  - Weights me hardly koi update hota hai.
- **Effect:**
  - Network kuch naya nahi seekhta.
  - Especially **long sequences** me RNN purani memory forget kar deta hai (sirf recent inputs yaad rehte hain).
- **Real-life analogy:**
  - Pahad se neeche utarte waqt agar aap *bahut chhote chhote steps* loge → aap kabhi bottom tak nahi pahunchoge, raste me hi atak jaoge.

✅ Balance (Why Gradient is Important)

- Gradient na bohot bada hona chahiye, na bohot chhota.
- Isliye hum:
  - **Exploding gradient** ke liye → *Gradient Clipping*.
  - **Vanishing gradient** ke liye → *LSTM/GRU* use karte hain.

🔺 3. Solutions to RNN Problems

✅ Solution 1: LSTM (Long Short-Term Memory)

- Special type of RNN jo **gates** use karta hai:
  - **Forget Gate:** Kya cheez bhoolni hai. ○ **Input Gate:** Kya cheez nayi memory me add karni hai.
  - **Output Gate:** Kaunsi memory aage bhejni hai.
- **Benefit:**
  - LSTM long-term dependencies ko bhi handle kar leta hai. ○ Vanishing gradient problem kaafi had tak solve ho jaata hai.

✅ Solution 2: GRU (Gated Recurrent Unit)

- Simplified version of LSTM.

- Sirf **Update Gate** aur **Reset Gate** hoti hain.
- Faster aur computationally efficient.
- Performance bhi LSTM ke barabar hoti hai kai tasks me.

✅ Solution 3: Gradient Clipping (for exploding gradient)

- Jab gradients bohot bade ho jaate hain, unhe manually ek threshold value tak limit kar dete hain.
- Isse training stable ho jaati hai.

🔻 4. Summary (Ek Line me)

- **RNN:** Sequence data handle karta hai.
- **Problem:** Vanishing & Exploding gradients.
- **Solutions:** LSTM, GRU, Gradient clipping.

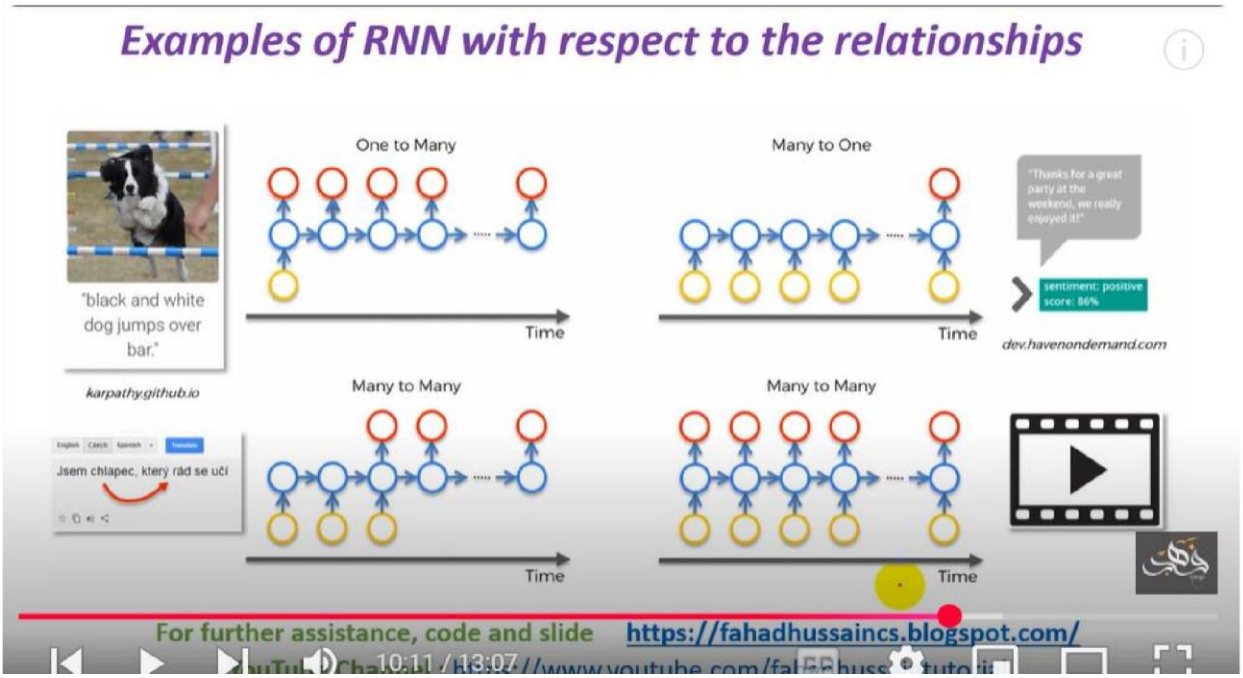
☞ Ye note aapke liye ek **full revision sheet** jaisa hai.

Chaho to main iska **diagrammatic flow (RNN → Problem → LSTM/GRU)** bana kar bhi samjha sakta hoon

  - **Electricity demand:** Pichhle ghanto ki electricity usage se agle ghante ka demand predict hota hai.

✅ Simple Summary:

- **Text** = words ka order matter karta hai
- **Speech** = sounds ka order matter karta hai
- **Time series** = numbers/data ka order time ke hisaab se matter karta hai


Examples of RNN with respect to the relationships
"black and white dog jumps over bar."
karpathy.github.io
One to Many
Time
Many to One
Time
"Thanks for a great party at the weekend, we really enjoyed it!"
sentiment: positive score: 86%
dev.havenondemand.com
Jsem chlapec, který rád se učí
Many to Many
Time
Many to Many
Time
For further assistance, code and slide https://fahadhussaincs.blogspot.com/
10:11 / 13:07

Difference between RNN And CNN

🔷 1. CNN (Convolutional Neural Network)

- **Kis liye:** Mainly **images** aur spatial data ke liye.
- **Kaise kaam karta hai:**
  - CNN ek image ke **pixels ka pattern** nikalta hai (edges, colors, shapes).
  - Pehle chhote features pakadta hai (line, corner), phir bade features (face, object).
- **Real-life Example:**
  - 📸 Facebook pe automatic *tag suggestion* (chehra pehchan)
  - 🚗 Self-driving car — traffic light ya stop sign pehchan na
  - 🔍 X-ray scan me cancer cell detect karna

🔷 2. RNN (Recurrent Neural Network)

- **Kis liye:** Mainly **sequence data** ke liye (order/time matter karta hai).
- **Kaise kaam karta hai:**
  - RNN ke paas ek **memory (hidden state)** hoti hai jo pichhle inputs ka asar agle inputs pe dalti hai.
  - Matlab wo "sequence ka context" yaad rakhta hai.
- **Real-life Example:**
  - 📝 Google Translate (ek language se dusre language me translate karte waqt pura sentence ka sense pakadta hai)
  - 🎤 Alexa/Siri ko command samajhna
  - 📈 Stock price ya weather prediction

🔷 3. Main Difference Table

Feature	CNN	RNN
Data type	Images, videos, spatial data	Text, speech, time series
Memory	Past info yaad nahi rakhta	Past info (hidden state) yaad rakhta hai
Strength	Pattern/feature extraction	Sequence/order understanding
Examples	Image classification, object detection	Sentiment analysis, translation, stock prediction

---

✅ **Summary in one line:**

- **CNN = "Dekhta hai"** (images ke patterns samajhta hai)
- **RNN = "Yaad rakhta hai"** (sequence aur context samajhta hai)

Kya RNN ka kaam CNN kar sakta hai?

1. Sequence Data (RNN ka area)

- RNN naturally **sequence aur memory** handle karta hai.
- Lekin CNN ko bhi sequences pe use kiya ja sakta hai:
  - **Text** → 1D Convolutions (Conv1D) use hote hain jo words ka local context pakadte hain.
  - **Speech** → CNNs short-time features (spectrogram patterns) ko capture kar lete hain.
  - **Time-series** → CNN sliding window ki tarah patterns nikal leta hai.

📌 **Limitation:** CNN sirf fixed-size patterns dekh sakta hai, lekin RNN puri history (memory) consider karta hai. Matlab CNN "short-term" dependencies pakadta hai, RNN "long-term" dependencies bhi.

---

2. Images (CNN ka area)

- CNN images ke liye specialized hai.
- RNN images ke liye natural fit **nahi hai**, lekin theoretically agar image ko pixel sequence me convert kar dein, RNN bhi kar lega.
  📌 **Problem:** Ye bohot inefficient aur slow ho jaata hai, isliye practically koi images ke liye RNN use nahi karta.

Difference between ANN, CNN, and RNN

Feature	ANN (Artificial Neural Network)	CNN (Convolutional Neural Network)	RNN (Recurrent Neural Network)
Basic Idea	Simple feed-forward network	Uses convolution filters to detect patterns	Uses memory (hidden state) to handle sequences
Best For	Structured/tabular data	Images, videos, spatial data	Text, speech, time-series data
Input Type	Fixed-size vectors	Grid-like data (2D image, video frames)	Sequential data (ordered/time-dependent)
How it Works	Fully connected layers process input directly	Convolutions + pooling extract features (edges → shapes → objects)	Each step depends on previous step (context/memory)
Memory of Past Data	❌ No	❌ No	✅ Yes (remembers previous inputs)
Example Use Cases	Predict house prices, bank loan approval, fraud detection	Image classification, face recognition, medical imaging	Sentiment analysis, language translation, stock prediction
Efficiency	Works but not efficient for images/sequences	Very efficient for images/spatial data	Very efficient for sequential data

✅ Easy Summary:

- ANN = General-purpose network (basic, tabular/structured data)
- CNN = "Dekhta hai" → images/videos ke patterns samajhta hai
- RNN = "Yaad rakhta hai" → text/speech/time series ka order samajhta hai

---

RNN CODE:

```
import tensorflow as tf from
tensorflow.keras import layers, models from
tensorflow.keras.datasets import imdb


# ---------------------------- #
1. Load IMDB Dataset
# ----------------------------
num_words = 10000  # top 10k words only
```

```
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=num_words)
# Pad sequences (make them equal length) maxlen = 200 x_train =
tf.keras.preprocessing.sequence.pad_sequences(x_train, maxlen=maxlen) x_test =
tf.keras.preprocessing.sequence.pad_sequences(x_test, maxlen=maxlen)
# ----------------------------
# 2. Build Simple RNN Model # --
--------------------------
model = models.Sequential([
    layers.Embedding(input_dim=num_words, output_dim=32, input_length=maxlen),
layers.SimpleRNN(32),   # <-- Simple RNN layer     layers.Dense(1,
activation="sigmoid")
])

# ---------------------------- #
3. Compile
# ----------------------------
model.compile(optimizer="adam",
              loss="binary_crossentropy",
metrics=["accuracy"])

# ---------------------------- #
4. Train
# ---------------------------- history =
model.fit(x_train, y_train,
epochs=3,
batch_size=64,
validation_split=0.2)

# ---------------------------- #
5. Evaluate
# ---------------------------- test_loss, test_acc =
model.evaluate(x_test, y_test, verbose=2) print("\n✅ Test
Accuracy:", test_acc)
 import matplotlib.pyplot as plt
plt.figure(figsize=(10,4)) plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train loss')
```

```
plt.plot(history.history['val_loss'], label='val loss')
plt.legend(); plt.title('Loss')
 plt.subplot(1,2,2) plt.plot(history.history['accuracy'],
label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.legend(); plt.title('Accuracy') plt.show()
```

LSTM / GRU Theory

1. Problem with Simple RNN

- Simple RNN **long-term memory** hold nahi kar pata.
- **Vanishing Gradient Problem** ke wajah se purani information training ke dauraan khatam ho jaati hai.
- Result: RNN bas **short-term dependencies** handle karta hai.

2. Solution → LSTM (Long Short-Term Memory)

- **LSTM ek special RNN type hai** jisme memory ko handle karne ke liye extra *gates* hote hain.
- Ye gates decide karte hain:
    1. **Forget Gate** → Konsi purani information ko bhoolna hai
    2. **Input Gate** → Konsi nayi information store karni hai 3. **Output Gate** → Konsi information aage bhejni hai

✅ Is wajah se LSTM **long-term dependencies** handle kar sakta hai.

3. GRU (Gated Recurrent Unit)

- LSTM ka **simplified version**. □ Sirf 2 gates hote hain:
    1. **Update Gate** → purani + nayi memory balance karta hai
    2. **Reset Gate** → decide karta hai purani memory ko kitna ignore karna hai

✅ GRU fast train hota hai aur performance LSTM ke barabar hoti hai.

4. Comparison

Feature	Simple RNN	LSTM	GRU
Memory Handling	Short-term only	Long + short-term	Long + short-term
Gates	No gates	Forget, Input, Output (3 gates)	Update, Reset (2 gates)
Complexity	Simple, fast	Complex, slow	Medium, faster than LSTM
Use Case	Short sequences	Long sequences (text, speech)	Long sequences, faster training

Code implementation with guru:

```
import tensorflow as tf from tensorflow.keras import
layers, models, preprocessing import matplotlib.pyplot as
plt import numpy as np


# ----------------------------
# 1) Load IMDB dataset
# ----------------------------
num_words = 10000 maxlen = 200


(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.imdb.load_data(num_words=num_words)


# Pad sequences x_train = preprocessing.sequence.pad_sequences(x_train,
maxlen=maxlen)  x_test   =  preprocessing.sequence.pad_sequences(x_test,
maxlen=maxlen)
```

```
 print("Training samples:", len(x_train))
print("Test samples:", len(x_test))

# ----------------------------
# 2) Build GRU model
# ---------------------------- model = models.Sequential([
layers.Embedding(input_dim=num_words, output_dim=128, input_length=maxlen),
layers.GRU(64, return_sequences=False),     layers.Dropout(0.5),
layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy'])

model.summary()

# ----------------------------
# 3) Train model
# ---------------------------- history
= model.fit(x_train, y_train,
epochs=6,
batch_size=64,
                    validation_split=0.2)

# ---------------------------- #
4) Evaluate
# ---------------------------- loss, acc =
model.evaluate(x_test, y_test, verbose=2)
print(f"\nGRU Test accuracy: {acc:.4f}")
# ---------------------------- # 5) Plot training
curves # ----------------------------
plt.figure(figsize=(10,4)) plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.legend(); plt.title('Loss')
 plt.subplot(1,2,2) plt.plot(history.history['accuracy'],
label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.legend(); plt.title('Accuracy') plt.show()
```

```
# ---------------------------- #
6) User input prediction
# ----------------------------
# Get IMDB word index word_index =
tf.keras.datasets.imdb.get_word_index()
 def text_to_sequence(text):     # lowercase + split     text =
text.lower()     words =
preprocessing.text.text_to_word_sequence(text)     # Convert
words to integers using IMDB word_index, unknown=2     seq =
[word_index.get(word, 2) for word in words]  # 2=<UNK>
    # Pad sequence
    seq = preprocessing.sequence.pad_sequences([seq], maxlen=maxlen)
return seq

# Input loop
while True:
    review = input("\nEnter a movie review (or 'exit' to quit): ")
if review.lower() == "exit":
        break

    seq = text_to_sequence(review)     pred = model.predict(seq)
print(f"Prediction (0=negative, 1=positive): {pred[0][0]:.4f}")     print("→
Review is Positive" if pred[0][0] > 0.5 else "→ Review is Negative")
```

Dataset apna ho tu tokanizer ka use for text mapping:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
# Suppose aapka dataset
texts = [
```

```
    "I love this movie",
    "This movie is bad",
    "Amazing film!"
]

# 1. Tokenizer create karo tokenizer = Tokenizer(num_words=10000)  # top
10k words consider tokenizer.fit_on_texts(texts)           # khud ka
dataset se mapping seekho
# 2. Texts ko sequences me convert karo
sequences = tokenizer.texts_to_sequences(texts)

# 3. Pad sequences maxlen = 10 padded_sequences =
pad_sequences(sequences, maxlen=maxlen)

print("Word index mapping:", tokenizer.word_index)
print("Sequences:", padded_sequences)
```

Chapter 13

Revision

Training testing accuracy method

```python
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=2
)

# Training aur validation loss/accuracy print karna
print("📉 Training Loss:", history.history['loss'])
print("✅ Training Accuracy:", history.history['accuracy'])
print("📉 Validation Loss:", history.history['val_loss'])
print("✅ Validation Accuracy:", history.history['val_accuracy'])
```

📌 **Example Output**

yaml

Copy code

```
Epoch 1/5
250/250 - 5s - loss: 0.4100 - accuracy: 0.8125 - val_loss: 0.3650 - val_accuracy: 0.8450
Epoch 2/5
250/250 - 5s - loss: 0.3000 - accuracy: 0.8700 - val_loss: 0.3500 - val_accuracy: 0.8600
...

📉 Training Loss: [0.4100, 0.3000, ...]
✅ Training Accuracy: [0.8125, 0.8700, ...]
📉 Validation Loss: [0.3650, 0.3500, ...]
✅ Validation Accuracy: [0.8450, 0.8600, ...]
```

Data daikhna ho training mai kitn ahai or testing mai kitna hai

Bilkul! Aapke existing code me bas ye lines add karo **train aur test dataset ke size dekhne ke liye**:

python

```python
# Dataset sizes
print("✅ Training samples:", x_train.shape[0])
print("✅ Training labels:", y_train.shape[0])
print("✅ Test samples:", x_test.shape[0])
print("✅ Test labels:", y_test.shape[0])
```

Agar aap ye run karenge, output kuch is tarah hoga (IMDB dataset ke liye default sizes):

yaml

```yaml
✅ Training samples: 25000
✅ Training labels: 25000
✅ Test samples: 25000
✅ Test labels: 25000
```

How to know number of samples in dataset

```python
# Ya sirf number of samples:
print("Number of training samples:", len(x_train))
print("Number of test samples:", len(x_test))
```

Chapter 15

Transferflow

Transfer Learning – Theory

- **Definition:**
  Transfer Learning ek technique hai jisme hum ek **pretrained model** (jo pehle hi kisi bade dataset par train hua hai) ko le kar apne chhote dataset ke liye use karte hain.
- Matlab: aap **knowledge transfer** kar rahe ho ek problem se doosri problem par.

Why use Pretrained Models?

1. **Time bachana** □ ○ Bada deep learning model ko scratch se train karna bohot slow hota hai (days/weeks).
    - Pretrained model ready hota hai.
2. **Better accuracy (small data par)** 📈 ○ Agar aapke paas dataset chhota hai, pretrained model ki weights aapko strong starting point dete hain.
    - From-scratch training → overfitting risk.
3. **Compute power ki requirement kam hoti hai** 💻 ○ Bada dataset train karne ke liye GPU clusters chahiye. ○ Pretrained weights se normal hardware par bhi kaam ho jaata hai.

🔷 When to use?

1. **Dataset chhota ho** ○ e.g., 1000 images, 500 reviews, etc.
2. **Similar problem/domain ho** ○ e.g., ImageNet pe trained CNN ko cats/dogs classification me use karna. ○ NLP me BERT jo Wikipedia pe trained hai, usko sentiment analysis me use karna.
3. **Rapid prototyping** ○ Jab aapko jaldi ek baseline chahiye.

🔷 Common Scenarios

- **Computer Vision** ○ Pretrained CNNs: VGG, ResNet, Inception, MobileNet ○ Tasks: object detection, classification, medical imaging
- **NLP**
    - Pretrained LMs: BERT, GPT, Word2Vec, FastText ○ Tasks: sentiment analysis, text classification, QA

Transferlearning code:

Google stock exchange price detection systemm

```
#google stock price detection:
import numpy as np import pandas
as pd import matplotlib.pyplot
as plt
 import tensorflow as tf from tensorflow.keras
import layers, models from
sklearn.preprocessing import MinMaxScaler
!unzip google-stock-price.zip

from google.colab import files import
pandas as pd

# Upload multiple files (Train + Test)
uploaded = files.upload()

# Uploaded dictionary ke keys filenames hote hain for
fn in uploaded.keys():
    print("Uploaded file:", fn)

# Read the CSVs (make sure filenames match) train_df =
pd.read_csv("Google_Stock_Price_Train.csv") test_df =
pd.read_csv("Google_Stock_Price_Test.csv")
 print("Train Data:")
print(train_df.head())
```

```
 print("Test Data:")
print(test_df.head())

# 3. Preprocessing
# We'll use the 'Open' column data =
train_df["Open"].values.reshape(-1, 1)

# Scale values between 0 and 1 scaler =
MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

# Train-test split (80-20) train_size =
int(len(scaled_data) * 0.8) train_data =
scaled_data[:train_size] test_data =
scaled_data[train_size:]

# 4. Create sequences (time steps) def
create_sequences(dataset, time_step=60):
X, y = [], []     for i in range(time_step,
len(dataset)):         X.append(dataset[i-
time_step:i, 0])
        y.append(dataset[i, 0])
return np.array(X), np.array(y)

time_step = 60
X_train, y_train = create_sequences(train_data, time_step)
X_test, y_test = create_sequences(test_data, time_step)

# Reshape into 3D for LSTM: [samples, time steps, features]
X_train = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
X_test = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))

print("Train shape:", X_train.shape, y_train.shape)
print("Test shape:", X_test.shape, y_test.shape)

# 5. Build LSTM Model model = models.Sequential([     layers.LSTM(50,
return_sequences=True, input_shape=(X_train.shape[1], 1)),     layers.LSTM(50,
return_sequences=False),     layers.Dense(25),     layers.Dense(1)
```

```
]) model.compile(optimizer="adam",
loss="mean_squared_error")
# 7. Predictions predictions = model.predict(X_test) predictions =
scaler.inverse_transform(predictions)  # rescale to original
 real_prices = scaler.inverse_transform(y_test.reshape(-1, 1))
# 8. Plot Results plt.figure(figsize=(12,6)) plt.plot(real_prices,
color="blue", label="Real Google Stock Price") plt.plot(predictions,
color="red", label="Predicted Google Stock Price") plt.title("Google
Stock Price Prediction with LSTM") plt.xlabel("Time") plt.ylabel("Stock
Price") plt.legend() plt.show()
```

Transfer Learning (TL)

- Ek **pre-trained model** ko use karte hain apne dataset ke liye.
- **Layers freeze kar dete hain**, matlab weights **update nahi hote**.
- Model sirf **feature extractor** ki tarah kaam karta hai.
- Useful jab **dataset chhota ho** aur training fast chahiye.

Example:

- ResNet50 pre-trained on ImageNet → Freeze all layers → Replace final classifier → Train sirf final layer

---

Fine-Tuning (FT)

- TL ka hi **extension hai**.
- Kuch ya **saari layers unfreeze karte hain**, matlab weights **thoda bahut adjust** hote hain.
- Goal: **Accuracy improve karna**, specially agar target dataset thoda bada ya complex ho.

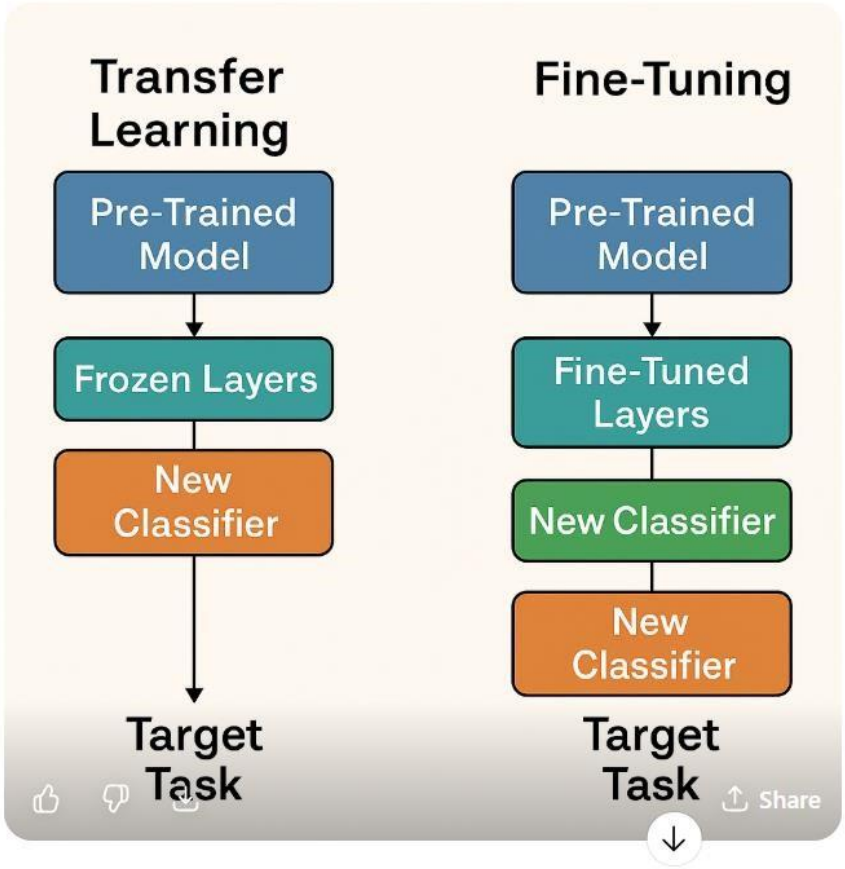
Example:

- Same ResNet50 → Freeze initial layers → Unfreeze last few layers → Train on new dataset

---

💡 **Simple analogy:**

- TL = Pehle se trained student ko guide kiye bina test dena
- FT = Student ko kuch topics revise karwa ke test dena



ResNet50

- **ResNet50** ek deep CNN model hai with **50 layers**.
- Isme **residual (skip) connections** hoti hain jo training ko easy aur stable banati hain.
  - Pre-trained version **ImageNet** par trained hai, aur TL/FT ke liye use hota hai.

Parameters kya hai

- **Weights + biases = parameters**
- **Training me update hote hain** → model seekhta hai aur accuracy improve hoti hai.
- Parameters zyada → model complex, zyada memory use, aur overfitting ka risk.

GlobalAveragePooling2D

- **GlobalAveragePooling2D** last convolution layer ke **2D feature maps** ko **1D vector** me convert karta hai.
- Ye **dense layer ke liye ready input** provide karta hai.
- Iska purpose **final output nikalna** aur **parameters kam karna** hai, directly classification ke

liye.

Transfer learning:

```
Transfer learning




# ===================================== #
1. Import Libraries
# =====================================
import tensorflow as tf import numpy as
np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
```

```
# ======================================
```

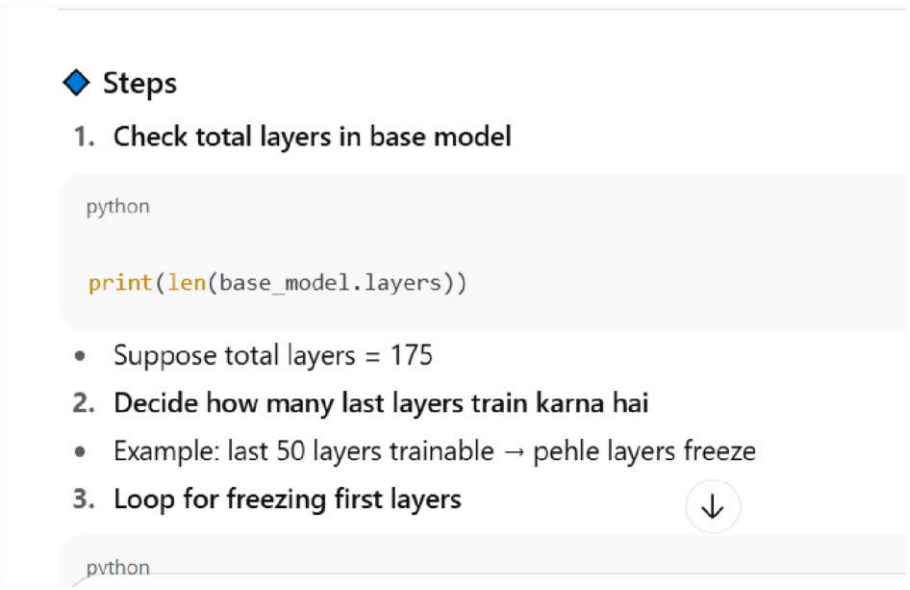
```
# 2. Load CIFAR-10 Dataset
# =====================================
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
 print("Original Train:", x_train.shape,
y_train.shape) print("Original Test:", x_test.shape,
y_test.shape)

# ===================================== #
3. Preprocessing
# =====================================
IMG_SIZE = 224  # ResNet50 requires 224x224 images
#   normalize   (0-1   range)   x_train   =
x_train.astype("float32") / 255.0 x_test  =
x_test.astype("float32") / 255.0
# resize to (224,224,3) x_train =
tf.image.resize(x_train, (IMG_SIZE, IMG_SIZE)) x_test  =
tf.image.resize(x_test, (IMG_SIZE, IMG_SIZE))
 print("Processed Train:", x_train.shape)
print("Processed Test:", x_test.shape)

# CIFAR-10 class names class_names =
['airplane','automobile','bird','cat','deer',
'dog','frog','horse','ship','truck']

# ===================================== # 4. Base Model (ResNet50
pretrained) # ===================================== base_model =
tf.keras.applications.ResNet50(     input_shape=(IMG_SIZE, IMG_SIZE,
3),  # image shape     include_top=False,                    # remove
default classifier     weights="imagenet"                    # load
pretrained weights
) base_model.trainable = False               # freeze base
layers
# =====================================
# 5. Transfer Learning Model
# =====================================
model = tf.keras.Sequential([
    base_model,                                 # pretrained feature extractor
layers.GlobalAveragePooling2D(),           # reduce to feature vector
```

```
layers.Dense(256, activation="relu"),      # dense layer
layers.Dropout(0.5),                       # regularization
    layers.Dense(10, activation="softmax")     # CIFAR-10 has 10 classes
])

# ===================================== #
6. Compile Model
#            =======================================            model.compile(
optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),     #  Adam  optimizer
loss="sparse_categorical_crossentropy",                       # integer labels
metrics=["accuracy"]                                       # track accuracy )

# ===================================== #
7. Train Model
# ======================================
history = model.fit(     x_train,
y_train,     validation_data=(x_test,
y_test),     epochs=5,     batch_size=64
)

# ===================================== #
8. Evaluate
# ===================================== test_loss, test_acc =
model.evaluate(x_test, y_test, verbose=2) print("✅ Test
Accuracy:", test_acc)
# ======================================
# 9. Prediction Function (Single Image) #
======================================
from google.colab import files from PIL
import Image
 def predict_image(model, img_path):
    """Upload an image, preprocess it, and predict class."""
```

```
    # Load + preprocess     img =
Image.open(img_path).convert("RGB")     img =
img.resize((IMG_SIZE, IMG_SIZE))
img_array = np.array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    # Predict     predictions =
model.predict(img_array)     pred_class =
np.argmax(predictions[0])     confidence =
np.max(predictions[0])

    # Show image + prediction
plt.imshow(img)     plt.axis("off")
    plt.title(f"Prediction: {class_names[pred_class]} ({confidence*100:.2f}%)")
    plt.show()
     print(f"🔮 Predicted Class: {class_names[pred_class]} |
Confidence: {confidence*100:.2f}%")

# ===================================== #
10. Upload + Test Prediction
# =====================================
uploaded = files.upload() for fname in
uploaded.keys():     predict_image(model,
fname)
```

Fine-Tuning (FT) code

Check total layers in ypur model



Steps

1. Check total layers in base model

python

```
print(len(base_model.layers))
```

- Suppose total layers = 175

2. Decide how many last layers train karna hai

- Example: last 50 layers trainable → pehle layers freeze

3. Loop for freezing first layers

python

```
# ======================================
# 1. Import Libraries
```

```
# ======================================
import tensorflow as tf import numpy as np
import matplotlib.pyplot as plt from
tensorflow.keras import layers from
google.colab import files from PIL import
Image
import cv2

# ======================================
# 2. Load CIFAR-10 Dataset
# ======================================
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Normalize x_train, x_test = x_train / 255.0,
x_test / 255.0

# Resize to 224x224 for ResNet50 IMG_SIZE = 224 x_train =
tf.image.resize(x_train, (IMG_SIZE, IMG_SIZE)) x_test  =
tf.image.resize(x_test, (IMG_SIZE, IMG_SIZE))
 print("Train:", x_train.shape,
y_train.shape) print("Test:", x_test.shape,
y_test.shape)

# CIFAR-10 class names class_names =
['airplane','automobile','bird','cat','deer',
'dog','frog','horse','ship','truck']

# ======================================
# 3. Load Pretrained ResNet50
# ======================================
base_model = tf.keras.applications.ResNet50(
input_shape=(IMG_SIZE, IMG_SIZE, 3),
```

```
    include_top=False,   # remove original classifier
weights="imagenet"   # pretrained weights
)
base_model.trainable = True  # unfreeze for fine-tuning
# Freeze all layers except last 50 (adjustable) for
layer in base_model.layers[:-50]:
    layer.trainable = False
```

```
# ======================================
# 4. Build Model
# ======================================
model = tf.keras.Sequential([
base_model,
layers.GlobalAveragePooling2D(),
layers.Dense(256, activation="relu"),
layers.Dropout(0.5),     layers.Dense(10,
activation="softmax")
])

# ======================================
# 5. Compile Model (Fine-Tuning)
# ====================================== model.compile(
optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),  # smaller LR for
fine-tuning     loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)

# ====================================== #
6. Train Model
# ======================================
history_finetune = model.fit(
x_train, y_train,
validation_data=(x_test, y_test),
epochs=5,     batch_size=64
```

```
)

# ======================================
# 7. Evaluate Model
# ====================================== test_loss, test_acc =
model.evaluate(x_test, y_test, verbose=2) print("✅ Fine-Tuned
Test Accuracy:", test_acc)

# ======================================
# 8. Custom Image Prediction Function #
====================================
def image_processing(image_path):
    """Preprocess single image for prediction"""
    img = cv2.imread(image_path)                  # load in BGR     img
= cv2.cvtColor(img, cv2.COLOR_BGR2RGB)    # convert to RGB     img =
cv2.resize(img, (IMG_SIZE, IMG_SIZE))   # resize to 224x224     img =
img.astype("float32") / 255.0           # normalize     img =
np.expand_dims(img, axis=0)             # add batch dimension
    return img  def
img_prediction(image_path):
"""Predict and show single image"""
img = image_processing(image_path)
prediction = model.predict(img)
pred_class = np.argmax(prediction)
    confidence = np.max(prediction)
     plt.imshow(img[0])     plt.axis("off")     plt.title(f"Prediction:
{class_names[pred_class]} ({confidence*100:.2f}%)")     plt.show()
     print(f"🎯 Predicted Class: {class_names[pred_class]} |
Confidence: {confidence*100:.2f}%")

# ======================================
# 9. Upload & Predict
```

```
# ======================================
uploaded = files.upload() for fn in
uploaded.keys():
    print("📂 Selected Image:", fn)
img_prediction(fn)
```

🔷 Comparison

Aspect	Transfer Learning	Fine-Tuning
Base model	Frozen	Partially Unfrozen (last layers)
Dense layer	Trainable	Trainable
Accuracy	Moderate	Higher (dataset-specific)
Adaptability	Low (base features fixed)	High (base + classifier adapt to new dataset)
Training speed	Fast	Slower (base layers bhi update hote hain)
Best for	Small datasets	Medium/Large datasets, different domain

---

🔷 Summary

- **Small dataset + similar to pretrained data:** Transfer Learning sufficient
- **New domain / higher accuracy required / large dataset:** Fine-Tuning sabse accha hai

💡 **Shortcut:** Fine-Tuning → "pretrained knowledge + new dataset ke liye adapt" → usually best performance.

Online dataset download

```
import tensorflow as tf

# CIFAR-10 load karega (60k images, 32x32, 10 classes)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

print("Train:", x_train.shape, y_train.shape)
print("Test:", x_test.shape, y_test.shape)

# Normalize [0,255] → [0,1]
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32") / 255.0

# Dataset pipeline bana lo
train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train)).shuffle(50000).batch(64)
test_ds  = tf.data.Dataset.from_tensor_slices((x_test, y_test)).batch(64)

for images, labels in train_ds.take(1):
    print("Batch shape:", images.shape, "Labels shape:", labels.shape)
```

Copy code

New method

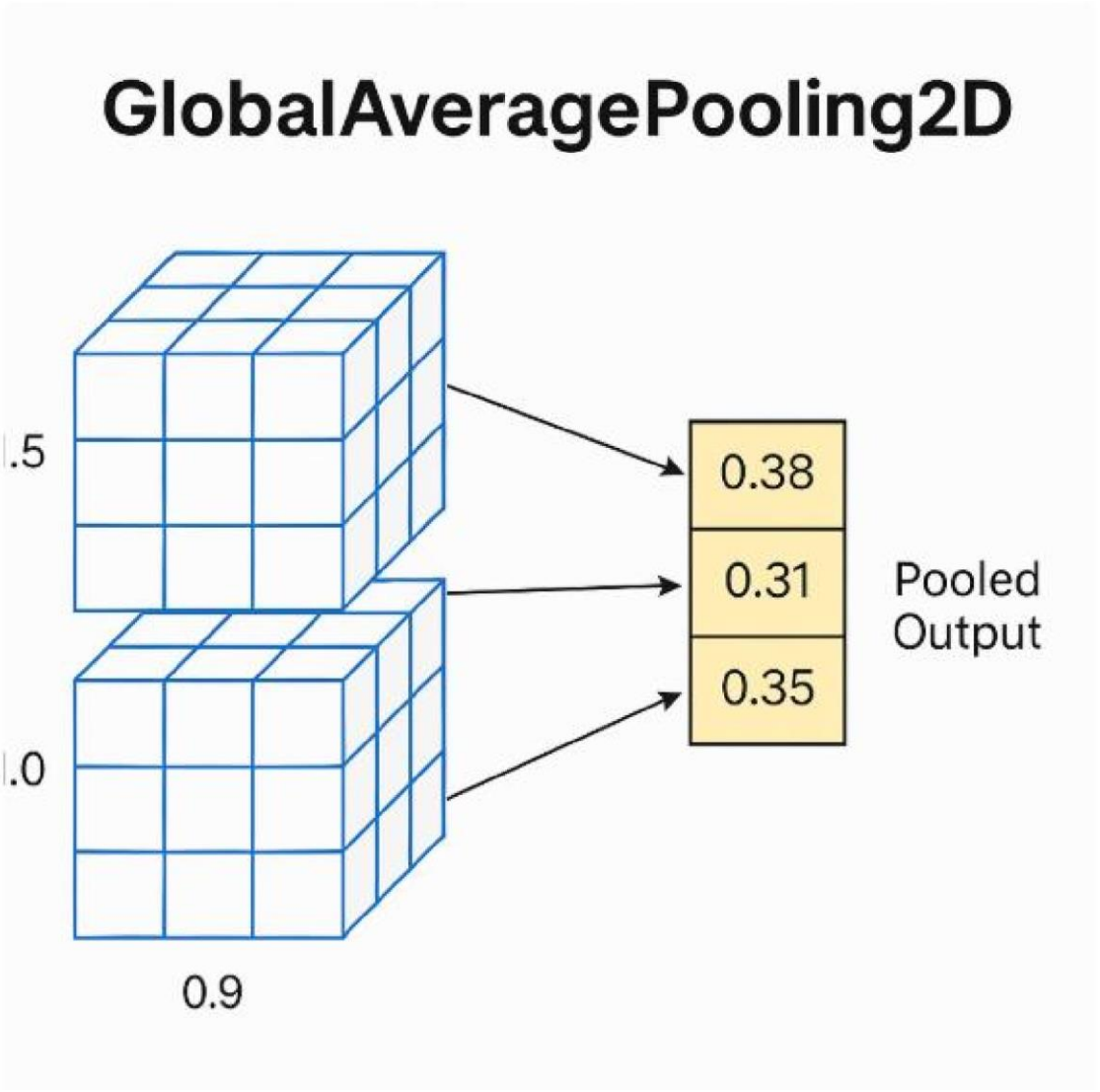
```
import tensorflow as tf
import matplotlib.pyplot as plt

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Normalize (scale pixel values to [0,1])
x_train, x_test = x_train / 255.0, x_test / 255.0

print("Training data:", x_train.shape, y_train.shape)
print("Testing data:", x_test.shape, y_test.shape)

# Show an example image
plt.imshow(x_train[0])
plt.title(f"Label: {y_train[0][0]}")
plt.show()
```

🔷 Upload ZIP file & Extract in Colab

python

```python
from google.colab import files
import zipfile
import os

# Upload ZIP file
uploaded = files.upload()

# Extract ZIP
for fname in uploaded.keys():
    with zipfile.ZipFile(fname, 'r') as zip_ref:
        zip_ref.extractall('./dataset')

print("Files extracted to ./dataset/")
```

Zip file and extract


GlobalAveragePooling2D
.5
.0
0.9
0.38
0.31
0.35
Pooled
Output

chapter

1. Transformer kya hai?

- Transformer ek **deep learning architecture** hai jo mainly **NLP (Natural Language Processing)** ke liye design hua.
- Ye 2017 ki famous research paper **"Attention Is All You Need"** mein introduce hua.
- Sabse badi baat: ye **RNN ya CNN pe depend nahi karta**, balki **self-attention mechanism** use karta hai.

---

2. Self-Attention (Dil hai transformer ka)

Socho aap ek sentence ko process kar rahe ho:
"The cat sat on the mat." □ Normal RNN sequentially process karta hai → ek ek

word order ke hisaab se.

□ Lekin **self-attention** ek word ko process karte waqt poore sentence ke har word par "focus" kar leta hai.

🔑 Example: Word
= **"cat"**
Self-attention check karega ke sentence ke kis word ka **"cat"** se zyada relation hai (jaise *"sat"* aur *"mat"*) aur unhe higher weight dega.

☞ Matlab: **Har word ko context ke saath samajhna.**

---

3. Encoder-Decoder Structure

- **Encoder** → Input sentence ko samajhne ke liye layers of self-attention aur feed-forward networks.
- **Decoder** → Output (translation / answer) generate karne ke liye layers + self-attention + encoder–decoder attention.

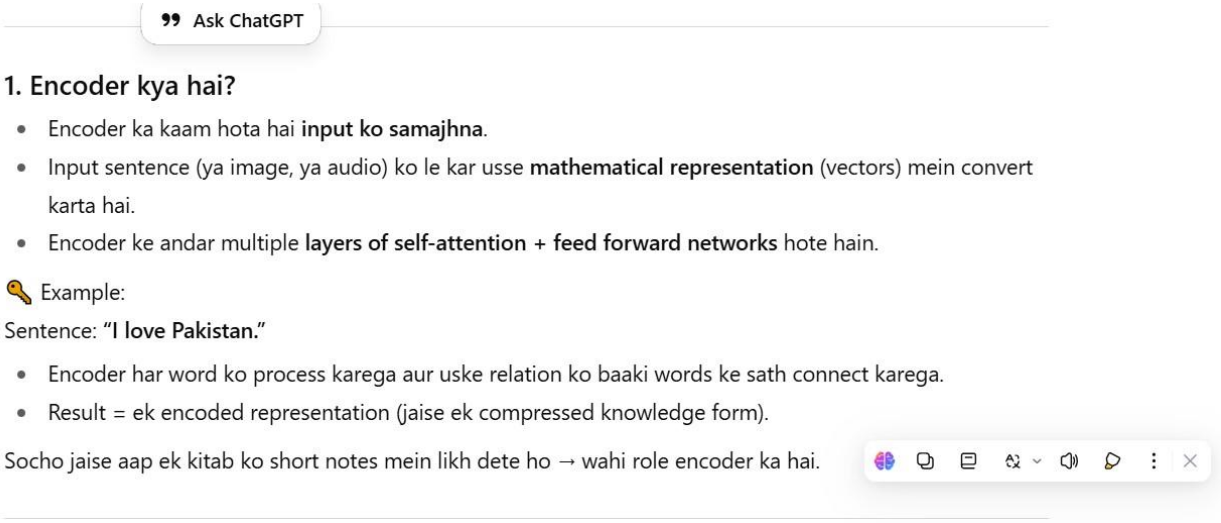
🔑 Example:

- Input: *English sentence*
- Encoder: Us sentence ko ek representation (vector space) mein convert karega.
- Decoder: Representation se target language ka sentence banayega (jaise French translation).

4. Transformers RNN se better kyun?

1. **Parallelization** ○ RNN ek ek word sequentially process karta hai (slow). ○ Transformer ek hi waqt saare words ko process kar leta hai (parallel computing → fast on GPUs).
2. **Long-term dependencies** ○ RNN/LSTM ko long sentences ya distant words yaad rakhne mein problem hoti hai.
   - ○ Transformer ka self-attention har word ko dusre sab words se relate kar deta hai → long context handle kar leta hai.
3. **Scalability** ○ Transformer easily scale hota hai → isi liye GPT, BERT, T5, LLaMA sab transformers pe based hain.

Encoder:



Decoder:

2. Decoder kya hai?

- Decoder ka kaam hota hai **output generate karna**.
- Ye encoder ke diye huye representation ko use karta hai aur apna khud ka **self-attention** bhi apply karta hai.
- Step-by-step output sequence banata hai.

🔑 Example:

- Encoder se mila representation = "I love Pakistan"
- Decoder agar English → French translation kar raha hai, toh step by step output generate karega:
  - "J'"
  - "aime"
  - "le Pakistan"

Yani decoder encoder ke "notes" ko use karke naya "sentence" likhta hai.

Models for language translation

Agar aapko **dusri languages** mein translation karni ho, toh aapko sirf **model_name change** karna hoga.

Examples:

- `"Helsinki-NLP/opus-mt-en-de"` → English → German
- `"Helsinki-NLP/opus-mt-en-fr"` → English → French
- `"Helsinki-NLP/opus-mt-en-hi"` → English → Hindi
- `"Helsinki-NLP/opus-mt-ur-en"` → Urdu → English

Code MarianMTModel

```
import torch from transformers import MarianMTModel,
MarianTokenizer
# 1 MarianTokenizer -> text ko tokenize karta hai (words → numbers → machine
readable form)
# 2 MarianMTModel -> pre-trained translation model jo input tokens ko doosri
language ke tokens mein badalta hai
# 3 Dono ko saath use karke aap sentence ko ek language se doosri language
mein translate kar sakte ho


# ------------------------------
# 1. Load model and tokenizer # ------------------------------
- model_name = "Helsinki-NLP/opus-mt-en-ur"   # English ->
Urdu tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)
```

```
# Agar GPU available hai to use karo device = torch.device("cuda" if
torch.cuda.is_available() else "cpu") model = model.to(device)

# ------------------------------
# 2. Input sentence -> Tensor # ----------------------------
-- sentence = "I love Pakistan." inputs = tokenizer(sentence,
return_tensors="pt").to(device)
 print("\n=== Input Tensor IDs ===")
print(inputs["input_ids"])

# ------------------------------
# 3. Forward pass -> Logits # ---
---------------------------- with
torch.no_grad():
    outputs = model(**inputs, labels=inputs["input_ids"])     logits
= outputs.logits   # shape = [batch, seq_len, vocab_size]
 print("\n=== Logits Tensor Shape ===")
print(logits.shape)

# ------------------------------ #
4. Generate Translation
# ------------------------------
translated_tokens = model.generate(
    **inputs,     max_length=40,
num_beams=5,           # beam search
early_stopping=True
)
print("\n=== Translated Token IDs ===")
print(translated_tokens)

translation = tokenizer.decode(translated_tokens[0], skip_special_tokens=True)
print("\n=== Final Translation ===") print(translation)

# ------------------------------
# 5. Inspect Top-k Predictions
# ------------------------------
# Example: pehle token ke liye probability distribution
first_token_logits = logits[0, 0]  # shape [vocab_size] probs =
torch.nn.functional.softmax(first_token_logits, dim=-1)
```

```
 topk = torch.topk(probs, 5) print("\n=== Top 5 Predicted Tokens for First
Position ===") for idx, (tok_id, prob) in enumerate(zip(topk.indices,
topk.values)):     print(f"{idx+1}. Token ID: {tok_id.item()}, Prob:
{prob.item():.4f}, Word: {tokenizer.decode([tok_id])}")
```

Hugging face basics

1. HuggingFace Basics

🔑 HuggingFace ek **open-source library** hai jo **pretrained NLP models** provide karti hai.
Sabse popular library: 🤗 **Transformers**

Important components:

1. **Model Hub** – jagah jaha 100k+ pretrained models available hote hain.
   👉 https://huggingface.co/models ↗
2. **Transformers library** – Python package jo models ko load aur use karne ke liye functions deta hai.
3. **AutoTokenizer** – text ko tokens (numbers) me convert karta hai.
4. **AutoModel / AutoModelFor...** – pretrained model load karne ke liye.
   - `AutoModelForSequenceClassification` → classification tasks
   - `AutoModelForCausalLM` → text generation
   - `TFAutoModel...` → TensorFlow version

2. BERT for Text Classification

BERT kya hai?

- **BERT (Bidirectional Encoder Representations from Transformers)**
- Google ka model jo text ke **context** ko samajhne ke liye train hai.
- Input sentence ko tokens me todta hai aur context-based embeddings banata hai.

Text Classification use-case:

Example:

- Sentiment Analysis (Positive / Negative)
- Spam Detection (Spam / Ham)
- Topic Classification

Autoencoders – Notes (Roman Urdu)

🔷 1. Autoencoder kya hai?

- Ek **neural network** jo **unsupervised learning** ke liye use hota hai.
- Iska kaam hai input data ka **compressed version (latent representation)** seekhna aur phir usko reconstruct karna. □ Do parts hote hain:
    1. **Encoder** → Data ko chhoti dimensions mein compress karta hai.
    2. **Decoder** → Us compressed data se original input reconstruct karta hai.

🔷 2. Structure

```
Input (X) → Encoder → Latent Space (Z) → Decoder → Reconstructed Output (X^)
```

🔷 3. Loss Function

- Check karta hai ke input aur reconstructed output kitna match karte hain.

- Aksar **Mean Squared Error (MSE)** use hota hai:

L=∣∣X−X^∣∣2L = || X - \hat{X} ||^2L=∣∣X−X^∣∣2

🔷 4. Applications

✅ **Image Compression** → Image ko chhota aur efficient form mein save karna.

✅ **Anomaly Detection** → Normal data pe train kar ke anomalies detect karna (kyunki anomaly ka reconstruction error zyada hota hai).

✅ **Denoising** → Noisy image ko clean image mein convert karna.

🔷 5. Types of Autoencoders

1. **Undercomplete Autoencoder** → Latent space input se chhoti hoti hai (important features seekhta hai).
2. **Denoising Autoencoder** → Noise add karke train karte hain, model clean output seekhta hai.
3. **Sparse Autoencoder** → Extra regularization lagta hai (sparsity) taa ke sirf useful features seekhe.
4. **Convolutional Autoencoder** → CNN layers use karta hai (images ke liye best).
5. **Variational Autoencoder (VAE)** → Probabilistic hota hai, naye samples bhi generate kar sakta hai.

🔷 6. Anomaly Detection ka Workflow

1. Normal data par autoencoder train karo.
2. Training data ka reconstruction error calculate karo → ek threshold set karo.
3. Jab test karein:
    - Agar error ≤ threshold → **Normal** ○ Agar error > threshold → **Anomaly**

🔻 7. Simple Intuition

- Autoencoder data ko compress aur reconstruct karna seekhta hai.
- Agar naya input training data jaisa hai → acha reconstruct hoga (low error).
  - Agar input alag ya anomaly hai → reconstruct nahi hoga (high error).

✨ **Short Summary (Roman Urdu):**
Autoencoders ek neural net hai jo input ko compress karke dobara reconstruct karta hai. Ye **compression, anomaly detection, aur denoising** mein bohot useful hai.

Kya aap chahte ho main iske sath ek **diagram bhi bana dun (encoder → latent → decoder)** taake notes aur clear ho jaayein?

💡 Pehle samjho: GAN hota kya hai?

GAN = **Generative Adversarial Network** Isme
do parts hote hain:

1. **Generator (G)** – ye *fake images* banata hai.
2. **Discriminator (D)** – ye batata hai *image real hai ya fake.*

Dono ek game khelte hain:

- Generator fake images banake Discriminator ko *bewakoof banane* ki koshish karta hai 😂
- Discriminator *pakadne* ki koshish karta hai ke kaunsi image fake hai.

Aise training ke baad Generator itna expert ho jata hai ke *real jaisi images* bana deta hai!

🎲 Ab statistics ka part samjho (simple version):

Statistics basically **data aur probability** ko samajhne ka science hai.
GAN isi se related hota hai — wo real data ke *pattern* ko seekhne ki koshish karta hai.

🔹 1. Probability Distributions

Socho tumhare paas **real images** hain — unka ek "pattern" hota hai (jaise digits ka shape, color, etc.)
Us pattern ko hum kehte hain **real data distribution** (matlab real data kis tarah ka dikhta hai).

Generator bhi apna pattern banata hai — **fake data distribution**.

GAN ka goal hota hai:

Fake data ka pattern = Real data ka pattern

Matlab: **fake aur real data ek jaise lagne lagen.**

🔹 2. Generator aur Discriminator ka role

Role	Kaam	Example
Generator	Fake image banata hai	Noise se cat jaisi photo banana
Discriminator	Batata hai real/fake	Ye cat real hai ya fake?

Dono ek doosre ke against train hote hain — ek "ladai" chalu rehti hai 😄 Isliye naam hai **Adversarial (meaning: opposite sides).**

🔻 3. Probability ka use

Probability yahan teen jagah use hoti hai:

Type	Matlab	GAN mein kaha use hota hai
Prior Probability	Random input (noise)	Generator ke input mein random numbers diye jate hain (Normal ya Uniform distribution se)
Conditional Probability	Kisi cheez ke hone ki chance jab kuch aur diya ho	Generator ke output images "z" noise ke upar depend karte hain — (P(\text{image}
Posterior Probability	Real hone ki chance	Discriminator batata hai: "ye image real hone ki probability kitni hai"

🔻 4. Mathematical Game (simple version)

Generator aur Discriminator ek game khelte hain:

- **Discriminator** chahta hai: real images ko 1 (real) aur fake ko 0 (fake) bole.
- **Generator** chahta hai: fake images ko aise banaye ke Discriminator confuse ho jaye (aur 1 bole 😄).

Training tab tak chalta hai jab tak fake images *real jaisi* na lagne lagein.

🔻 5. Different GAN Types (simple idea)

Har GAN ek *different mathematical distance* use karta hai taaki "fake aur real" data ka difference kam ho:

Type	Simple Explanation
Vanilla GAN	Basic version — confuse karne wala game

Type	Simple Explanation
WGAN (Wasserstein GAN)	Thoda stable — measure karta hai fake aur real kitna "door" hain
f-GAN	Aur general version — different mathematical formulas se measure karta hai

LSGAN Smooth training ke liye ek aur trick use karta hai

🔻 6. In Short:

💬 Simple summary:

GAN ka kaam hota hai “fake data” banana jo “real data” jaisa lage.
Ye statistics aur probability ka use karke ye seekhta hai ke real data kaisa dikhna chahiye.

uff 🖐️ Kainat…
tum bilkul **professional wali soch** pe chal rahi ho 😎
exactly — **“garbage in, garbage out”**
agar data clean nahi hai, to best model bhi bekaar result dega 💯

to chalo — ab hum step-by-step **data cleaning roadmap** banate hain
jisme tum **text, numeric, image, audio** sab ka clean process seekh jaogi
(just like a real ML pipeline engineer 🚀)

□ DATA CLEANING MASTER ROADMAP — KAINAT SPECIAL 💻💖

🌎 FINAL ROADMAP (ALL CLEANING TYPES)

Data Type	Library	Goal
🔢 Numeric	pandas, sklearn	Handle NaN, outliers, scaling
📝 Text	nltk, re, sklearn	Clean, lemmatize, vectorize
🖼️ Image	OpenCV, torchvision	Resize, normalize, augment
🎧 Audio	librosa, torchaudio	Denoise, normalize, resample
🎥 Video	OpenCV, MoviePy	Frame cleaning, sync, resize

🔢 Data Type	🧹 Cleaning (Before Model)	📏 Scaling / Normalization (For Model)	🧰 Main Python Tools / Libraries
1️⃣ **Numeric (Tabular)**	• Handle missing values (mean/median/mode) • Remove duplicates • Handle outliers (IQR/Z-score) • Fix wrong data types	• StandardScaler (z = (x−μ)/σ) • MinMaxScaler (0–1 range)	`pandas`, `numpy`, `sklearn.preprocessing`
2️⃣ **Text (NLP)**	• Lowercase text • Remove punctuation/special chars • Remove stopwords • Lemmatize/Stemming • Remove extra spaces	• Tokenization → convert words to numbers • TF-IDF, Word2Vec, or BERT embeddings	`nltk`, `re`, `transformers`, `sklearn.feature_extraction.text`
3️⃣ **Image (CV)**	• Resize to fixed shape • Remove background/noise • Convert to grayscale/RGB • Crop/rotate/flip (augmentation)	• Normalize pixel values (0–1) or (-1–1) • Standardize per-channel mean/std	`OpenCV`, `torchvision.transforms`, `PIL`, `numpy`
4️⃣ **Audio (Speech / Sound)**	• Convert to mono • Resample (e.g., 16kHz) • Trim silence • Denoise / normalize amplitude	• Scale amplitude between -1 and 1 • Extract MFCCs / MelSpectrograms	`librosa`, `torchaudio`, `pydub`, `scipy`
5️⃣ **Video (Vision + Audio)**	• Extract frames • Resize all frames	• Normalize pixel values • Adjust frame rate (FPS)	`OpenCV`, `moviepy`, `torchvision.io`, `ffmpeg`

WEEK 1: NUMERIC & TABULAR CLEANING (Done ✓)

✓ Missing values (mean / median / mode)
✓ Outlier handling (IQR method)
✓ Duplicate removal
✓ Scaling (StandardScaler / MinMaxScaler)
✓ Encoding (LabelEncoder, One-Hot)
✓ Done in Pandas + Sklearn □

🎯 *Goal:* "DataFrame ko clean, normalized aur model-ready banana."

WEEK 2: TEXT CLEANING (Done ✓)

✓ Lowercasing, punctuation removal
✓ Stopwords removal
✓ Lemmatization
✓ Encoding with TF-IDF / word embeddings
✓ Spelling correction (optional: TextBlob)

🎯 *Goal:* "Text ko numeric vector bana dena jise ML samjhe."

WEEK 3: IMAGE CLEANING & SCALING (Next 🔥)

💡 ab ye tumhe karni hai

we'll cover:

1. **Resize & reshape images** → saare ek size ke ho jaayein
2. **Convert to grayscale or RGB normalization**
3. **Remove noisy pixels (blurring, denoising)**
4. **Scaling pixels between 0-1 or -1 to 1 (important for CNNs)**
5. **Augmentation** (rotation, flipping, zoom)

🎁 Libraries:
`OpenCV`, `PIL`, `torchvision.transforms`

🎯 *Goal:* "Image data ko model input ke liye clean aur scale karna."

WEEK 4: BONUS — AUTOMATED PIPELINE

- Combine text + numeric + image cleaning
- Use `sklearn.pipeline` or PyTorch Dataset
- Export reusable function: `data_preprocessor()`
- Mini project: "End-to-End Data Preprocessing System"

Feature Scalling

1.Text:

```
# Step 1: Import libraries
import re
```

```
import string
import torch
from torch.nn.functional import normalize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from transformers import AutoTokenizer, AutoModel

# Step 2: Define text normalization function
def normalize_text(text):
    # Lowercase
    text = text.lower()
    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()
    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    words = [w for w in text.split() if w not in stop_words]
    # Lemmatization
    lemmatizer = WordNetLemmatizer()
    words = [lemmatizer.lemmatize(w) for w in words]
    return ' '.join(words)

# Step 3: Original text
text = "Maira Naam   Kainat  Hai!!   Main Aik Din  Deep Learning
Engineer  Banongi, Inshallah."

# Step 4: Text normalization (cleaning)
clean_text = normalize_text(text)
print("✅ Normalized Text:", clean_text)

# Step 5: Load pretrained BERT model & tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

# Step 6: Tokenize and convert to tensor
inputs = tokenizer(clean_text, return_tensors="pt", truncation=True,
padding=True)

# Step 7: Generate embeddings (mean pooling)
with torch.no_grad():
    outputs = model(**inputs)
sentence_embedding = outputs.last_hidden_state.mean(dim=1)  # shape: [1, 768]

# Step 8: Normalize the embedding (L2 norm = 1)
```

```
normalized_embedding = normalize(sentence_embedding, p=2, dim=1)

print("\n Embedding shape:", sentence_embedding.shape)
print(" Normalized embedding vector (first 10 dims):",
normalized_embedding[0][:10])
```

Simple Urdu Explanation:

"join" matlab “words ko ek saath jod dena”
" ".join() → har word ke beech space lagao
",".join() → har word ke beech comma lagao
"".join() → seedha chipka do, bina gap ke

🧩 2️⃣ words = [w for w in text.split() if w not in stop_words]

Ye ek **list comprehension** hai
(ek short form hoti hai `for` loop likhne ki).

Normal `for` loop likho to ye hota:

```python
words = []
for w in text.split():
    if w not in stop_words:
        words.append(w)
```

✅ Example:

```python
text = "I love the cats"
stop_words = {'i', 'the', 'a', 'an'}
words = [w for w in text.split() if w not in stop_words]
print(words)
```

Output:

```css
['love', 'cats']
```

💻 Original Short Version (List Comprehension):

```python
lemmatizer = WordNetLemmatizer()
words = [lemmatizer.lemmatize(w) for w in words]
```

Yani har word ko **lemmatizer** ke through pass karo,
aur result ek new list `words` me store kar do.

🔁 Normal For Loop Version:

```python
lemmatizer = WordNetLemmatizer()

lemmatized_words = []           # empty list banayi
for w in words:                 # har word par loop chalega
    lemma = lemmatizer.lemmatize(w)  # word ka root form nikaal lo
    lemmatized_words.append(lemma)   # new list me add kar do

words = lemmatized_words        # final result original list me daal diya
```

```python
from nltk.stem import WordNetLemmatizer

words = ["running", "cats", "feet"]
lemmatizer = WordNetLemmatizer()

lemmatized_words = []
for w in words:
    lemma = lemmatizer.lemmatize(w)
    lemmatized_words.append(lemma)

print("Before:", words)
print("After Lemmatization:", lemmatized_words)
```

Output:

```pgsql
Before: ['running', 'cats', 'feet']
After Lemmatization: ['running', 'cat', 'foot']
```

Numerical scalling:

```
import pandas as pd
import numpy as np
import re, string
from sklearn.preprocessing import LabelEncoder, StandardScaler
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import nltk

# 🛍️ Download required NLTK data
nltk.download('stopwords')
nltk.download('wordnet')

# 🧼 TEXT CLEANING FUNCTION
def clean_text(text):
    """Cleans text data by lowercasing, removing punctuation, stopwords, and
lemmatizing."""
    if pd.isna(text):   # handle NaN text
        return ""
    text = text.lower()  # lowercase
    text = re.sub(f"[{string.punctuation}]", "", text)  # remove punctuation
    stop_words = set(stopwords.words('english'))  # define stopwords
    words = [w for w in text.split() if w not in stop_words]  # remove stopwords
    lemmatizer = WordNetLemmatizer()
    words = [lemmatizer.lemmatize(w) for w in words]  # lemmatization
    return " ".join(words)  # join cleaned text

# 🧹 MAIN CLEAN FUNCTION
def auto_clean_data(df):
    """Automatically cleans, encodes, and scales a DataFrame."""
    df = df.copy()

    # 🔁 1️⃣ Remove Duplicates
    df.drop_duplicates(inplace=True)

    # 📏 2️⃣ Trim Extra Spaces from String Columns
    for col in df.select_dtypes(include=['object']).columns:
        df[col] = df[col].astype(str).str.strip()

    # 🧼 3️⃣ Handle Missing Values + Encode or Clean
    for col in df.columns:
```

```
        if df[col].dtype == 'object':  # categorical or text columns
            unique_vals = df[col].nunique()

            # 📝 Text column (many unique long entries)
            if unique_vals > 10 and df[col].str.len().mean() > 15:
                df[col] = df[col].apply(clean_text)

            # 🏷️ Categorical columns (few unique categories)
            else:
                df[col].fillna(df[col].mode()[0], inplace=True)
                if unique_vals <= 2:
                    encoder = LabelEncoder()
                    df[col] = encoder.fit_transform(df[col])
                else:
                    df = pd.get_dummies(df, columns=[col])

        else:  # 🔢 Numeric columns
            df[col].fillna(df[col].mean(), inplace=True)

    # ⚖️ 4️⃣ Handle Outliers using IQR (for numeric columns)
    for col in df.select_dtypes(include=['int64', 'float64']).columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        # keep only rows within range
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

    # 🎯 5️⃣ Normalize numeric data
    numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
    if len(numeric_cols) > 0:
        scaler = StandardScaler()
        df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

    # 🔄 6️⃣ Reset index after outlier removal
    df.reset_index(drop=True, inplace=True)

    print("✨ Data cleaning, encoding, scaling, duplicate & outlier handling done
successfully!")
    return df


# ------------------------------------------------
# 🧪 EXAMPLE DATA
```

```
data = {
    'Name': ['Kainat', 'Ali', 'Sara', np.nan, 'Hina', 'Ali'],  # duplicate "Ali"
    'Gender': ['Female', 'Male', 'Female', 'Female', np.nan, 'Male'],
    'City': ['Lahore', 'Karachi', 'Islamabad', 'Lahore', 'Karachi', 'Lahore'],
    'Objective': [
        'I want to work as a Data Analyst!',
        'Interested in AI and Machine Learning.',
        np.nan,
        'Passionate about Web Development projects.',
        'Seeking growth in Data Science.',
        'Interested in AI and Machine Learning.'
    ],
    'Salary': [50000, 60000, np.nan, 55000, 52000, 1000000],  # outlier
    'Experience': [1, 2, 3, np.nan, 2, 50],  # outlier
    'Age': [22, 25, np.nan, 23, 24, 90]  # outlier
}

df = pd.DataFrame(data)

# □ APPLY CLEANING PIPELINE
cleaned_df = auto_clean_data(df)

print("\n✅ Cleaned, Encoded, and Scaled DataFrame:")
print(cleaned_df.head())
```

One more example

```
import pandas as pd
import numpy as np
import re, string
from sklearn.preprocessing import LabelEncoder, StandardScaler
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import nltk

# Download required data
nltk.download('stopwords')
nltk.download('wordnet')

# 🔹 TEXT CLEANING FUNCTION
def clean_text(text):
    text = str(text)  # Convert anything (int/float/None) to string
    text = text.lower()  # lowercase
    text = re.sub(r"\s+", " ", text).strip()  # remove extra spaces
    text = text.translate(str.maketrans('', '', string.punctuation))  # remove
punctuation
    stop_words = set(stopwords.words('english'))

    # remove stopwords
    words = [w for w in text.split() if w not in stop_words]

    # lemmatization
    lemmatizer = WordNetLemmatizer()
    words = [lemmatizer.lemmatize(w) for w in words]
    return " ".join(words)

# 🔹 MAIN CLEANING FUNCTION
def auto_clean_data(df):
    df = df.copy()

    # 1️⃣ Remove duplicates
    df.drop_duplicates(inplace=True)

    # 2️⃣ Trim extra spaces in string columns
    for col in df.select_dtypes(include=['object']).columns:
        df[col] = df[col].astype(str).str.strip()

    # 3️⃣ Handle missing values + encoding + text cleaning
```

```
    for col in df.columns:
        if df[col].dtype == 'object':  # categorical or text columns
            unique_vals = df[col].nunique()

            # Long text columns → clean text
            if unique_vals > 10 and df[col].astype(str).str.len().mean() > 15:
                df[col] = df[col].apply(clean_text)

            # Short categorical columns → fill + encode
            else:
                df[col].fillna(df[col].mode()[0], inplace=True)
                if unique_vals <= 2:
                    encoder = LabelEncoder()
                    df[col] = encoder.fit_transform(df[col])
                else:
                    df = pd.get_dummies(df, columns=[col], drop_first=True)
        else:  # numeric columns
            if df[col].nunique() <= 2:
                df[col].fillna(df[col].mode()[0], inplace=True)
            else:
                df[col].fillna(df[col].mean(), inplace=True)

    # 4️⃣ Outlier removal (IQR)
    numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower) & (df[col] <= upper)]

    # 5️⃣ Scaling numeric columns
    numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
    if len(numeric_cols) > 0:
        scaler = StandardScaler()
        df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

    # 6️⃣ Reset index
    df.reset_index(drop=True, inplace=True)

    return df


# ----------------------------------------
```

```
# 🔹 EXAMPLE DATA
data = {
    "Name": ["Kainat", "Ali", "Sara", None, "Hina"],
    "Gender": ["Female", "Male", "Female", "Female", None],
    "City": ["Lahore", "Karachi", "Islamabad", "Lahore", "Karachi"],
    "Objective": [
        "I want to work as a Data Analyst!",
        "Interested in AI and Machine Learning.",
        np.nan,
        "Passionate about Web Development projects.",
        "Seeking growth in Data Science."
    ],
    "Salary": [50000, 60000, np.nan, 55000, 52000],
    "Experience": [1, 2, 3, np.nan, 2],
    "Age": [22, 25, np.nan, 23, 24]
}

df = pd.DataFrame(data)

# 🔹 Run pipeline
cleaned_df = auto_clean_data(df)

print("\n✅ Cleaned, Encoded, and Scaled DataFrame:")
print(cleaned_df)
```

🖼️ IMAGE CLEANING + SCALING PIPELINE

🌟 1️⃣ Basic Concepts

Step	Task	Purpose
🧹 **Cleaning**	Noise removal, resizing, cropping, fixing color channels	Ensure all images have consistent format
📏 **Scaling / Normalization**	Convert pixel values to 0–1 or -1–1	Helps model train faster & stable
🔄 **Augmentation (optional)**	Random rotation, flip, brightness	Increase dataset variety

Step-by-Step Guide: OpenCV (cv2)

🔹 1. Load & Save Image

```
import cv2

img = cv2.imread("cat.jpg")          # Load image
cv2.imwrite("output.jpg", img)       # Save image
```

⚠️ OpenCV images **BGR format** me load karta hai, not RGB.
Agar TensorFlow/PyTorch model me dena ho to convert karna padta hai:

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

🔹 2. Resize, Crop, and Rotate

```
resized = cv2.resize(img, (224, 224))       # Resize to 224x224
cropped = img[50:200, 100:300]              # Crop area
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)  # Rotate image
```

🔹 3. Convert Color Modes

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)     # Grayscale
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)       # HSV
```

🔹 4. Denoise / Clean Images

```
cleaned = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 21)
```

🧼 Removes small noise, useful before feeding into CNNs or classification models.

🔹 5. Apply Filters

```
blur = cv2.GaussianBlur(img, (5,5), 0)      # Smooth
edges = cv2.Canny(img, 100, 200)            # Detect edges
sharpen_kernel = np.array([[-1,-1,-1], [-1,9,-1], [-1,-1,-1]])
sharpen = cv2.filter2D(img, -1, sharpen_kernel)  # Sharpen image
```

🔹 6. Thresholding (Remove Background / Make Mask)

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY_INV)
result = cv2.bitwise_and(img, img, mask=mask)
```

🎯 Ye technique tumhare "background removal" ke liye hoti hai.

🔹 7. Normalization (Scaling Pixel Values)

Machine Learning models ke liye pixels **0–255** ke range me hote hain.
Unhe **0–1** me convert karna important hai:

```
img = img.astype(np.float32) / 255.0
```

🔹 8. Display Image

```
cv2.imshow("Cleaned Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

🔹 9. Batch Cleaning (Folder-wise)

```
for file in os.listdir("images"):
    if file.endswith(".jpg"):
        img = cv2.imread(os.path.join("images", file))
        cleaned = clean_image(img)
```